

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Arhitectură Agentică pentru Asistență Inteligentă în
Dezvoltarea Software**

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației***

Conducător științific
Ș.L. dr. ing. Mihai DOGARIU

Absolvent
Eduard-Daniel NĂNESCU

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **Eduard-Daniel NĂNESCU, 441A**

1. Titlul temei: Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software

2. Descrierea temei:

Contribuția originală a lucrării va consta în dezvoltarea unei arhitecturi software bazată pe Modele Mari de Limbaj (LLMs - Large Language Models), capabilă să asiste utilizatorul în scrierea, depanarea, documentarea și explicarea codului sursă. Aplicațiile vizate sunt eficientizarea procesului de dezvoltare software și automatizarea sarcinilor repetitive întâlnite în programare. Sistemul propus nu vizează antrenarea de la zero a modelelor, ci implementarea unei soluții agentice care utilizează modele pre-antrenate pentru a executa acțiuni complexe. Cercetarea aferentă va include: sinteza bibliografică a realizărilor actuale din domeniu, utilizarea unui mecanism care permite modelului să utilizeze instrumente pentru a interacționa cu mediul virtual, implementarea unor tehnici de inginerie a prompturilor și gestionarea contextului pentru a asigura relevanța răspunsurilor, validarea experimentală a prototipului dezvoltat prin scenarii de utilizare reale utilizând medii de dezvoltare precum PyCharm [1]/VS Code [2] și biblioteci specifice precum LangChain [3], LangGraph [4] sau GoogleADK [5], concluzionarea rezultatelor obținute și a contribuției originale, cât și enunțarea perspectivelor ulterioare de cercetare.

3. Contribuția originală:

Proiectarea și implementarea arhitecturii agentului, incluzând logica de selecție și utilizare a instrumentelor, gestionarea contextului și dezvoltarea unei interfețe sau integrarea cu interfețe existente care să permită interacțiunea facilă cu asistentul.

4. Resurse și bibliografie:

- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. "React: Synergizing reasoning and acting in language models." In The eleventh international conference on learning representations. 2022.
- Bozkurt, Aras, and Ramesh C. Sharma. "Generative AI and prompt engineering: The art of whispering to let the genie out of the algorithmic world." Asian Journal of Distance Education 18, no. 2 (2023): i-vii.
- Alto, Valentina. Building LLM Powered Applications: Create intelligent apps and agents with large language models. Packt Publishing Ltd, 2024.
- Zhu, He, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao et al. "Oagents: An empirical study of building effective agents." arXiv preprint arXiv:2506.15741 (2025).

5. Data înregistrării temei: 2025-12-11 22:54:00

Conducător(i) lucrare,
Ș.L. dr. ing. Mihai DOGARIU

Student,
Eduard-Daniel NĂNESCU

Director departament,
Conf. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **621d9dfde3**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Arhitectură Agentică pentru Asistență Inteligentă în Dezvoltarea Software*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Eduard-Daniel NĂNESCU

.....

Cuprins

LISTA FIGURILOR	10
LISTA TABELELOR	12
LISTA DE ACRONIME	13
INTRODUCERE	14
CONTEXT ȘI MOTIVAȚIE	14
SCOPUL ȘI OBIECTIVELE LUCRĂRII	14
METODOLOGIE	15
CONTRIBUȚII PROPRII ȘI REZULTATE	15
STRUCTURA LUCRĂRII	16
1. CONTEXT ȘI POZIȚIONARE	17
1.1. Evoluția către modelele mari de limbaj (Large Language Model (LLM))	17
1.2. Maturizarea modelelor mari de limbaj și cercetări actuale	18
1.3. Soluții agentice pentru industria dezvoltării software	20
1.3.1. Analiza preliminară a soluțiilor existente pe piață	23
1.4. Cerințe pentru o soluție agentică în dezvoltarea software	25
1.5. Limitări ale arhitecturilor agentice	26
1.6. Justificarea alegerilor de implementare	26
1.7. Plasarea în industrie	28
1.7.1. Soluții proprietare	28
1.7.2. Soluții open-source	30
1.7.3. Flux agentic ce rulează local	31
1.7.4. Observabilitate	31
1.7.5. Securitate și confidențialitate	31
1.7.6. Cost	31
1.8. Analiză proprie și decizii arhitecturale	31
1.8.1. Arhitectură modulară	32
1.8.2. Interfața cu utilizatorul	32
1.8.3. Tehnici proprii de gestionare a contextului și a apelurilor la unelte	32
1.8.4. Suport pentru date multi-modale	32
1.8.5. Observabilitate pe două straturi	33
1.8.6. Cercetarea tehnicilor de utilizare optime pentru arhitecturi locale în dezvoltarea software	33
1.8.7. Sinteza recomandărilor extrase din experimentare	33
2. IMPLEMENTARE	34
2.1. Mediul de dezvoltare	34
2.2. Structura	34
2.3. Configurare	36

2.4.	Interfața cu utilizatorul	36
2.5.	Funcționalități	38
2.6.	Instalare și utilizare	39
2.6.1.	Cerințe preliminare	39
2.6.2.	Instalare	39
2.6.3.	Configurare la prima rulare	39
2.6.4.	Utilizare	40
2.6.5.	Modul headless	40
2.7.	Gestionarea contextului și a memoriei	41
2.7.1.	Memoria și istoricul conversației	41
2.7.2.	Contextul	41
2.7.3.	Retrieval Augmented Generation (RAG)	43
2.7.4.	Gestionarea datelor multi-modale	43
2.7.5.	Compactarea	44
2.7.6.	Tehnici pentru eficientizarea ferestrei de context	45
2.8.	Componente ale arhitecturilor agentice moderne	45
2.8.1.	Unelte	46
2.8.2.	Modul de planificare	48
2.8.3.	Skill-uri	48
2.8.4.	Subagenți	49
2.9.	Extensibilitate	50
2.9.1.	Model Context Protocol (MCP) și adăugarea de unelte	50
2.9.2.	Adăugarea de subagenți	51
2.9.3.	Creare de skill-uri și personalizare	52
2.9.4.	Adăugarea tehnicilor de compactare	52
2.10.	Securitatea în mediul agentic	52
2.10.1.	Sandboxing	53
2.10.2.	Permisuni	53
2.10.3.	Guardrails, intenții periculoase și cereri în afara scopului (Out of Scope (OOS))	54
2.11.	Observabilitate	54
2.11.1.	Langfuse	54
2.11.2.	Panou web	55
3.	REZULTATE EXPERIMENTALE	58
3.1.	Poziționarea față de concurența	58
3.1.1.	Funcționalități prezente în concordanță cu aplicațiile concurente propri- etare și Open Source Software (OSS):	60
3.1.2.	Funcționalități absente față de aplicațiile concurente:	61
3.2.	Influența lungimii și a exactității prompt-ului asupra performanței. De ce un prompt prea exact nu garantează rezultate mai bune	62
3.3.	Contextul în mediul agentic și compactare. Consecințele metodelor de compactare	65
3.3.1.	Metode care taie date din context	66
3.3.2.	Metode care parafrazează date din context	66
3.3.3.	Metode auxiliare	66
3.3.4.	Secvența de metode de compactare recomandată	66

3.4. Interacțiunea dintre modele, context și unelte. Modele optime pentru sarcini de programare	67
3.4.1. Rezultate	68
3.4.2. Analiza rezultatelor	68
3.4.3. Modelele open-source: evaluarea valorii și a potențialului de utilizare . .	71
4. DEZVOLTĂRI VIITOARE	72
4.1. Automatizări, integrări cu alte programe	72
4.2. Integrarea cu mediile de dezvoltare	72
4.3. Suport nativ pentru Windows	72
4.4. Construirea unui graf de cunoștințe pentru optimizarea înțelegerii proiectelor . .	72
4.5. Suport pentru metode de compactare și sandboxing avansate și Software Development Kit (SDK) pentru agenți	73
CONCLUZII	74
BIBLIOGRAFIE	75
ANEXE	80
Anexa 1: config/config.yml	80

LISTA FIGURILOR

1.1.	Arhitectura transformer [6]	17
1.2.	Evoluția recentă a modelelor mari de limbaj [7]	18
1.3.	Performanta arhitecturii Longformer: în timp ce atenția standard scalează exponențial, Longformer scalează liniar cu lungimea contextului [8]	18
1.4.	Diferite configurații de atenție folosite în arhitectura Longformer în comparație cu mecanismul obișnuit de atenție [8]	18
1.5.	Fine-tuning-ul se realizează doar pe A și B [9]	19
1.6.	Rezultatele procesului de Simple Self Distillation [10]	19
1.7.	Ilustrarea procedurii de Reinforcement Learning (RL) [11]	20
1.8.	Exemple de Chain-of-Thought (CoT) [12]	20
1.9.	Paradigma ReAct [13]	21
1.10.	Îmbunătățirile pe care le aduce ReAct [13]	21
1.11.	Îmbunătățirea adusa de ReWOO: worker-ul are mai multă libertate, putând apela mai multe unelte până să producă un răspuns. P = Plan, E = Evidence, A = Answer, O = Observation, A = Act, T = Thought [14]	22
1.12.	Modele tind să aibă o acuratețe foarte slaba contra întrebărilor ce vizează informații care se afla la mijlocul ferestrei curente de context [15]	22
1.13.	Vizualizarea problemei și al rezultatului dorit la întrebare. De multe ori, modelele întâmpina dificultăți în accesarea acestor informații ascunse în mijlocul ferestrei de context [15]	23
1.14.	Clasamentul modelelor în SWE-Bench Pro [16] la momentul actual în funcție de performanța rezolvării sarcinilor complexe în limba engleza	25
1.15.	Clasamentul modelelor în SWE-Bench Pro [16] la momentul actual dupa de cost	25
1.16.	Reprezentarea ferestrei de context într-o conversație uzuala (Sursa: [17])	26
1.17.	Reprezentarea ferestrei de context într-o conversație în Claude Code	31
1.18.	Reprezentarea utilizării în Claude (varianta web) (Sursa: [18])	31
2.1.	Arhitectura modulară a aplicației <code>vibe-cli</code>	35
2.2.	Structura directoarelor vitale ale proiectului	36
2.3.	Prima versiune a interfeței, realizata cu ajutorul bibliotecii Textual	37
2.4.	Continuarea conversației cu prima versiune a interfeței, realizata cu ajutorul bibliotecii Textual	37
2.5.	A doua versiune a interfeței, realizata fără biblioteca Textual	37
2.6.	Continuarea conversației cu a doua versiune a interfeței, realizata fără biblioteca Textual	37
2.7.	Exemplu de conversație in interfața <code>vibe-cli</code>	38
2.9.	Împărțirea ferestrei de context (Sursa: [19])	42
2.10.	Fluxul de prelucrare a datelor multi-modale în <code>vibe-cli</code>	44
2.11.	Schema componentelor care alcătuiesc fereastra de context si compactarea la prag	44
2.12.	Compactarea dupa apelul uneltei	45
2.13.	Implementarea unei funcții pe care agentul o folosește pentru a lista directoriile și fișierele la o anumita cale pe sistem	46

2.14. Suita de unelte expuse agentului în vibe-cli și relațiile dintre acestea	47
2.15. Implementarea unei capabilitati de revizuire de Pull Request (PR)	49
2.16. Arhitectura orchestrator-subagenți din vibe-cli	49
2.17. Implementarea unei funcții pe care agentul o folosește pentru a porni un subagent	50
2.18. Înregistrarea unui nou subagent	51
2.19. Cererea de aprobare a permisiunii de a scrie un fișier	53
2.20. Interfața Langfuse	55
2.21. Imagini cu interfața web a aplicației	56
2.22. Detalii ale panoului web: apelarea uneltelor, și reconstrucția interacțiunilor . . .	56
2.23. Detalii ale panoului web: evoluția contextului si costul estimat	57
3.1. Comparație vizuală între prompt-ul detaliat și cel simplu	65
3.2. Număr de parametri ai modelelor locale	69
3.4. Eficiența cost-calitate: scorul mediu obținut per dolar cheltuit (scară logaritmică). Modelele locale Ollama nu sunt incluse deoarece au cost zero.	70

LISTA TABELELOR

1.1.	Comparația câtorva din modelele existente și a capacitățile acestora în comparație cu prețul de utilizare	24
1.2.	Comparația cadrelor de lucru pentru întrebuințări agentice	27
1.3.	Comparație biblioteci pentru interfața cu utilizatorul	28
1.4.	Produse Inteligență Artificială (IA) ale principalilor furnizori de modele de limbaj	29
1.5.	Top 6 asistenți open source după numărul de "stele" pe GitHub	30
2.1.	Hardware recomandat pentru modele open-source	34
2.2.	Comenzi disponibile în interfață	40
3.1.	Comparație între soluția dezvoltată și principalele unelte agentice de programare pentru terminal disponibile pe piață	59
3.2.	Comparație între soluția propusă și principalii asistenți de programare Open Source Software (OSS) din ecosistem	60
3.3.	Rezultatele pentru prompt-ul detaliat: metrici per proiect și pe nivele de severitate	63
3.4.	Rezultatele pentru prompt-ul simplu: metrici per proiect și pe nivele de severitate	64
3.5.	Modelele testate: tip, număr de parametri, mărime pe disc și suport multi-modal.	67
3.6.	Clasamentul modelelor după scor mediu, durata totală de rulare și cost. Modelele rulate local prin ollama au cost zero. Modelele proprietare nu au datele referitoare la mărimea modelului publice	68

LISTA DE ACRONIME

AML	Adversarial Machine Learning (Învățare Automată Adversarială)	MoE	Mixture of Experts (Amestec de Experti)
API	Application Programming Interface (Interfață de Programare a Aplicațiilor)	NLP	Natural Language Processing (Procesarea Limbajului Natural)
BLEU	Bilingual Evaluation Understudy (Substituent de Evaluare Bilingvă)	OOS	Out of Scope (În Afara Domeniului de Aplicabilitate)
BTT	Backpropagation Through Time (Retropropagare prin Timp)	OSS	Open Source Software (Software cu Sursă Deschisă)
CD	Continuous Deployment (Implementare Continuă)	PDF	Portable Document Format (Format Portabil de Document)
CI	Continuous Integration (Integrare Continuă)	PR	Pull Request (Cerere de Integrare)
CLI	Command Line Interface (Interfață în Linie de Comandă)	RAG	Retrieval Augmented Generation (Generare Augmentată prin Recuperare)
CNN	Convolutional Neural Network (Rețea Neuronală Convoluțională)	RAM	Random Access Memory (Memorie cu Acces Aleatoriu)
CoT	Chain-of-Thought (Lanț de Gândire)	RL	Reinforcement Learning (Învățare prin Recompensă)
CPU	Central Processing Unit (Unitate Centrală de Procesare)	RNN	Recurrent Neural Network (Rețea Neuronală Recurentă)
E2E	End to End (De la Capăt la Capăt)	ROUGE	Recall-Oriented Understudy for Gisting Evaluation (Substituent Orientat spre Recuperare pentru Evaluarea Rezumării)
FN	False Negative (Negativ Fals)	SDD	Spec-Driven Development (Dezvoltare Ghidată de Specificații)
FP	False Positive (Pozitiv Fals)	SDK	Software Development Kit (Kit de Dezvoltare Software)
GPT	Generative Pre-trained Transformer (Transformator Pre-trenat Generativ)	SHA	Secure Hash Algorithms (Algoritmi de Hash Securizat)
GPU	Graphics Processing Unit (Unitate de Procesare Grafică)	SSE	Server-Side Events (Evenimente pe Partea Serverului)
GUI	Graphical User Interface (Interfață Grafică cu Utilizatorul)	STDIO	Standard Input-Output (Intrare-Ieșire Standard)
HTML	Hyper-Text Markup Language (Limbaj de Marcare Hipertext)	STDOUT	Standard Output (Ieșire Standard)
HTTP	Hyper-Text Transfer Protocol (Protocol de Transfer Hipertext)	TDD	Test-Driven Development (Dezvoltare Ghidată de Teste)
IA	Inteligență Artificială	TN	True Negative (Negativ Adevărat)
IDE	Integrated Development Environment (Mediu de Dezvoltare Integrat)	ToT	Tree-of-Thought (Arbore de Gândire)
JSON	Java Script Object Notation (Notăție de Obiecte JavaScript)	TP	True Positive (Pozitiv Adevărat)
KV	Key-Value (Cheie-Valoare)	TUI	Terminal User Interface (Interfață de Utilizator în Terminal)
LLM	Large Language Model (Model de Limbaj de Dimensiuni Mari)	URL	Uniform Resource Locator (Localizator Uniform de Resurse)
LSTM	Long Short Term Memory (Memorie pe Termen Lung-Scurt)	UUID	Universally Unique Identifier (Identificator Universal Unic)
MCP	Model Context Protocol (Protocol de Context al Modelului)	VRAM	Video Random Access Memory (Memorie Video cu Acces Aleatoriu)
		YAML	YAML Ain't Markup Language (YAML Nu Este un Limbaj de Marcare)

INTRODUCERE

CONTEXT ȘI MOTIVAȚIE

Motivația acestei lucrări izvorăște din experiența personală în dezvoltarea software. Atribuțiile unui inginer sunt numeroase și diverse: pe lângă scrierea codului, dezvoltatorul investește timp în planificare, documentație, revizuire, teste unitare, de integrare și End to End (E2E), instrumentare și monitorizare, procese de Continuous Integration (CI)/Continuous Deployment (CD), administrarea infrastructurii și, când apar probleme, depanare și analiza cauzelor principale. Concomitent trebuie să rămână la curent cu cele mai noi practici, securitate cibernetică, rețelistică, precum și să coopereze cu alți colegi.

Evoluția recentă a sistemelor bazate pe Inteligență Artificială (IA) ce folosesc un Large Language Model (LLM) a deschis calea către un proces în care multe sarcini sunt delegate sistemelor automate, atenția inginerului mutându-se de la scrierea de cod la supravegherea acestor instrumente și asumarea deciziilor arhitecturale. Trecerea de la o experiență pur conversațională la integrarea cu mediile de dezvoltare și terminalul sistemului de operare a fost facilitată de competențele avansate de prelucrare și generare de text ale modelelor.

În prezent, câțiva furnizori mari (Google [20], Anthropic [21], OpenAI [22]) controlează o bună parte din cota de piață, fortificând ecosistemul prin produse proprii (asistenți de cercetare, programare, automatizări) și restricționând sau scumpind accesul în afara acestuia. Există modele de tip Open Source Software (OSS), dar ecosistemul uneltelor gratuite construite peste ele este încă în dezvoltare ceea ce creează un mediu propice pentru un asistent independent de furnizor, optimizat pentru costuri reduse, ce poate rula local, precum cel propus în această lucrare.

SCOPUL ȘI OBIECTIVELE LUCRĂRII

Scopul este realizarea unui **asistent inteligent extensibil și transparent** bazat pe IA, **agnostic față de furnizorii de modele de limbaj**, cu modele ce rulează local sau în cloud și care poate interacționa cu mediul virtual (sistem de operare, mediu de dezvoltare, internet) prin tool-uri¹, realizat de un flux de lucru agentic².

Obiectivul principal este un produs funcțional ce poate ajuta un inginer software, atingând următoarele necesități: interfață versatilă prin terminal (Terminal User Interface (TUI)), set de unelte pentru sarcini de complexitate ridicată, gestionarea conversației, contextului și memoriei, observabilitate (metrici, costuri) și siguranță prin medii sigure de execuție.

¹Tool-urile sunt funcții cu descrieri detaliate pe care agentul le populează cu parametri, le execută și recepționează răspunsul.

²Agenții sunt aplicații ce folosesc IA pentru a interacționa cu mediul înconjurător.

METODOLOGIE

Realizarea lucrării a urmat o metodologie iterativa în patru etape:

1. studiul literaturii și al uneltelor concurente (Claude Code, Codex Command Line Interface (CLI), Copilot CLI, Aider), din care au rezultat cerințele și justificarea alegerilor tehnologice;
2. definirea modulelor și interfețelor: graf de execuție, sistem de unelte, proces de compactare, subagenți, sandboxing, permisiuni, observabilitate, privilegiind agnosticismul față de furnizor și extensibilitatea prin fișierele de configurare YAML Ain't Markup Language (YAML);
3. dezvoltarea incrementală a aplicației `vibe-cli` în Python folosind LangGraph, `prompt_toolkit`/Rich pentru TUI și Flask pentru panoul web local;
4. validarea prin trei protocoale experimentale: evaluare de revizui de Pull Request (PR) pe 50 de revizui umane din 6 proiecte OSS, evaluarea stadiilor procesului de compactare și o comparație a 16 modele de limbaj pe 6 sarcini de programare.

CONTRIBUȚII PROPRII ȘI REZULTATE

Contribuția principală a lucrării o reprezintă proiectarea și implementarea aplicației `vibe-cli`, un asistent inteligent de programare, agnostic față de furnizorul de modele de limbaj, transparent și extensibil. Pe lângă produsul în sine, contribuțiile originale se pot grupa astfel:

- **proces configurabil de compactare a contextului**, structurat în stadii independente (deduplicare, trunchiere a ieșirilor de unelte, sumarizare, substituie prin referințe, stergere bazată pe importanță), configurabil integral prin YAML și extensibil prin subclasare;
- **sistem de filtre deterministe peste apelurile de unelte**, ce impune protocolul *read-before-write* prin verificarea unui hash Secure Hash Algorithms (SHA) al fișierului înainte de modificare, prevenind pierderea de modificări concurente și conflictele cu starea reală a sistemului de fișiere;
- **mecanism de detectare automată a capabilităților multi-modale** ale modelelor, necesar pentru menținerea agnosticismului față de furnizor;
- **arhitectura echipelor de agenți și skill-uri** (cercetare web, căutare în cod, sarcini generale) cu compactare proprie, configurabili în întregime prin YAML;
- **panou web local de observabilitate**, ce expune utilizatorului metrice detaliate despre consumul de tokeni, costuri, latente și comportamentul agentului — informații frecvent ascunse în produsele comerciale studiate; interfața cu utilizatorul;
- **trei protocoale experimentale proprii** pentru validarea cantitativă a soluției și extragerea unor recomandări transferabile la ecosistemul mai larg de unelte similare.

Rezultatele obținute confirmă atingerea obiectivelor propuse. Aplicația finală oferă un set complet de capabilități agentice (sandboxing pe trei niveluri, permisiuni granulare, suport Model Context Protocol (MCP) pe toate protocoalele de transport uzuale, intrări multi-modale, subagenți și proces de compactare), comparabil din punct de vedere funcțional cu uneltele comerciale dominante, dar diferențiat prin transparenta, configurabilitate și absența dependenței

de un furnizor unic. Evaluarea experimentală a arătat că pe testele de evaluare proprii de revizuire de PR soluția livrează rezultate comparabile cu cele ale concurenței, că procesul de compactare poate reduce semnificativ consumul de tokeni fără degradarea calității răspunsurilor în conversațiile lungi, și că modele OSS de dimensiuni medii rulate local pot atinge un raport cost-performanță competitiv pe sarcini specifice de programare. Aceste rezultate sunt detaliate în capitolul *Rezultate experimentale* și sintetizate în *Concluzii*.

STRUCTURA LUCRĂRII

Lucrarea este organizată în cinci capitole. Capitolul *Context și poziționare* prezintă evoluția modelelor de limbaj, stadiul actual al arhitecturilor agentice și analiza comparativă a soluțiilor existente, încheindu-se cu justificarea alegerilor de implementare. Capitolul *Implementare* descrie în detaliu arhitectura aplicației `vibe-cli`, componentele majore și modul lor de interacțiune. Capitolul *Rezultate experimentale* prezintă cele trei protocoale de validare și interpretarea datelor obținute. Capitolul *Dezvoltări viitoare* schițează direcțiile de evoluție identificate, iar capitolul *Concluzii* sintetizează contribuțiile și rezultatele lucrării.

Capitolul 1

CONTEXT ȘI POZIȚIONARE

O buna înțelegere a tehnologiilor, standardelor și a obstacolelor actuale din industrie necesită studierea literaturii de specialitate. În a doua parte a capitolului analizăm locul pe care îl ocupa soluția propusă în peisajul industrial actual, prin comparație cu aplicațiile concurente atât proprietare cât și open source.

1.1 Evoluția către modelele mari de limbaj (LLM)

Drumul către modelele mari de limbaj a fost marcat de câteva contribuții fundamentale: perceptronul [23] a demonstrat în 1958 fezabilitatea învățării din date pentru clasificarea binară a vectorilor liniar separabili; rețelele convoluționale (Convolutional Neural Network (CNN)) [24] au introdus extragerea ierarhică de trăsături locale prin filtre de convoluție și *pooling*, devenind fundamentul *computer vision*; pentru date secvențiale au fost propuse rețelele recurente (Recurrent Neural Network (RNN)), iar pentru a depăși problema dispariției gradientului în Backpropagation Through Time (BTT), arhitectura Long Short Term Memory (LSTM) [25] a introdus stări de celula controlate prin porți de uitare, intrare și ieșire, devenind multă vreme standardul în Natural Language Processing (NLP).

Punctul de cotitură a fost reprezentat de arhitectura *transformer* [6] (Figura 1.1), care înlocuiește procesarea secvențială cu mecanismul de *self-attention*: toate elementele unei secvențe sunt analizate simultan, iar fiecare token primește informații contextuale din întreaga secvență indiferent de distanță, deblocând paralelizarea pe Graphics Processing Unit (GPU). Mecanismul de *scaled dot-product attention* este definit pentru matricile de interogări $\mathbf{Q}$, chei $\mathbf{K}$ și valori $\mathbf{V}$ prin $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k}) \mathbf{V}$, iar extensia *multi-head attention* folosește mai multe capete în paralel, fiecare specializându-se pe un alt tip de dependență.

Odată eliminată limitarea computațională, cercetătorii au putut antrena modele pe seturi de date fără precedent, deschizând drumul către modelele mari de limbaj (LLM). Echipa OpenAI [22] a dezvoltat Generative Pre-trained Transformer (GPT)-3, cu aproximativ **175 miliarde de parametri** [26], care prin furnizarea câtorva exemple sau a unei simple instrucțiuni în limbaj natural putea genera cod sursă, rezuma texte, traduce sau crea conținut creativ, marcând tranziția de la sisteme specializate la modele cu competente generale.

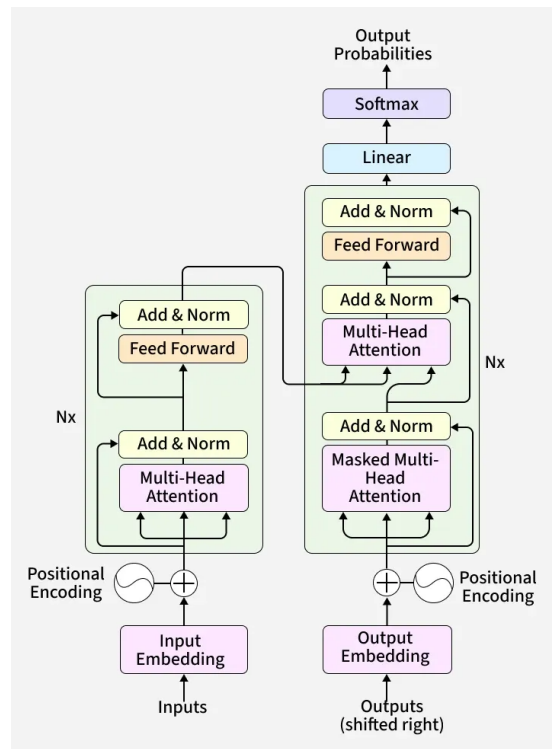


Figura 1.1: Arhitectura transformer [6]

1.2 Maturizarea modelelor mari de limbaj și cercetări actuale

Odată cu apariția primelor *foundation models* (modele mari de uz general antrenate pe date diverse) precum GPT-4, GPT-5, Sonnet sau Opus, industria a cunoscut o creștere exponențială (Figura 1.2). Aceste modele rezolvă probleme din domenii diverse, iar fiecare generație aduce îmbunătățiri în raționament, înțelegerea limbajului și rezolvarea problemelor complexe.

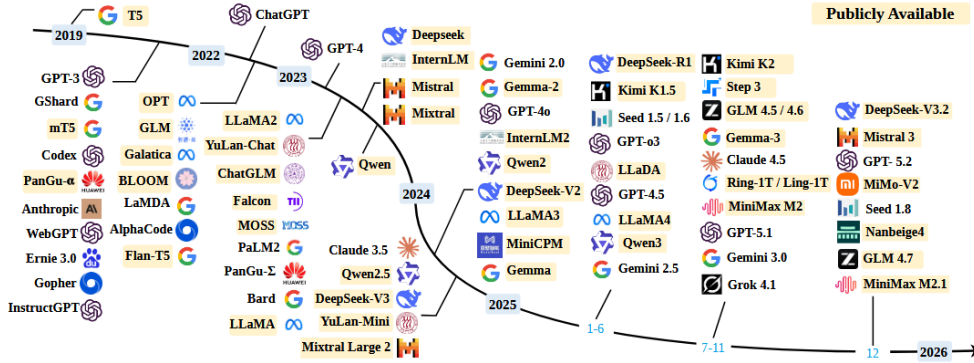


Figura 1.2: Evoluția recentă a modelelor mari de limbaj [7]

O direcție importantă de optimizare a fost procesarea secvențelor lungi. “Longformer: The Long-Document Transformer” [8] reformulează mecanismul de atenție astfel încât consumul de memorie să crească liniar cu lungimea contextului, iar abordări mai recente folosesc serii trigonometrice pentru a stabili compresia cache-ului Key-Value (KV) [27], îmbunătățind raționamentul pe termen lung [28].

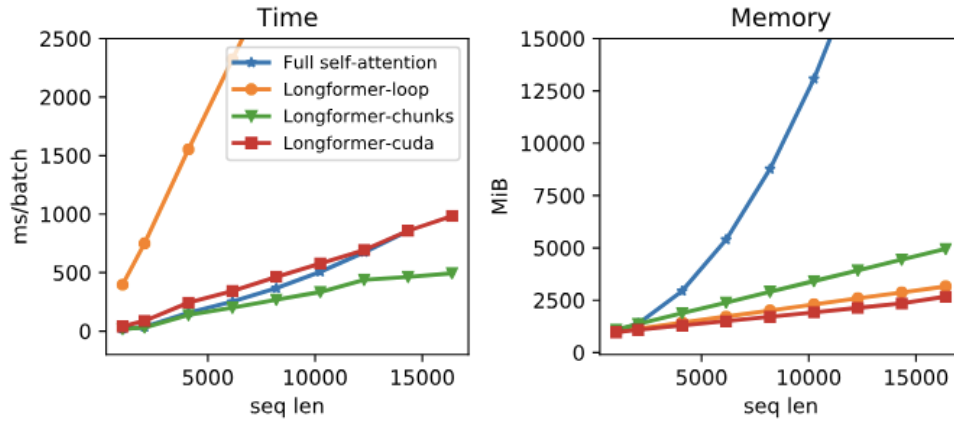


Figura 1.3: Performanța arhitecturii Longformer: în timp ce atenția standard scalează exponențial, Longformer scalează liniar cu lungimea contextului [8]

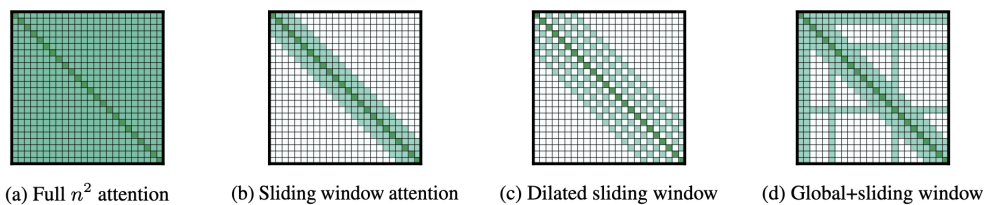


Figura 1.4: Diferite configurații de atenție folosite în arhitectura Longformer în comparație cu mecanismul obișnuit de atenție [8]

Pentru *embedding*-uri (vectori multi-dimensionali ce reprezintă token-ul plus poziția acestuia în secvența), “RoFormer” [29] codifica poziția printr-o matrice de rotație ce încorporează direct dependența relativă în formula atenției.

Pentru *fine-tuning* (ajustarea parametrilor unui model pentru un domeniu specific), LoRA [9] înghețează ponderile pre-antrenate $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$ și injectează o descompunere de rang mic $\Delta \mathbf{W} = \mathbf{B}\mathbf{A}$ ($\mathbf{B} \in \mathbb{R}^{d \times r}$, $\mathbf{A} \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$), reducând numărul de parametri antrenabili de până la 10.000 de ori.

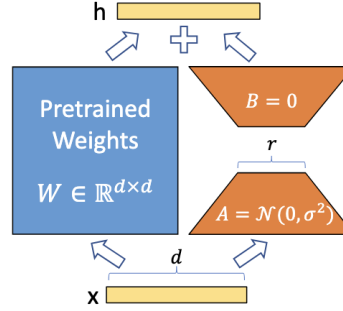


Figura 1.5: Fine-tuning-ul se realizează doar pe A și B [9]

Viteza de inferență a beneficiat de optimizări precum decodarea speculativă [30], care livrează mai mulți tokeni în paralel pornind de la aproximările unui model mai mic, fără a modifica distribuția. În paralel, procedeele de *cuantizare* (reducerea preciziei numerice a parametrilor) și *distilare* (antrenarea unui model mic să reproducă răspunsurile unui mai mare) micșorează modelele cu pierderi minime de performanță. Pentru distilare, [10] propune o alternativă proprie ce folosește eșantioane generate cu diverse temperaturi pentru *fine-tuning* supervizat.

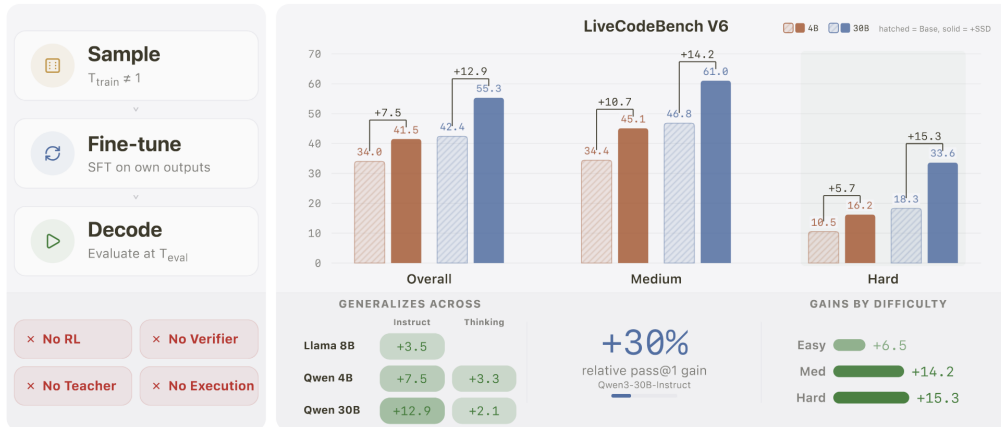


Figura 1.6: Rezultatele procesului de Simple Self Distillation [10]

Cuantizarea este tratată în [31] prin transformări (rotații aleatoare) ce induc o distribuție “Beta”, obținând reduceri semnificative de memorie, timp de inferență și lățime de bandă atât pentru ponderi cât și pentru cache-ul KV, esențial pentru ferestre de context lungi. Aceste avansuri sunt combinate cu LoRA în QLoRA [32].

Evoluția raționamentului profund se bazează în mare măsură pe Reinforcement Learning (RL): modelul este recompensat pentru ieșiri care manifesta gândire explicită (Chain-of-Thought (CoT) [12]), iar răspunsurile greșite sau nefundamentate sunt penalizate. [33] discută importanța unor semnale de recompensă precise pentru aplicații de programare, prin generarea de cazuri de test complexe.

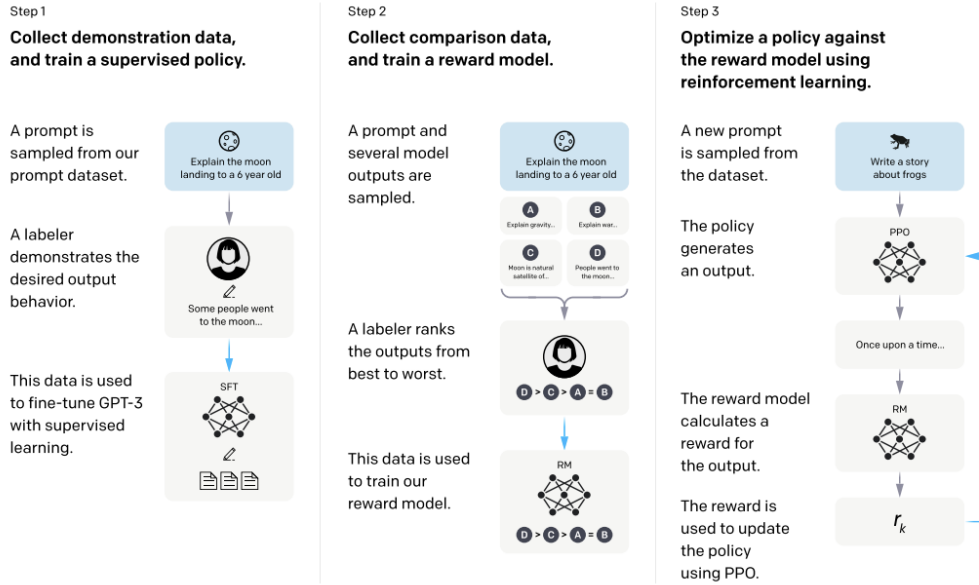


Figura 1.7: Ilustrarea procedeului de RL [11]

În final, și studiile asupra și *prompt*-ului oferit modelului au fost de impact: fiind auto-regresive, modelele depind puternic de un prompt bine construit, ceea ce a dat naștere disciplinei de *Prompt Engineering*, sistematizată în [34]. Pentru rezultate consistente și versionabile, DSPy [35] abstractizează prompt-urile prin grafuri de transformări însoțite de un compilator ce produce instrucțiuni optimizate.

1.3 Soluții agentice pentru industria dezvoltării software

Arhitecturile agentice presupun înzestrarea modelelor mari de limbaj cu uneltele necesare interacționării cu ecosistemul în care rulează. Fundamentul a fost pus de CoT [12], precedat ulterior de Tree-of-Thought (ToT) [36], care a demonstrat că un model poate naviga probleme complexe prin descompunerea raționamentului în pași intermediari expliciți. Metoda Plan-and-Solve [37] extinde această idee instruind modelul să conceapă mai întâi un plan ce divide sarcina în sub-obiective, eliminând erorile de pași lipsa frecvente în CoT clasic.

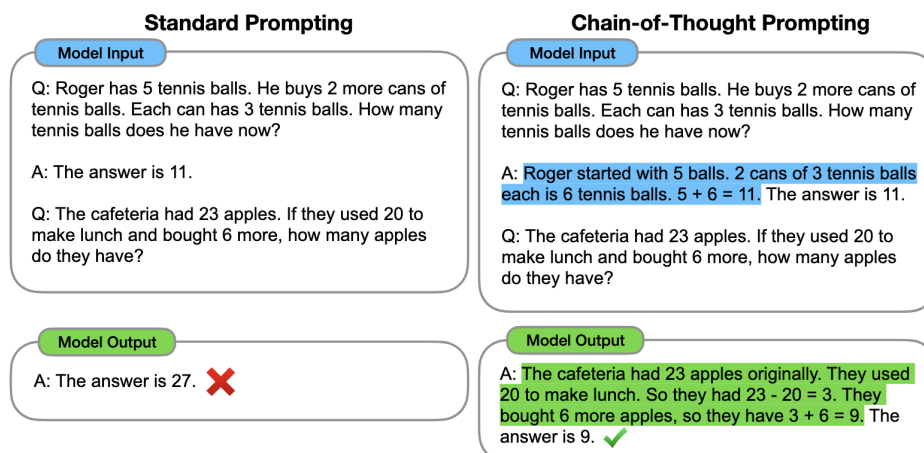


Figura 1.8: Exemple de CoT [12]

Mai târziu, abordarea ReAct [13] a combinat raționamentul cu acțiunea, permițând modelu-

lui să alterneze între generarea de gânduri intermediare și execuția de acțiuni concrete în mediul extern, după tiparul *Thought* → *Action* → *Observation*, repetat până la producerea răspunsului final. Aceasta sinergie a încercat să combată fenomenul *halucinațiilor* (răspunsuri neancorate în fapte reale) și inconsistența în răspunsuri, însă presupunea o intercalare costisitoare între raționament și observații: modelul genera un gând, apela o unealta, aștepta răspunsul, apoi decidea următoarea acțiune pe baza tuturor tokenilor anteriori.



Figura 1.9: Paradigma ReAct [13]

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

Figura 1.10: Îmbunătățirile pe care le aduce ReAct [13]

Pentru a elimina aceasta ineficienta, arhitectura ReWOO [14] decuplează complet procesul de raționament de observațiile externe: modelul generează întregul plan dintr-o singura trecere, iar observațiile sunt colectate independent și integrate ulterior. Aceasta separare reduce consumul de tokeni de până la 5 ori și îmbunătățește acuratețea cu 4% pe HotpotQA.

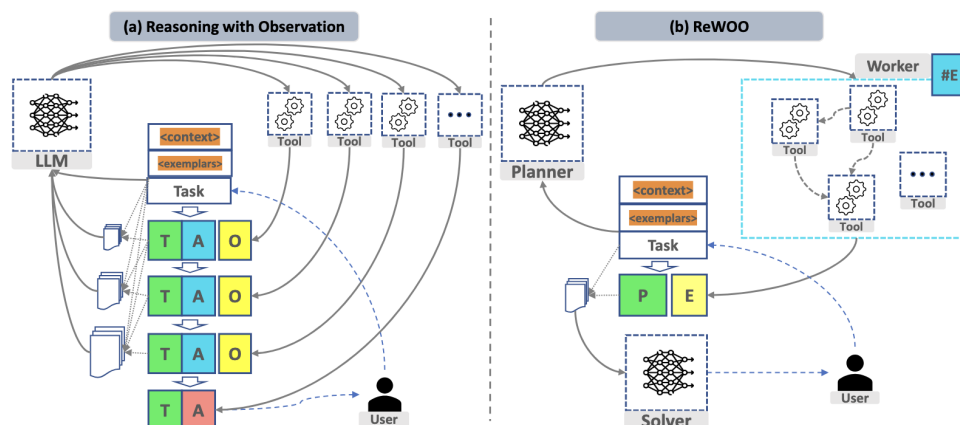


Figura 1.11: Îmbunătățirea adusa de ReWOO: worker-ul are mai multă libertate, putând apela mai multe unelte până să producă un răspuns. P = Plan, E = Evidence, A = Answer, O = Observation, A = Act, T = Thought [14]

Pe măsura ce agenții au fost înzestrați cu unelte tot mai numeroase (căutare web, terminal, control browser), fereastra de context a devenit o limitare critica. Deși modele precum Opus 4.8 (Anthropic) sau GPT-5.5 (OpenAI) anunță o fereastră de 1 milion de tokeni, “Lost in the Middle” [15] demonstrează ca performanța modelelor se degradează semnificativ când informația relevantă se afla în mijlocul unui context lung (Figura 1.12, Figura 1.13), implicații directe asupra agenților care acumulează istoric, apeluri la unelte și observații.

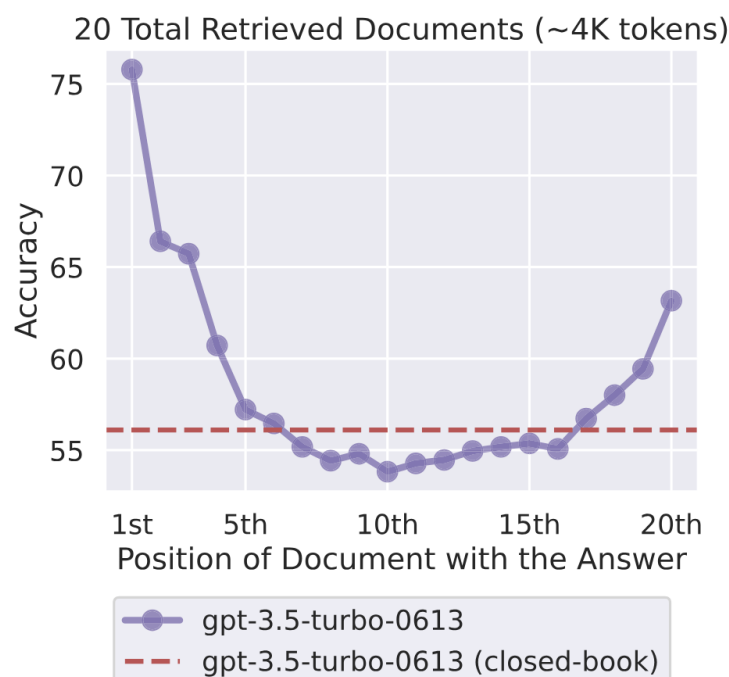


Figura 1.12: Modele tind să aibă o acuratețe foarte slabă contra întrebărilor ce vizează informații care se afla la mijlocul ferestrei curente de context [15]

Input Context

Write a high-quality answer for the given question using only the provided search results (some of which might be irrelevant).

Document [1](Title: Asian Americans in science and technology) Prize in physics for discovery of the subatomic particle J/ψ . Subrahmanyan Chandrasekhar shared...

Document [2](Title: List of Nobel laureates in Physics) The first Nobel Prize in Physics was awarded in 1901 to Wilhelm Conrad Röntgen, of Germany, who received...

Document [3](Title: Scientist) and pursued through a unique method, was essentially in place. Ramón y Cajal won the Nobel Prize in 1906 for his remarkable...

Question: who got the first nobel prize in physics

Answer:

Desired Answer

Wilhelm Conrad Röntgen

Figura 1.13: Vizualizarea problemei și al rezultatului dorit la întrebare. De multe ori, modelele întâmpina dificultăți în accesarea acestor informații ascunse în mijlocul ferestrei de context [15]

Cadrul de lucru Michelangelo [38] extinde testele clasice de cautare in date numeroase, testând abilitatea modelului de a sintetiza informații dispersate, și confirma un spațiu semnificativ de îmbunătățire chiar și pentru ultimele generații. Pentru eficienta cache-ului KV, LycheeCluster [39] fragmentează contextul pe baza delimitatorilor și construiește un index ierarhic ce transformă căutarea în cache într-un proces logaritm, obținând o accelerare **de până la 3.6 ori**.

1.3.1 Analiza preliminară a soluțiilor existente pe piață

Industria dezvoltării software are probabil una din cele mai profitabile întrebuițări ale acestor arhitecturi agentice, datorită volumului masiv de munca existent în domeniu, așa ca nu a durat mult până ce marii furnizori de pe piața modelelor de IA au început să ofere produse care să asiste programatorii în sarcinile lor. La momentul actual exista mai multe soluții competitive: Codex [40] de la OpenAI, Gemini CLI [41] de la Google sau Claude Code [42] de la Anthropic. Toate acestea sunt soluții proprietare care sunt făcute să ruleze cu modelele proprii și pentru care utilizatorul trebuie să plătească. Exista și soluții open-source precum OpenHands, Cline sau Aider cu care deținătorii de modele se lupta pentru a domina piața, deși lupta nu este strânsă din cauza monopolului soluțiilor proprietare.

Din tabelul 1.1 se poate deduce ca utilizarea frecventa a unui model de ultima generație poate ajunge sa coste o mică avere. Într-o conversație lunga, fiecare pereche răspuns-întrebare din istoric este re-introdusa ca date de intrare pentru următoarea interacțiune, iar fiecare apel către unelte rămâne și el în context, crescând exponențial costul de întreținere a conversatiei.

Din punct de vedere al capabilității de rezolvare a sarcinilor reale, clasamentul celor mai bune modele se schimba frecvent. Utilizând SWE-Bench Pro [16] putem măsura performanța contra unor sarcini complexe (Figura 1.14) și raportul intre performanta si cost (Figura 1.15).

Companie	Model	Preț [\$/MTok]	Capabilități
Anthropic (link [43])	Claude Opus 4.8	\$5/input MTok, \$25/output MTok	Cel mai capabil model disponibil, îmbunătățire semnificativă pentru programare agentica față de 4.6 și 4.7, gândire adaptivă, 1M fereastra de context, 128k tokens max. la ieșire
	Claude Sonnet 4.6	\$3/input MTok, \$15/output MTok	Mai rapid, compromite niște inteligența pentru performanță, gândire adaptivă, 1M fereastra de context, 64k tokens max. la ieșire
	Claude Haiku 4.5	\$1/input MTok, \$5/output MTok	Cel mai rapid, fără gândire adaptivă, 200k fereastra de context, 64k tokens max. la ieșire
OpenAI (link [44])	gpt-5.5	\$5/input MTok, \$30/output MTok	Nouă clasa de inteligență pentru programare și munca profesională, gândire adaptivă cu niveluri multiple de efort (low/medium/high/xhigh), suport multi-modal, 1M fereastra de context și 128k tokens max. la ieșire
	gpt-5.4-mini	\$0.75/input MTok, \$4.5/output MTok	Model mai mic, rapid dar puternic pentru programare și subagenți. 400k fereastra de context și 128k tokens max. la ieșire
	gpt-5.4-nano	\$0.20/input MTok, \$1.25/output MTok	Model mic și foarte rapid pentru sarcini ușoare în volum mare. 200k fereastra de context și 128k tokens max. la ieșire
Google (link [45])	Gemini 3.1 Pro	\$2/input MTok, \$12/output MTok < 200k tokens, \$4/input MTok, \$118/output MTok ≥ 200k tokens	Model pentru sarcini complexe ce necesită cunoștințe dezvoltate. Fereastra de context 1M in / 64k out
	Gemini 3.1 Flash	\$0.5/input MTok, \$3/output MTok	Inteligența comparabilă cu modelul Pro la o viteză mai mare. Fereastra de context 1M in / 64k out
	Gemini 3.1 Flash-Lite	\$0.25/input MTok, \$1.5/output MTok	Model construit pentru costuri reduse și volum mare de sarcini. Fereastra de context 1M in / 64k out
	Gemma 4	Open Source	Familie de modele open-source pentru rulare locală, cost redus și personalizare
DeepSeek (link [46])	DeepSeek V4	\$0.30/input MTok, \$0.50/output MTok	Model general de ultimă generație, suportă mod cu și fără gândire adâncă, 1M fereastra de context, cache hit: \$0.03/input MTok
	DeepSeek R1	\$0.55/input MTok, \$2.19/output MTok	Model specializat în raționament și gândire adâncă pentru probleme complexe, 128k fereastra de context, cache hit: \$0.14/input MTok
Qwen (link [47])	Qwen 3	Open Source	Familie de modele open-source (Apache 2.0) cu arhitectura hibridă Mixture of Experts (MoE), gândire adaptivă activabilă, performanță de vârf la programare și raționament. Disponibil în variante de 0.6B până la 235B parametri
	Qwen3-Coder	Open Source	Model specializat pe cod din familia Qwen3, optimizat pentru generare și înțelegere de cod, arhitectura MoE, gândire adaptivă, 1M fereastra de context.
Mistral AI (link [48])	Mixtral 8x22B	Open Source	Model Mixture-of-Experts open-source (Apache 2.0), 8 experți de 22B parametri fiecare, disponibil pentru rulare locală sau prin Application Programming Interface (API). 66k fereastra de context
	Devstral Small 2	\$0.10/input MTok, \$0.30/output MTok	Model open-source (Apache 2.0) specializat pe cod, bazat pe arhitectura Mistral Small 3.1, optimizat pentru agenți de programare și completare de cod. 128k fereastra de context

Tabelul 1.1: Compararea câtorva din modelele existente și a capabilitățile acestora în comparație cu prețul de utilizare

Nota: Toate modelele au și versiuni pentru diferite nivele de efort pentru gândire (reasoning effort) sau pentru date multi-modale ce pot influența și costul dar și rezultatele

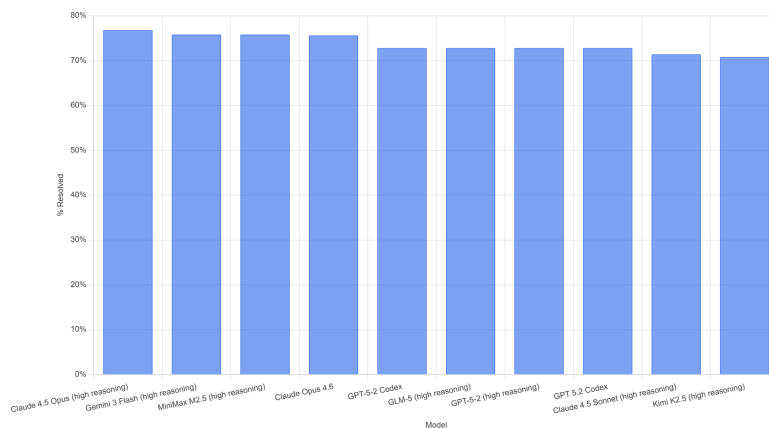


Figura 1.14: Clasamentul modelelor în SWE-Bench Pro [16] la momentul actual în funcție de performanța rezolvării sarcinilor complexe în limba engleza

Cele mai puternice modele necesită resurse semnificative pentru antrenare și inferență, costuri reflectate asupra utilizatorului final (cu strategii de taxare per-token sau abonament). Modelele cu gândire adâncă au cost mai mare per-token și consuma mai mulți tokeni. Fiecare deținător de modele oferă astfel variante de complexități diferite (de exemplu gama Gemini cu versiunile “Flash”, “Thinking”, “Pro”), iar aplicațiile au algoritmi ce rutează prompt-urile către modelul potrivit în funcție de complexitate, pentru a echilibra costul.

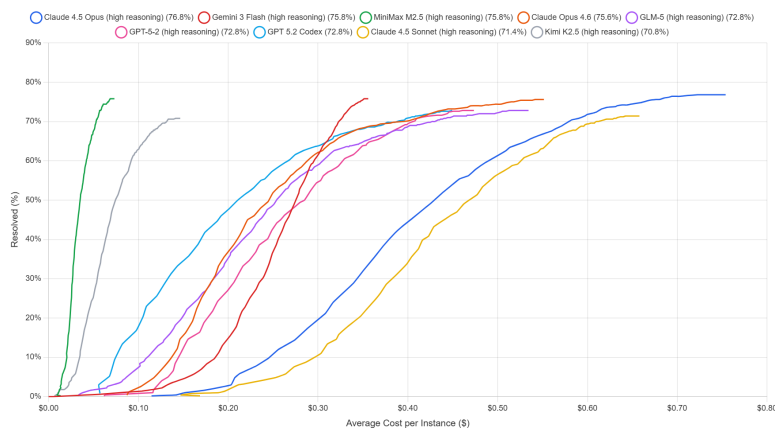


Figura 1.15: Clasamentul modelelor în SWE-Bench Pro [16] la momentul actual după de cost

1.4 Cerințe pentru o soluție agentică în dezvoltarea software

Pentru a realiza un asistent inteligent în dezvoltarea software trebuie atinse câteva obiective principale: o interfață accesibilă din diferite medii de dezvoltare, unelte ce pot rula în terminalul oricărui sistem de operare, posibilitatea de a alege dintr-o gama de modele, comenzi pentru a controla parametrii și skill-uri specializate.

Agenții funcționează, pe lângă procedeele expuse anterior, pe una dintre filozofii Spec-Driven Development (SDD) (programare pe baza documentelor de specificație) sau Test-Driven Development (TDD) (programare după cerințe sumare și executarea testelor la final). Pe lângă acestea, trebuie să ne asigurăm ca modelul răspunde doar prompt-urilor cu scop legitim și nu poate rula comenzi care pun în pericol integritatea sistemului, aspecte tratate în capitolele ce urmează.

1.5 Limitări ale arhitecturilor agentice

Bariera principală în avansarea acestor asistenți este **fereastra de context**. Pentru sisteme cu numeroase linii de cod sau proiecte distribuite în mai multe servicii, contextul limitează masiv capacitățile agenților. Deși există tehnici de optimizare (compresie, rezumare, fișiere de cunoștințe, rularea de cod pentru acțiuni repetitive), acestea nu sunt suficiente: multe funcționalități care fac un agent performant (unelele) consuma rapid contextul. Din cei 200.000 de tokeni anunțați, doar 160.000 sunt cu adevărat disponibili, restul fiind rezervați pentru promptul de sistem, auto-compactare sau ieșire.

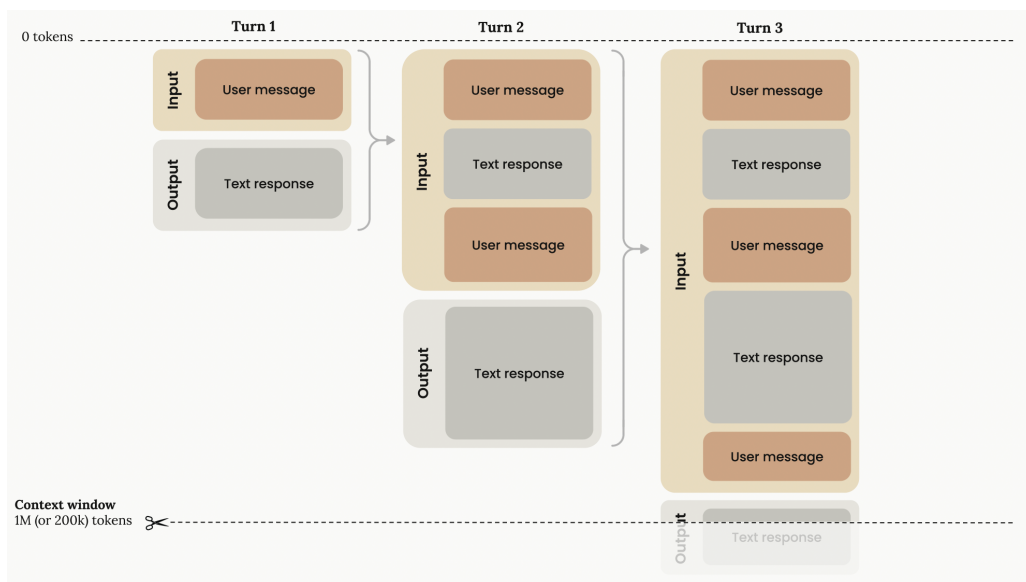


Figura 1.16: Reprezentarea ferestrei de context într-o conversație uzuală (Sursa: [17])

În plus, modelele curente tind să genereze date de ieșire mai detaliate decât este necesar (cantitate peste calitate), iar în sarcini de revizuire de PR se grăbesc să relateze multe probleme simple sau de formatare, ratând probleme importante de logica. De asemenea, agenții nu au o bună înțelegere legată de când ar trebui să oprească investigațiile, rămânând adesea blocați în bucle infinite. Pentru toate aceste probleme se lucrează în mod curent la rezolvări de care vom vorbi ulterior în capitolele ce urmează.

1.6 Justificarea alegerilor de implementare

Modelele disponibile inițial vor fi din gama Gemini, Anthropic, OpenAI și modele open-source rulate local prin Ollama sau în cloud. În materie de limbaj, **Python** [49] a fost alegerea evidentă: fiind un standard în industrie, ecosistem matur și portabilitate pe multiple platforme. Pentru cadrul de lucru agentic, am cautat o soluție gândită pentru utilizarea uneltelor, compatibilitate multi-model, ramuri de execuție, management al stărilor și memoriei și fluxuri de execuție robuste, comparația din tabelul 1.2 a stand astfel la baza alegerii.

La nivel conceptual, LangGraph modelează un flux de lucru agentic ca un *graf orientat* cu noduri de calcul (apeluri LLM, unelte, funcții) și muchii *condiționale* sau *ciclice* esențiale pentru bucle *reason-act-observe*. Graful operează pe o *stare* partajată tuturor nodurilor, iar mecanismul de *puncte de control* permite serializarea stării după fiecare pas pentru reluare, inspecție sau *observarea stărilor precedente*, oferind persistența între sesiuni și puncte *human-in-the-loop*.

În final am ales **LangGraph** datorită controlului fin asupra fluxului de lucru agentic și a integrării native cu Langfuse [53].

Criteriu	LangChain [3]	LangGraph [4]	Google Gen AI / Vertex [50]	LiteLLM [51]	OpenAI SDK [52]
Rol Principal	Orchestrator general de LLM-uri și integrări.	Cadru de lucru pentru agenți cu stare și fluxuri ciclice.	Platformă enterprise integrată în Google Cloud.	Proxy sau router universal pentru 100+ LLM-uri.	Client oficial pentru modelele OpenAI.
Nivel Abstr.	Înalt (multe componente gata de utilizat).	Mediu/Înalt (control fin asupra grafurilor).	Înalt (Managed Services în Vertex AI).	Jos (strat de traducere API).	Jos/Mediu (interacțiune directă).
Capabilități Agentice	Limitate (predominant fluxuri liniare).	Excelent (bucle, feedback și multi-agent).	Bun (suport nativ pentru tool-uri).	N/A (doar rutare de mesaje).	Depinde de utilizator (manual sau Assistants API).
Suport Multi-Model	Da (agnostic).	Da (prin ecosistemul LangChain).	Optimizat pentru Gemini.	Absolut (scopul principal).	Nu (doar OpenAI).
Management Stare	Încorporat (tampoane de memorie).	Superior (persistență și „time travel”).	Gestionat de platformă.	N/A (fără stare).	Threads (doar în Assistants API).
Învățare	Medie (ecosistem foarte mare).	Abruptă (necesită logica grafurilor).	Ușoară (Graphical User Interface (GUI) intuitiv în Cloud).	Ușoară (identic cu OpenAI Software Development Kit (SDK)).	Foarte Ușoară, documentat

Tabelul 1.2: Comparația cadrelor de lucru pentru întrebări agentice

Pentru interacțiunea cu utilizatorul, scopul a fost o interfață disponibilă în orice mediu de dezvoltare. Alegerea firească a fost un TUI, întrucât terminalul este disponibil universal. Tabelul 1.3 compară opțiunile pentru implementare.

Inițial am pornit cu Textual datorită sistemului sau puternic de dispunere, dar am abandonat ideea în implementare din cauza problemelor în conversații lungi cu elemente animate. Motivațiile suplimentare pentru restul alegerilor sunt dezvoltate punctual pe parcursul implementării.

Bibliotecă	Arhitectură	Avantaje	Dezavantaje
Textual [54]	Motor TUI pe ecran complet. Arbore de elemente bazat pe Document Object Model	• Sistem de dispunere similar cu Cascading Style Sheets • Formatare de text încorporată nativ • Complet asincron și bazat pe evenimente	• Folosește o bară de derulare falsă (nu funcționează derularea nativă) • Supraîncărcarea Document Object Model-ului cauzează latență dacă mesajele vechi nu sunt șterse • Dificultate la copierea/lipirea nativă a textului
Prompt Tool-kit [55] și Rich [56]	Experiență interactivă folosind tamponul principal al terminalului. Afișare prin Standard Output (STDOUT).	• Derulare perfectă prin bara de derulare nativă a terminalului • Zero latență sau probleme de memorie pe măsură ce istoricul crește • Selectarea și căutarea eficientă	• Nu este un motor real de dispunere GUI (doar flux liniar) • Dificil de implementat panouri laterale sau complexe • Limitat strict la input ancorat în partea de jos
Urwid [57]	TUI clasic pe ecran complet folosind tamponul alternativ al terminalului. Ierarhie strictă de widget-uri.	• Extrem de matur, performant, stabil și testat în producție • Gestionare excelentă a intrării standard	• API învechit și curbă de învățare mai abruptă • Sincron implicit (necesită integrare suplimentară)

Tabelul 1.3: Comparație biblioteci pentru interfața cu utilizatorul

1.7 Plasarea în industrie

Având o imagine clară asupra stadiului actual și a alegerilor arhitecturale, abordam plasarea proiectului în industrie și deciziile prin care propunem un produs competitiv.

1.7.1 Soluții proprietare

Momentan, cea mai mare cota de piață o au furnizorii de modele mari de limbaj precum Anthropic, Google sau OpenAI care lansează aplicații care au ca obiectiv blocarea utilizatorilor în ecosistemul lor personal, prin unelte ce urmăresc domenii diverse, fenomen observabil în tabelul 1.4.

Tabelul 1.4: Produse IA ale principalilor furnizori de modele de limbaj

Companie	Produs	Descriere	Audienta	Preț
Anthropic [21]	Claude [58] (web/app)	Asistent IA conversațional pentru analiza, scriere, programare și raționament, cu suport multi-modal și artefacte interactive	Utilizatori generali, profesioniști	Freemium (Pro: \$20/luna, Max: \$100–200/luna)
	Claude Code [42]	Agent de programare agentic care rulează în terminal, Integrated Development Environment (IDE) sau Slack [59]; poate naviga coduri mari, crea PR-uri și rula teste automat	Dezvoltatori software	Inclus în Pro/Max (API: cost per token)
	Claude Cowork	Agent agentic de productivitate care lucrează cu fișiere locale și aplicații cloud (Slack, Google Drive) pentru a automatiza sarcini recurente	Profesioniști, echipe	Inclus în Pro/Max
	Claude Design	Instrument de prototipare vizuala integrat în Claude, care generează interfețe GUI interactive și mockup-uri pe baza descrierilor în limbaj natural	Designeri, dezvoltatori	Inclus în Pro/Max
Google [20]	Gemini [60] (web/app)	Asistent IA conversațional multi-modal cu acces la căutare Google, extensii și generare de imagini; disponibil pe web, Android [61] și iOS [62]	Utilizatori generali	Freemium (Advanced: \$19.99/luna în Google One AI Premium)
	Gemini CLI [41]	Agent de programare open-source (Apache 2.0) care rulează în terminal, cu suport MCP, căutare Google și context de 1M tokeni	Dezvoltatori software	Gratuit (60 req/min, 1000 req/zi cu cont Google)
	NotebookLM [63]	Asistent de cercetare alimentat de Gemini care analizează documente încărcate (Portable Document Format (PDF), site-uri, video-uri) și generează rezumate, podcast-uri audio și conexiuni între surse	Cercetători, studenți	Gratuit (Plus: \$7.99/luna, Business: contact)
	Vertex AI [50]	Platforma enterprise pentru construcția, antrenarea și implementarea de agenți IA și modele Machine Learning, cu acces la 200+ modele și instrumente Machine Learning Operations	Echipe enterprise, ingineri Machine Learning	Plătit (pay-as-you-go, de la \$0.0001 per operație)
	Firebase Studio [64]	Mediu de dezvoltare cloud (Integrated Development Environment (IDE)) cu agenți IA integrați pentru construcția de aplicații full-stack (backend, frontend, mobil) direct din browser	Dezvoltatori software	Gratuit în previzualizare (3 spații de lucru; se închide în 2027)
OpenAI [22]	ChatGPT [65] (web/app)	Asistent IA conversațional cu suport multi-modal, căutare web, generare de imagini, analiza de date și extensii; disponibil pe web, iOS, Android și desktop	Utilizatori generali, profesioniști	Freemium (Plus: \$20/luna, Pro: \$200/luna)
	Codex [40]	Agent de inginerie software cloud care execută sarcini de programare în paralel, în sandbox-uri izolate preîncărcate cu codul din repository-ul GitHub [66]	Dezvoltatori software, echipe	Inclus în ChatGPT Pro/Business/Enterprise (API: \$1.50/\$6 per MTok)
Microsoft [67]/ GitHub	GitHub Copilot [68]	Agent de programare integrat în GitHub și disponibil ca aplicație CLI, extensie Integrated Development Environment (IDE) (VS Code [2], JetBrains [69]) și în github.com; oferă completare de cod, chat, revizuire PR-uri și rezolvare automată de tichete	Dezvoltatori software, echipe	Freemium (gratuit pentru repo-uri publice; Pro: \$10/luna, Business: \$19/luna, Enterprise: \$39/luna)

Alternativele open-source se lovesc de probleme de natura concurențială: tarife diferențiale pentru cheile API (uneori cu risc de restricționare), lipsa fondurilor și a contribuitorilor, precum și a publicității. În ciuda acestor dificultăți, accesul gratuit, suportul pentru modele open-source rulate local și natura agnostica de furnizor sunt avantaje majore în fața costurilor crescânde.

1.7.2 Soluții open-source

Pe lângă aplicațiile furnizorilor exista și câteva aplicații open source care s-au bucurat de o oarecare adopție, regăsite în tabelul 1.5.

Tabelul 1.5: Top 6 asistenți open source după numărul de "stele" pe GitHub

Aplicație	Audienta și scop	Puncte forte	Puncte slabe	Funcționalități cheie
OpenClaw (GitHub [70]) Licență MIT	Utilizatori care doresc un asistent IA personal, mereu activ, accesibil pe orice canal de comunicare	• Suport multi-canal masiv (WhatsApp [71], Telegram [72], Slack [59], Discord [73]) • Sandboxing cu Docker [74], Secure Shell și OpenShell • Aplicații native macOS [75], iOS, Android [61] • Agnostic față de furnizor	• Orientat spre asistent personal, nu exclusiv programare • Configurare complexă pentru multi-canal • Nu are integrare directă în Integrated Development Environment (IDE)	Multi-provider, multi-canal, sandboxing, control vocal, suprafață interactivă, sarcini programabile, skills, multi-agent
OpenHands (GitHub [76]) Licență MIT	Dezvoltatori și echipe enterprise care necesită un agent autonom complet cu izolare securizată	• Sandboxing în containere Docker • GUI web și CLI disponibile • SDK Python pentru construcția de agenți personalizați • Suport Kubernetes [77] • Agnostic față de furnizor	• Necesita Docker • Configurare inițială complexă • Nu are integrare directă în Integrated Development Environment (IDE) • Curba de învățare mai abruptă	Sandboxing, SDK, GUI web, CLI, Kubernetes, integrări Slack [59]/Jira [78]
Cline (GitHub [79]) Licență Apache 2.0	Dezvoltatori care prefera lucrul direct din Integrated Development Environment (IDE) cu control granular și automatizări avansate	• Extensie VS Code [2] și extensie JetBrains [69] • SDK Node.js [80] și CLI neinteractiv pentru CI/CD • Echipe multi-agent și agenți programați • Suport MCP și extensii • Agnostic față de furnizor	• Consum ridicat de tokeni datorită contextului mare • Fără sandboxing nativ incorporat • Complexitate ridicată, multe funcționalități • Dependent de ecosistemul VS Code/JetBrains	Extensie Integrated Development Environment (IDE), SDK, CLI, multi-agent, MCP, agenți programați, integrări Slack/Discord/Telegram
Goose (GitHub [81]) Licență Apache 2.0	Utilizatori generali și dezvoltatori care doresc un agent IA nativ, performant și extensibil	• Aplicație nativă desktop (macOS, Linux, Windows) • Construit în Rust [82] deci are o performanță ridicată • Suport MCP (70+ extensii) • Agent general, nu doar pentru cod • Agnostic față de furnizor (15+ furnizori, suport Agent Communication Protocol)	• Mai puțin specializat pe programare • Ecosistem de extensii mai tânăr • Nu are integrare directă în Integrated Development Environment (IDE) • Documentație în curs de maturizare	Aplicație nativă, CLI, API, MCP, Agent Communication Protocol, Rust, multi-platform, Linux Foundation
Aider (GitHub [83]) Apache 2.0	Dezvoltatori individuali care prefera un flux de lucru minimalist din terminal cu integrare Git [84]	• Mapare automată a codului sursă • Integrare Git nativă cu commit-uri automate • Suport 100+ limbaje de programare • Verificare cod și testare automată • Agnostic față de furnizor	• Fără sandboxing • Exclusiv terminal, fără GUI • Capabilități agentice limitate față de concurența • Nu are extensie nativă de Integrated Development Environment (IDE)	Terminal, Git auto-commit, repo map, voice-to-code, verificare statică, modele locale

1.7.3 Flux agentic ce rulează local

Aplicația oferă **confidențialitate totală a datelor și control complet**: toate datele rămân pe dispozitivul utilizatorului (cu excepția cazului în care folosește modele furnizate de terți), configurațiile sunt locale și complet personalizabile, iar agentul are acces direct la mediul de lucru. Astfel se elimină intervenția umană pentru sarcini repetitive (cu excepția acțiunilor periculoase ce necesită permisiuni speciale), permițând automatizări complexe prin integrarea cu terminalul sistemului.

1.7.4 Observabilitate

Majoritatea panourilor furnizorilor sunt sumare. Aplicațiile în terminal oferă mai multe date, dar nu detaliază ce unelte consuma cel mai mult context.

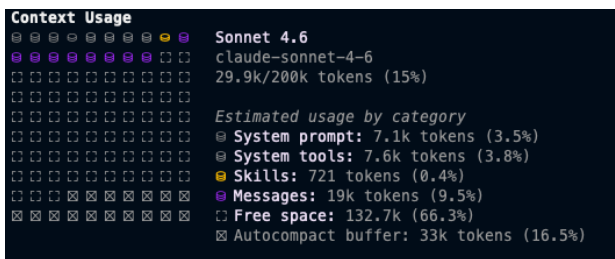


Figura 1.17: Reprezentarea ferestrei de context într-o conversație în Claude Code

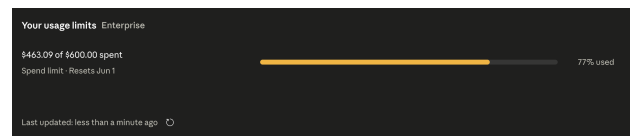


Figura 1.18: Reprezentarea utilizării în Claude (varianta web) (Sursa: [18])

Astfel, observabilitatea joacă un rol important pentru transparența: utilizatorul poate interoga orice metrică relevantă în timp real folosind Langfuse și panoul web inclus cu aplicația.

1.7.5 Securitate și confidențialitate

Confidențialitatea datelor este totală (excepție: modelele furnizorilor sau Langfuse cloud, opționale). Pentru securitate, aplicația include: sandboxing prin containere (execuție de comenzi în medii sigure), gardarea logică a apelării uneltelor (de exemplu, *read-before-write*) și cererea de aprobare a utilizatorului pentru acțiuni periculoase.

1.7.6 Cost

Nu se percepe cost pentru utilizare. Pentru modelele open-source rulate local, singurele costuri suplimentare sunt cele de hardware, curent electric și internet. Aplicația este optimizată pentru a reduce consumul de tokeni, deci și pentru modele furnizate de terți, costul rămâne minim.

1.8 Analiză proprie și decizii arhitecturale

În baza analizei stadiului actual al domeniului consolidăm aici deciziile arhitecturale care articulează identitatea aplicației *vibe-cli* și o diferențiază de produsele studiate. O parte preiau practici validate în industrie (TUI cu tampon nativ al terminalului, precum în Claude Code și Aider, sau organizarea agentului sub forma graf de stare și control uman), iar restul deciziilor propun mecanisme proprii (proces de compactare configurabil, echipe de subagenți cu model dedicat, panou web local pentru transparență end-to-end). Detaliile tehnice se găsesc în capitolele *Implementare* și *Rezultate experimentale*.

1.8.1 Arhitectură modulară

Aplicația este structurată pe patru subsisteme cu responsabilități separate (`src/`, `ui/`, `web/`, `eval/`), personalizabile prin fișiere de configurație YAML și prin preferințe persistate local, ceea ce permite evoluția independentă a modulelor și testarea izolată.

1.8.2 Interfața cu utilizatorul

Prototipul inițial bazat pe TUI pe ecran complet (Textual) a fost abandonat după ce a confirmat experimental limitările din tabelul 1.3 (bară de derulare falsă, latență în conversații lungi, dificultate la copiere). Soluția finală folosește tamponul nativ al terminalului prin `prompt_toolkit` și `Rich`, în acord cu Claude Code, Codex CLI și Aider. Peste această bază am proiectat un set propriu de elemente vizuale pentru transparența activității agentului: indicator de activitate cu starea curentă și cronometru, fișiere jurnal persistente ale uneltelor, panouri de `diff` la scrierea în fișiere, *împachetare lipire* pentru texte lungi și antet de stare cu metrice de sesiune.

1.8.3 Tehnici proprii de gestionare a contextului și a apelurilor la unelte

Peste tehnicile standard de compactare am introdus trei mecanisme proprii:

1. un proces de compactare configurabil integral din fișiere YAML, structurat ca registru de strategii extensibile prin subclase. Spre deosebire de produsele din industrie, fiecare stadiu este expus utilizatorului pentru a permite studiul efectului acestor metode asupra coerenței agentului;
2. filtre deterministe (*filtre*) ce verifică precondiții logice între apelurile de unelte, dintre care *read-before-write* (citire înainte de scriere) folosește un hash SHA pentru integritatea contextului fișierului;
3. politică de sandboxing, cu suport pentru subagenți prin contextul părintelui pentru izolare uniformă.
4. Subagenții sunt complet configurabili per tip (model, prompt, unelte permise, adâncimea recursivității, paralelism, timp de expirare), trăsătură fără echivalent în produsele proprietare studiate.

1.8.4 Suport pentru date multi-modale

Arhitectura agnostică de furnizor a impus rezolvarea unei probleme proprii: detectarea automată a capabilităților fiecărui model, întrucât metadatele disponibile de la furnizori nu sunt consistente. Am ales o strategie în care aplicația trimite efectiv conținutul multi-modal și, la refuz, marchează lipsa suportului în fișierul de preferințe, fără să scoată la suprafață eroarea către utilizator. Gestionarea contextului folosește o schemă hibridă: fișierele textuale și Portable Document Format (PDF) se pot citi în format text cu uneltele uzuale (zone de interes, trunchiere), iar dacă modelul susține nativ tipul de date, agentul poate cere o vedere completă prin citiri succesive de maxim 20 de pagini, configurabile. Imaginile sunt injectate direct în context cu limită implicită de 10 MB, iar fișierele sunt persistate automat ca artefacte recuperabile prin referință după compactare.

1.8.5 Observabilitate pe două straturi

Observabilitatea este oferită pe două nivele complementare, dezactivate implicit:

1. strat de tracing extern prin Langfuse Cloud ce pentru fiecare invocare a grafului LangGraph generează un *trace* (urma a executiei) ierarhic cu *span*-uri (compartimentări) per apel LLM, unealtă și nod, etichetate cu identificatori;
2. panou web local, care vizualizează direct fișierele de sesiune fără bază de date intermediară. Panoul, dezvoltat propriu, expune metrici absente din produsele comerciale studiate: consum de tokeni pe categorii de evenimente, evoluția dimensiunii contextului raportată la limita modelului, latența uneltelor, debitul de generare, costul estimat per apel pe baza unui tabel de prețuri configurabil și istoricul evenimentelor de compactare cu diferențele aferente.

1.8.6 Cercetarea tehnicilor de utilizare optime pentru arhitecturi locale în dezvoltarea software

Am construit trei suite de evaluare proprii, detaliate în *Rezultate experimentale*:

1. influența lungimii și specificității prompt-ului de sistem asupra performanței, prin agentul rulat în sarcina de revizuire de PR pe 50 de revizuiți umane ideale extrase din 6 proiecte OSS, cu metrici de precizie, recunoaștere și F_1 ;
2. interacțiunea dintre stadiile procesului de compactare, observând efectele asupra integrității contextului și recunoașterii;
3. (3) compararea a 16 modele (proprietary și open-source, între 1.5 și 32 de miliarde de parametri) pe 6 sarcini de programare executate pe un proiect real, cu notare hibridă (similaritate semantică, *LLM-as-judge*, teste deterministe ascunse).

1.8.7 Sinteza recomandărilor extrase din experimentare

Pe baza experimentelor enunțate anterior, am formulat un set propriu de recomandări pentru utilizarea ecosistemului de soluții agentice: ale

1. alegerea modelului după raportul cost-performanță, nu după numărul de parametri;
2. performanțe slabe pentru modelele sub 20 de miliarde de parametri în medii cu apel intensiv de unelte;
3. preferința pentru limba engleză ca date de intrare (mai ales la modelele mici antrenate preponderent pe date în limba engleză), sau pentru anumite limbaje de programare;
4. configurarea procesului de compactare în funcție de sarcină;
5. utilizarea modelelor open-source ca alternativă viabilă când confidențialitatea sau costul sunt constrângeri principale.

Detalierea fiecărei recomandări, ancorate în date, se găsește în capitolul de rezultate experimentale.

Capitolul 2

IMPLEMENTARE

2.1 Mediul de dezvoltare

Cerințele hardware pentru dezvoltarea și rularea acestei aplicații sunt minime: pentru modelele din cloud este suficient un laptop ce poate rula Python 3 cu conexiune la internet, întrucât inferență nu este realizată local. Pentru rularea unui model local, cerințele cresc semnificativ, depinzând de modelul ales.

Tabelul 2.1: Hardware recomandat pentru modele open-source

Model	Parametri	GPU Recomandat	Video Random Access Memory (VRAM)	Random Access Memory (RAM)
LLaMA 3	70B	2x NVIDIA A100 80GB sau 2x RTX 4090	40–160 GB (cuantizat vs FP16)	64–128 GB
Mistral	7B	NVIDIA RTX 3060 / RTX 4090	4–16 GB	8–32 GB
Qwen 2.5 (14B)	14B	NVIDIA RTX 3090 / RTX 4090	16–24 GB	16–64 GB
Gemma 4 (E4B)	4.5B efectivi (8B total)	NVIDIA RTX 3060 / RTX 4060	4–16 GB (cuantizat vs BF16)	8–32 GB

Pentru dezvoltare am folosit un laptop MacBook Pro cu chip Apple M5 Max și 48 GB RAM, pe macOS Tahoe 26.4.1 cu Python 3.10.

2.2 Structura

Am structurat aplicația în mai multe directoare specializate:

- Directorul **src** conține logica agentului: graful LangGraph, uneltele, sistemul de permisiuni, clientul MCP, procesul de compactare, sub-agentii, sandbox-ul Docker și tracing-ul;
- Directorul **ui** conține interfața cu utilizatorul: buclă principală, comenzile slash, sistemul de skill-uri, gestionarea sesiunilor și a preferințelor;
- Directorul **web** conține panoul analitic care vizualizează sesiunile, contextul, consumul de tokeni și activitatea uneltelor;

- Directorul `eval` conține suita de evaluări cu metrici precum Bilingual Evaluation Understudy (BLEU), Recall-Oriented Understudy for Gisting Evaluation (ROUGE) și potrivire semantică.
- Documentația detaliată per subsistem se găsește în `docs`, iar configurarea implicită în `config/config.yml`. Codul sursă complet al aplicației, organizat pe fișiere, este reprodus în *Anexe*, la finalul lucrării.

Vederea de ansamblu asupra modulelor aplicației și a interacțiunilor dintre ele este prezentată în figura 2.1.

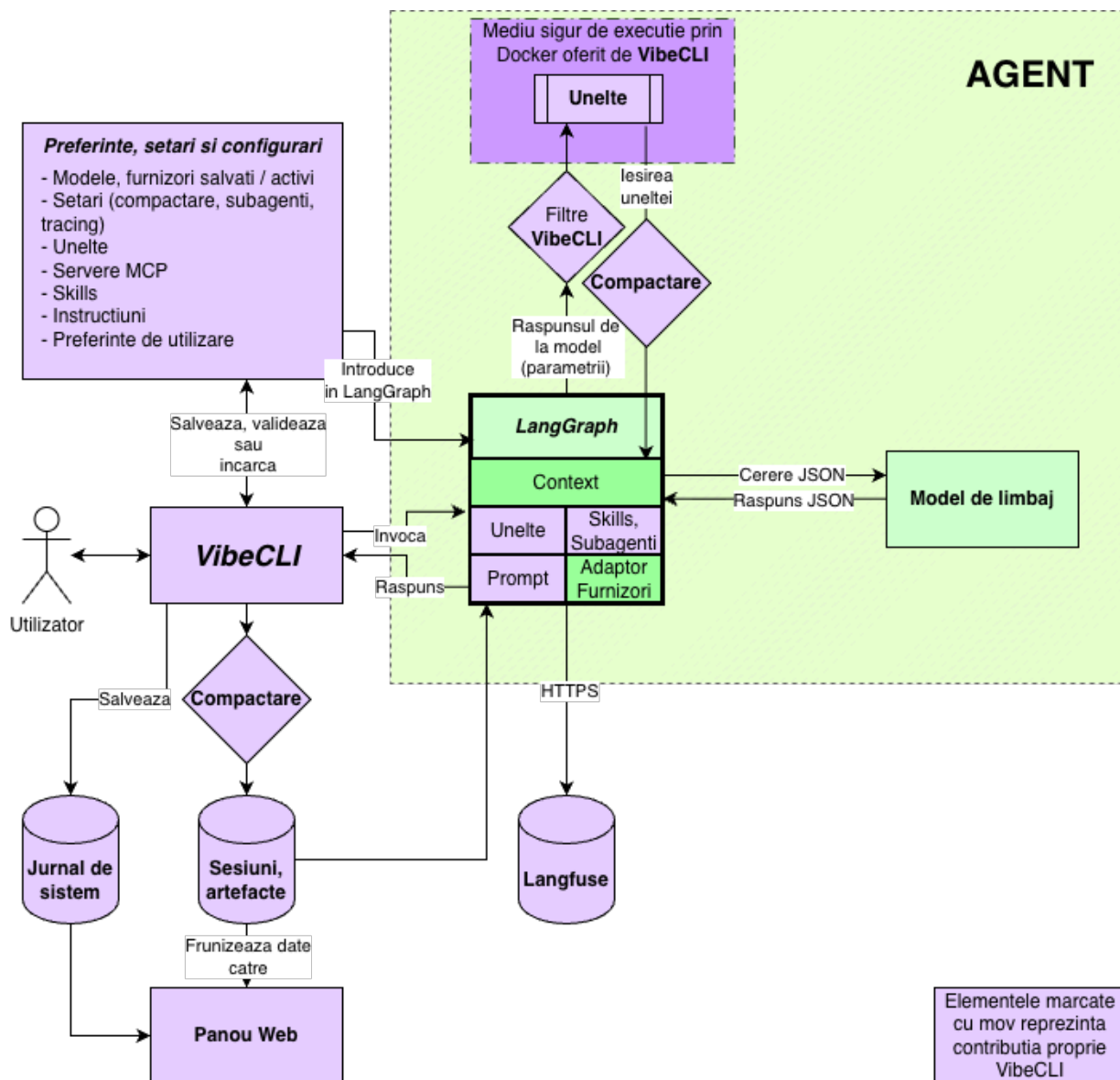


Figura 2.1: Arhitectura modulară a aplicației `vibe-cli`

vibe-cli/	src/	config/
- config/	- main.py	- config.yml
- docs/	- tools.py	\ - eval.yml
- eval/	- tool_guards.py	
- examples/	- providers.py	ui/
- scripts/	- permissions.py	- app.py
- src/	- sandbox.py	- ui.py
- tests/	- mcp_client.py	- commands.py
- ui/	- artifacts.py	- skills.py
- web/	- attachments.py	- sessions.py
- ARCHITECTURE.md	- capabilities.py	- preferences.py
- FIRST_START.md	- clipboard.py	- provider_handler.py
- HEADLESS_USAGE.md	- log_context.py	- pastes.py
- Makefile	- logging_setup.py	- paths.py
- pyproject.toml	- multimodal.py	- profile.py
- README.md	- tracing.py	- config_init.py
- UPDATING.md	- compaction/	- vibe_md.py
\ - uv.lock	\ - subagents/	\ - updater.py

Figura 2.2: Structura directoarelor vitale ale proiectului

2.3 Configurare

Configurarea aplicației se face prin intermediul fișierului `config.yml` care conține catalogul de furnizori (Ollama, OpenAI, Claude, Amazon Bedrock [85], Gemini), modelele disponibile pentru fiecare, procesul de compactare, setările de tracing, configurația sandbox-ului sau a subagenților și a uneltelor. Cheile API sunt declarate ca variabile de mediu sau în fișierul de preferințe și sunt încărcate la pornire. Dacă nici o cheie nu este setată și Ollama nu este accesibil, aplicația solicită interactiv introducerea unei chei la pornire prin comanda `/model` sau `/key`.

De asemenea pentru procesul de evaluare al capacităților folosit pentru teste de sanitate și regresie, se pot modifica setările din fișierul `~/.vibe-cli/eval/eval.yml`

Preferințele utilizatorului (furnizorul implicit, modelul selectat per furnizor, nivelul de efort, serverele MCP, listele de permisiuni) sunt persistate în `~/.vibe-cli/preferences.json` și supraviețuiesc între sesiuni. Sesiunile de conversație sunt salvate ca jurnale de evenimente în `~/.vibe-cli/sessions/`, sortate per proiect (directorul de lucru).

2.4 Interfața cu utilizatorul

Interfața este mediul de legătură între utilizator și agent și totodată spațiu de configurare. Apelează funcții ale agentului în același proces Python.

Primul prototip a folosit biblioteca Textual, dar pe măsură ce conversația creștea, mișcarea ferestrei se comporta inconsistent. Soluții similare (Claude Code, Codex) nu folosesc un TUI propriu-zis, ci scriu în terminal prin Standard Output (STDOUT), lăsând derularea nativă să gestioneze istoricul. Am abandonat astfel Textual în favoarea unei interfețe bazate pe `prompt_toolkit` și `Rich`, obținând o experiență fluidă, fără probleme de derulare sau latență.

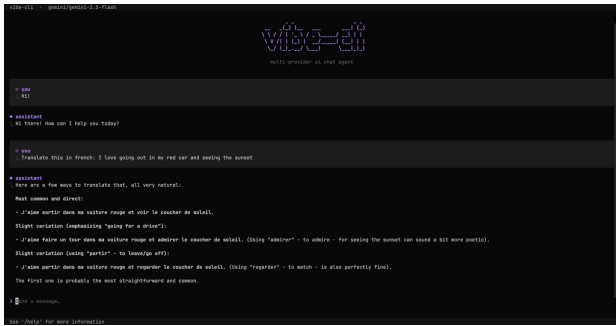


Figura 2.3: Prima versiune a interfeței, realizată cu ajutorul bibliotecii Textual

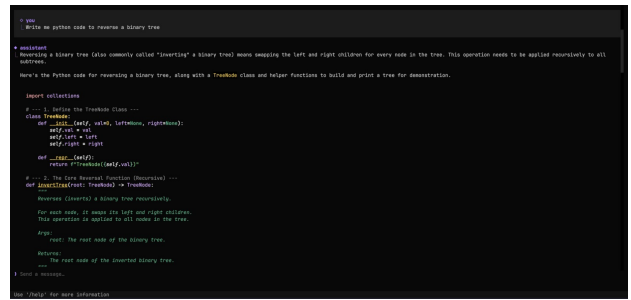


Figura 2.4: Continuarea conversației cu prima versiune a interfeței, realizată cu ajutorul bibliotecii Textual

Schimbarea s-a produs de la primele prototipuri, fără a întârzia implementarea, iar soluția finală este estetică, simplă și performantă.

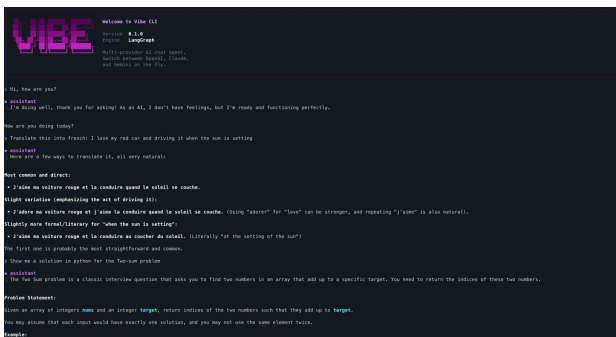


Figura 2.5: A doua versiune a interfeței, realizată fără biblioteca Textual

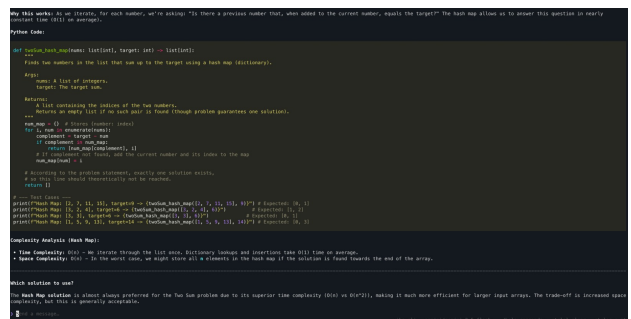


Figura 2.6: Continuarea conversației cu a doua versiune a interfeței, realizată fără biblioteca Textual

Interfața oferă mai multe elemente vizuale menite să informeze utilizatorul în timp real:

- **Indicator de activitate animat cu starea curentă și cronometru** (de exemplu `thinking... 1.2s`).
- **Jurnal persistent de unelte**: fiecare apel lasă o linie $\leftrightarrow$ `Tool(arg)`, astfel încât execuția uneltelor este vizibilă.
- **Panouri de diff pentru write_file**, redactate ca diff unificat cu numărul de linii adăugate/șterse.
- **Împachetare lipire**: textele lipite mai lungi de 200 de caractere sunt înlocuite cu un substituent `[Paste #N <len> chars]`, conținutul original fiind restaurat înainte de trimitere.
- **Footer de stare**: durata interacțiunii, setări active, atașamente, informații despre context.
- **Auto-completare pentru comenzile slash și actualizare automată a versiunii**.

```
> Hi, write me a poem in french.

+ assistant
  | Voici un poème en français pour vous :

-----

L'Écho du Soir

Le vent murmure aux feuilles tombantes, La lune dore les collines lointaines. Les étoiles dansent au-dessus des champs silencieux, Comme si le ciel répondait à nos prières muettes.
Je marche seul sur ce chemin de terre battue, Pensant à toi qui es déjà bien loin. Ton rire résonne encore en mon cœur fidèle, Un souvenir doux que je porte sans trépidation.
Les saisons passent comme des vagues d'automne, Emportant avec elles nos rêves d'enfance. Mais l'amour reste ancré dans le temps qui court, Comme une rivière pure et ininterrompue.
L'écho du soir nous réunit encore, Dans la danse éternelle des ombres et de la lumière.

+ Sorted in 3.7s

> Send a message...
```

Figura 2.7: Exemplu de conversație în interfața vibe-cli

```
Edited: poem.md +19 -0
@@ -0,0 +1,19 @@
+# The Whispering Willow
+
+## Verse 1
++Beneath the willow's silver bough,
++The moonlight dances soft and slow,
++Where shadows play in gentle flow,
++And quiet secrets softly grow.
+
+## Verse 2
++The river winds through ancient stones,
++Reflecting dreams of skies above,
++While time has carved its silent throne,
++In moments we can never love.
+
+## Verse 3
++But hope remains where light breaks through,
++Like morning dew on petals new,
++A promise kept by stars and you,
++That beauty finds a way to anew.
```

(a) Crearea unui fișier nou prin interfața vibe-cli

```
Edited: poem.md +4 -4
@@ -8,8 +8,8 @@
## Verse 2
-The river winds through ancient stones,
-Reflecting dreams of skies above,
-While time has carved its silent throne,
-In moments we can never love.
+The river flows with gentle grace,
+Through valleys deep in quiet place,
+Where ancient trees their roots embrace,
+And nature's song begins its chase.

## Verse 3
```

(b) Editarea unui fișier existent prin interfața vibe-cli, cu afișarea diferențelor

2.5 Funcționalități

Funcționalitățile principale ale aplicației includ:

- Unelte inteligente de interacțiune (citire/editare fișiere, căutare web, rulare în terminal);
- Gardarea apelului logic al uneltelor (de exemplu, `edit_file` nu poate fi apelat fără un `read_file` prealabil; citirea de fișiere mari este partiționată pentru a nu umple contextul);
- Alegerea modelului și a efortului de gândire;
- Suport multi-modal (4 tipuri: imagini, PDF, text, documente) prin cale locală sau Uniform Resource Locator (URL), când modelul suportă tipul de date (vezi tabelul 3.5);
- Gestionarea sesiunilor per-proiect, cu salvare, ștergere, listare și continuare.
- Sistem de permisiuni per-unealta cu liste de preferințe persistente;
- Conectarea cu servere MCP prin Standard Input-Output (STDIO), Hyper-Text Transfer Protocol (HTTP) sau Server-Side Events (SSE);

- Skill-uri bazate pe fișiere Markdown cu încărcare progresivă;
 - Subagenți specializați extensibili (general, code search, web research), executați în paralel cu modele configurabile per-agent și recursivitate;
 - Proces configurabil de compactare a con-
- textului (trunchiere, sumarizare, deduplicare).
 - Sandbox Docker opțional pentru execuția izolată a comenzilor.
 - Tracing prin Langfuse Cloud și panou web pentru observabilitate.

2.6 Instalare și utilizare

2.6.1 Cerințe preliminare

Aplicația necesită Python 3.10 sau o versiune ulterioară și este disponibilă pe sistemele de operare Linux și macOS; instalarea pe Windows nu este suportată. Pentru accesul la modelele de limbaj este necesar fie un server Ollama local pornit prin `ollama serve`, fie cel puțin o cheie API validă pentru unul dintre furnizorii cloud (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY` sau `GOOGLE_API_KEY`). În lipsa oricărei configurații prealabile, aplicația oferă la pornire un flux interactiv pentru selecția furnizorului și introducerea cheii.

2.6.2 Instalare

Metoda recomandată este executarea script-ului oficial de instalare, care detectează și instalează la nevoie gestionarul de pachete `uv`, după care obține aplicația direct din proiectul găzduit pe GitHub:

```
curl -fsSL https://raw.githubusercontent.com/Moshulika/vibe-cli/main/scripts/install.sh | bash
```

Alternativ, instalarea se poate efectua manual prin `uv tool install` sau `pipx install`, indicând proiectul GitHub ca sursă. Dezinstalarea se face prin `uv tool uninstall vibe-cli`. Pentru dezvoltare, după clonarea proiectului, comanda `make dev-setup` instalează dependențele și interceptoarele `pre-commit`, iar `make ui` lansează interfața de chat. Sunt expuse de asemenea ținte `make` pentru rularea suitei de teste (`make test`), formatarea codului (`make format`) și pornirea panoului analitic web (`make web`).

2.6.3 Configurare la prima rulare

După instalare, comanda `vibe` este disponibilă global din orice director. La prima pornire, aplicația creează arborele de configurație sub `~/.vibe-cli/`, ce conține fișierul de preferințe `preferences.json` (ce conține printre altele: furnizorul implicit, modelul selectat per furnizor, nivelul de efort, cheile API și permisiunile per unealta), un identificator anonim `profile.json`, directorul cu **3** skill-uri exemplificative și jurnale structurate. Furnizorii configurați anterior sunt validați automat printr-un apel către punctul de acces al fiecăruia, fără a consuma tokeni; cheile invalide sunt șterse și semnalate utilizatorului. În absența unei configurații valide, un flux interactiv solicită selecția furnizorului și introducerea cheii API (cu mascarea intrării), validând-o înainte de salvare.

2.6.4 Utilizare

Aplicația suportă 2 moduri de operare: sesiunea interactivă TUI, descrisă mai jos, și modul ne-interactiv, prezentat în secțiunea următoare. După configurare, utilizatorul interacționează cu modelul printr-o sesiune persistentă de chat, salvată automat în evenimente granulare sub directorul corespunzător proiectului curent. Cele 22 comenzi disponibile suportă auto-completare. Sunt disponibile și scurtături de tastatură precum `CTRL+L` pentru a goli conversația și `CTRL+C` pentru închiderea aplicației cu salvarea automată a sesiunii. Pe lângă intrarea textuală, utilizatorul poate și alte tipuri de date, atunci când modelul dispune de capabilități multi-modale. Panoul web analitic poate fi pornit separat prin `vibe web` și oferă vizualizări pentru consumul de tokeni, costuri, latență, evenimente de compactare și activitatea sub-agentilor.

Comanda	Descriere
<code>/help</code>	Listarea tuturor comenzilor și a scurtăturilor de tastatură
<code>/model</code>	Selectarea furnizorului și modelului, sau setarea cheii API
<code>/key</code>	Setarea sau actualizarea cheii API per furnizor
<code>/effort</code>	Selectarea nivelului de efort al modelului
<code>/mcp</code>	Configurarea serverelor MCP (adăugare/ștergere/activare)
<code>/skills</code>	Listarea, activarea sau reîncărcarea skill-urilor
<code>/subagents</code>	Listarea tipurilor de sub-agenti și configurarea modelelor per tip
<code>/plan</code>	Activarea/dezactivarea modului de planificare
<code>/agent</code>	Activarea/dezactivarea modului agentic
<code>/permissions</code>	Vizualizarea și editarea listelor de permisiuni per unealta
<code>/clear</code>	Ștergerea conversației curente și rotirea la o sesiune nouă
<code>/sessions</code>	Listarea sesiunilor salvate pentru proiectul curent
<code>/resume</code>	Continuarea ultimei sesiuni sau a uneia specifice prin ID
<code>/compaction</code>	Inspectarea procesului de compactare și selectarea modelului
<code>/compact</code>	Forțarea unei compactări imediate a contextului
<code>/tracing</code>	Configurarea și activarea tracing-ului Langfuse
<code>/score</code>	Evaluarea ultimei interacțiuni pentru Langfuse (good/bad)
<code>/attach</code>	Atașarea datelor multi-modale de tip imagini, PDF sau fișiere text ca și cale de sistem
<code>/paste</code>	Atașarea datelor multi-modale de tip imagini, PDF sau fișiere text din clipboard
<code>/dataset</code>	Adăugarea ultimei interacțiuni la un dataset Langfuse
<code>/update</code>	Actualizarea la ultima versiune disponibilă în GitHub
<code>/exit</code>	Salvarea conversației și închiderea aplicației

Tabelul 2.2: Comenzi disponibile în interfață

2.6.5 Modul headless

Pe lângă sesiunea interactivă, aplicația expune un mod *headless*, neinteractiv, activat prin parametrul `-p/-prompt` (cu suport pentru citirea promptului din `stdin` prin `vibe -p -`). Acesta execută o singură tură întrebare-răspuns (un apel către model urmat de buclă de procesare a uneltelor) și tipărește rezultatul final, în format text sau `json` (`-o text|json`), după care returnează un cod conform convențiilor: 0 la succes, 1 la eroare de execuție, 2 la argumente invalide și 130 la întreruperea prin `SIGINT`. Modul este *hermetic* în mod implicit: dezactivează MCP, salvarea sesiunii, tracing-ul și interacțiunile cu utilizatorul, cu opțiunea de a salva sesiunea prin `-save-session`. Autentificarea se realizează automat prin detectarea variabilelor de mediu corespunzătoare furnizorilor suportați (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GOOGLE_API_KEY`, `BEDROCK_API_KEY`, `OLLAMA_URL`), scurtcircuitată de flag-ul `-provider`.

Sunt expuse de asemenea opțiuni pentru controlul fin al execuției: selecția modelului (`-model`), reglarea efortului de gândire prin `-effort`, modificarea promptului de sistem (`-system` pentru înlocuire, exclusiv cu `-append-system-prompt` pentru extindere), restricționarea uneltelor disponibile (`-allowedTools`) și limitarea bugetului de iterații (`-max-turns`). Ieșirea `json` include furnizorul, modelul, răspunsul final, lista apelurilor de unelte, skill-urile utilizate, statistici de consum, numărul de iterații și durata execuției, facilitând integrarea în procese automatizate de tip CI/CD sau cu aplicații de tip wrapper ¹.

2.7 Gestionarea contextului și a memoriei

Aplicațiile ce utilizează principii agentice au nevoie de o logica complexa pentru gestionarea contextului și a memoriei din două motive diferite: personalizare și consistenta conversației.

2.7.1 Memoria și istoricul conversației

Aplicațiile concurente folosesc un model comun pentru memorie persistenta: un folder ascuns per proiect (`.claude`, `.copilot`, `.codex`) cu preferințe locale, un folder global pentru preferințe ce țin de utilizator, și un istoric al conversațiilor per proiect cu identificator unic, reintrodus integral în context la reluarea conversației. Memoria se separa în două straturi: *memoria contextuala* (limitata de fereastra de context a modelului, necesitând compactare la depășire) și *memoria integrala* (stocata separat, păstrând istoricul textual și artefactele neafectate).

În implementare am preluat aceste convenții și le-am adaptat: un singur fișier de sesiune cu două câmpuri pentru fiecare eveniment (original și compactat), preferințe locale și globale completate progresiv pe baza interacțiunilor de catre modelul de limbaj (pentru personalizarea raspunsurilor si a actiunilor pe placul utilizatorului, date extrase din conversatiile cu acesta), și stocare exclusiv locala pentru păstrarea intimității.

2.7.2 Contextul

Contextul este unul dintre punctele de interes pentru dezvoltarea sistemelor agentice datorită implicațiilor pe care le are în executarea sarcinilor complexe și de lunga durata sau cu cantități mari de date.

¹O aplicație de tip wrapper este o aplicație care înglobează o alta aplicație și folosește ieșirea acesteia pentru a o prelucra în continuare

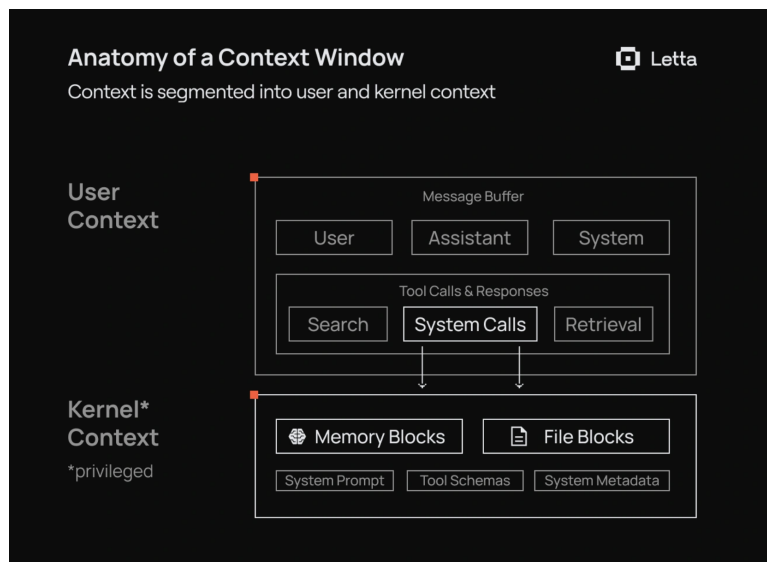


Figura 2.9: Împărțirea ferestrei de context (Sursa: [19])

Mai multe structuri se lupta pentru o parte din fereastra de context:

- Promptul de sistem - instrucțiuni pentru comportamentul agentului. Conține printre altele rolul pe care acesta îl joacă, restricții comportamentale, exemple de răspunsuri și capacități accesibile.
- Mesajele utilizatorului - prompt-ul pe care utilizatorul îl trimite către agent.
- Răspunsurile asistentului - răspunsul oferit către utilizator la cererea acestuia. Întotdeauna contextul declarat (spre ex. 200.000 tokeni) va avea mărimea adevărată $C = C_{\max} - O_{\max}$, unde C este contextul adevărat, $C_{\max}$ contextul declarat, și $O_{\max}$ numărul maxim de tokeni de ieșire suportați de model. Din aceasta valoare se scad și alte valori rezervate de sistem, precum spațiul tampon pentru compactare.
- Atașamente - documente, imagini, audio și videoclipuri
- Skill-uri, sau alte instrucțiuni - descrierea și titlul tuturor skill-urilor sau a rutinelor salvate care pot fi folosite de către agent sunt injectate în context pentru ca agentul să poată alege în cunoscuta de cauza un skill sau o rutina din cele disponibile.
- Unelte - similar skill-urilor, semnătura uneltelor, descrierea acestora, datele de ieșire și instrucțiunile lor sunt disponibile modelului prin injectarea în context. Se aplică atât uneltelor locale sau celor expuse prin MCP.
- Ieșirea uneltelor - datele de ieșire ale uneltelor, spre exemplu conținutul unui fișier rezultat din utilizarea unei unelte pentru citirea fișierelor poate fi injectat direct în context. În cazul fișierelor mari, acest lucru poate însemna o problemă pentru fereastra de context.
- Căutări pe internet - căutările pe internet returnează răspunsuri Java Script Object Notation (JSON) sau Hyper-Text Markup Language (HTML) care uzual nu conțin strict date punctuale, ci și întregul conținut al paginii. Acest lucru ajută pentru poziționarea elementelor în pagina dar adaugă multe caractere ce umplu fereastra de context.

2.7.3 Retrieval Augmented Generation (RAG)

Pentru conservarea contextului în lucrul cu documente de mărime mare, sau cu colecții de documente, a fost adoptată tehnica RAG [86]. Acesta se bazează pe un proces independent care ingestează documentele de interes prin vectorizare și partiționare semantică prin care agentul poate cauta cu ușurință, ajungând astfel să extragă dintr-un document de dimensiuni mari doar bucăți de interes.

Dezavantajul acestei tehnici este complexitatea infrastructurii necesare. În primul rând este nevoie de un serviciu separat care să proceseze și să stocheze aceste date, iar în final modelul trebuie să poată accesa aceste date într-un mod eficient.

Odată cu progresele aduse mărimii acestei ferestre de context, modelele de vârf ajung să aibă până la unul sau două milioane de tokeni ca fereastra de context. Astfel, nevoia pentru RAG scade. Pentru majoritatea întrebunțărilor, aceasta mărime a ferestrei este mai mult decât suficientă, chiar și pentru conversații mai complexe, în ciuda limitărilor cunoscute precum scăderea performanțelor pe date ce se afla în mijlocul ferestrei de context [15].

În implementarea actuală *nu am folosit tehnici RAG*, atât din cauza complexității de implementare și a infrastructurii necesare, cât și a beneficiilor estompate de performanțele curente ale modelelor de limbaj cu ferestre mari de context.

2.7.4 Gestionarea datelor multi-modale

Data fiind alegerea de abandonare a tehnicii RAG, în implementare avem nevoie de o modalitate inteligentă de a prelucra datele multi-modale pentru conservarea contextului. Arhitectura acestei capabilități poate fi consultată în figura 2.10.

Prima problemă în lucrul cu date multi-modale apare din cauza necesității pentru suportul unei plaje foarte largi de modele de limbaj. Din cauza arhitecturii agnostice de furnizor care permite folosirea oricărui model, avem nevoie să detectăm capabilitățile de prelucrare a tipurilor de fișiere pentru fiecare model în parte. Din păcate, furnizorii de modele sau creatorii care le includ în Ollama, nu specifică nicăieri consistent în metadatele modelului dacă acesta prezintă capabilități pentru lucrul cu anumite tipuri de fișiere.

Din acest motiv, am avut de ales între două soluții de implementare:

- Punerea utilizatorului în situația în care trebuie să se documenteze și să declare în configurație capabilitățile modelului pe care dorește să îl folosească;
- Încercarea automată de trimitere de date multi-modale către model, iar în cazul în care acesta refuză datele, să bifăm în fișierul de preferințe faptul că acest model nu suportă respectivul tip de date.

Din motive legate de experiența utilizatorului am ales să continuăm cu a doua soluție.

După decizia legată de detectarea capabilităților am trecut la gestionarea contextului pentru aceste date. Am proiectat o abordare hibridă, compusă din două ramuri pe care agentul poate alege să le apeleze. În prima fază, pentru fișierele de tip text sau PDF, agentul poate citi fișierele în format pur textual folosindu-se de uneltele uzuale de citit fișiere, beneficiind de optimizările acestora, precum folosirea punctuală în zone de interes sau trunchierea datelor de ieșire. Dacă modelul consideră că are nevoie de o vedere completă, atunci când suportă fișierul PDF ca tip de date pentru intrare, se poate uita direct pe întregul fișier prin mai multe citiri succesive de maxim 20 de pagini (configurabil). Imaginile sunt introduse direct în context dacă modelul suportă acest tip de date și sunt limitate la o dimensiune de 10 MB.

Compactarea acestor date este realizată prin metode ce rulează imediat după folosirea uneltilor (trunchiere, referențiere sau sumarizare), fișierele fiind stocate automat ca artefacte recuperabile.

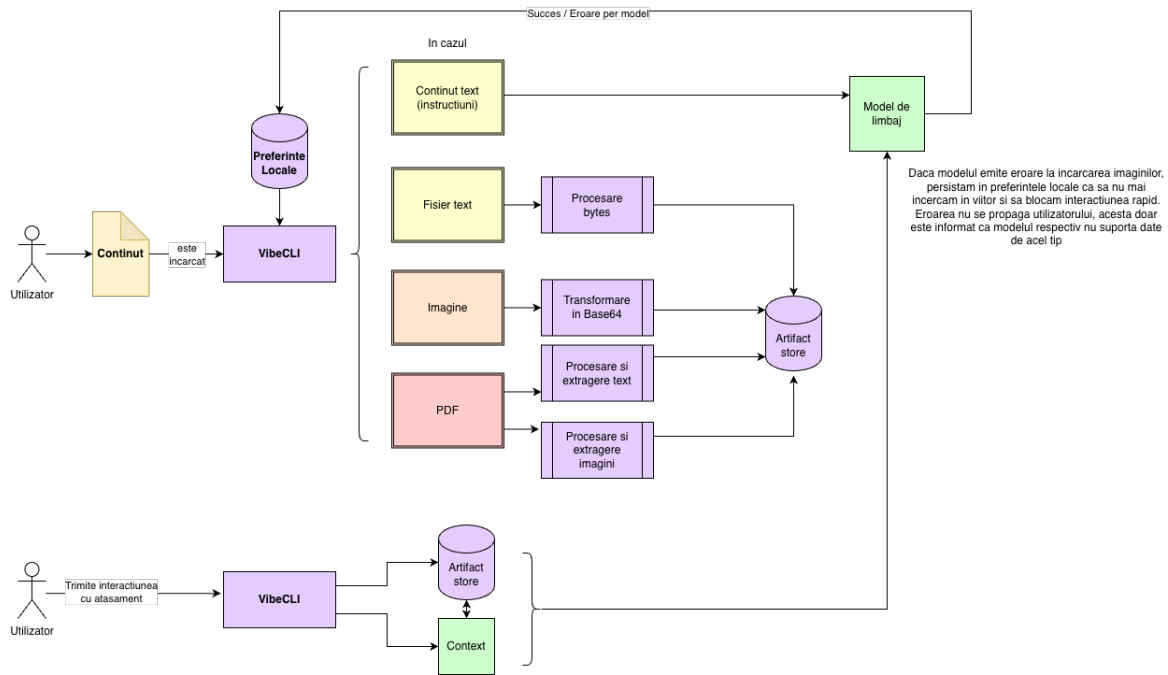


Figura 2.10: Fluxul de prelucrare a datelor multi-modale în vibe-cli

2.7.5 Compactarea

Compactarea conversației este un cumul de tehnici ce asigura compatibilitatea conversației curente cu restricțiile de intrare ale modelului. Aceasta este folosita predominant în sistemele agentice, deoarece aici se pot genera cantități mari de date ce epuizează fereastra de context rapid.

Compactarea are loc uzual la produsele existente în piață în mod automat când conversația atinge în jur de 80% din mărimea maxima a ferestrei de context pentru modelul respectiv (figura 2.11). Acest lucru oferă o margine de eroare pentru a evita pornirea procesului de compactare până nu se termina interacțiunea curentă.²

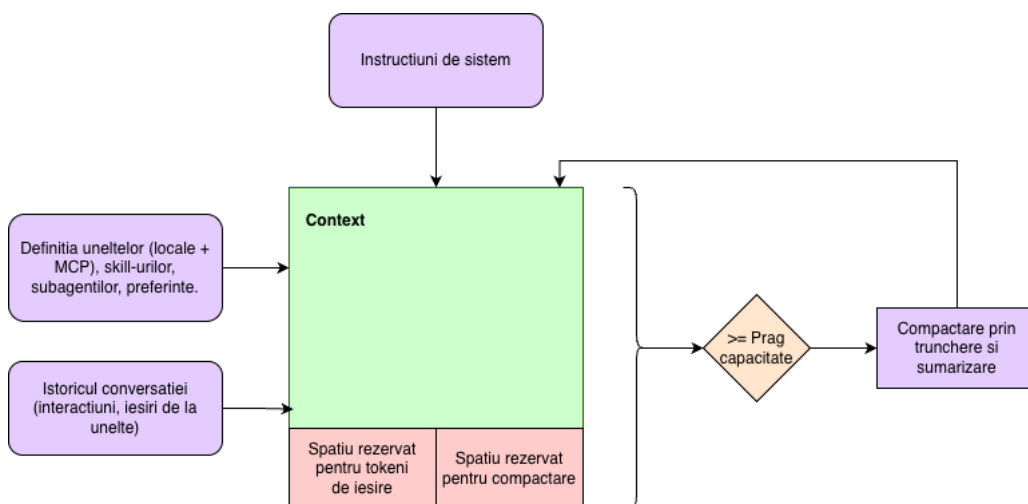


Figura 2.11: Schema componentelor care alcătuiesc fereastra de context și compactarea la prag

²O interacțiune reprezintă întrebarea utilizatorului și toate procesele pe care le execută agentul până la producerea răspunsului final de către modelul de limbaj

Tehnicile uzuale se împart în transformări locale pe ieșirea uneltelor (formatare deterministă, trunchiere cap-coadă, sumarizare prin LLM, substituția cu referințe în artefacte externe) și transformări globale pe întreaga conversație (deduplicare prin scor de similaritate, eliminare de mesaje în funcție de tip, sumarizarea sau trunchierea integrală a istoricului). Primele se execută imediat după rularea uneltei (figura 2.12), ultimele doar la atingerea pragului de compactare (figura 2.11).

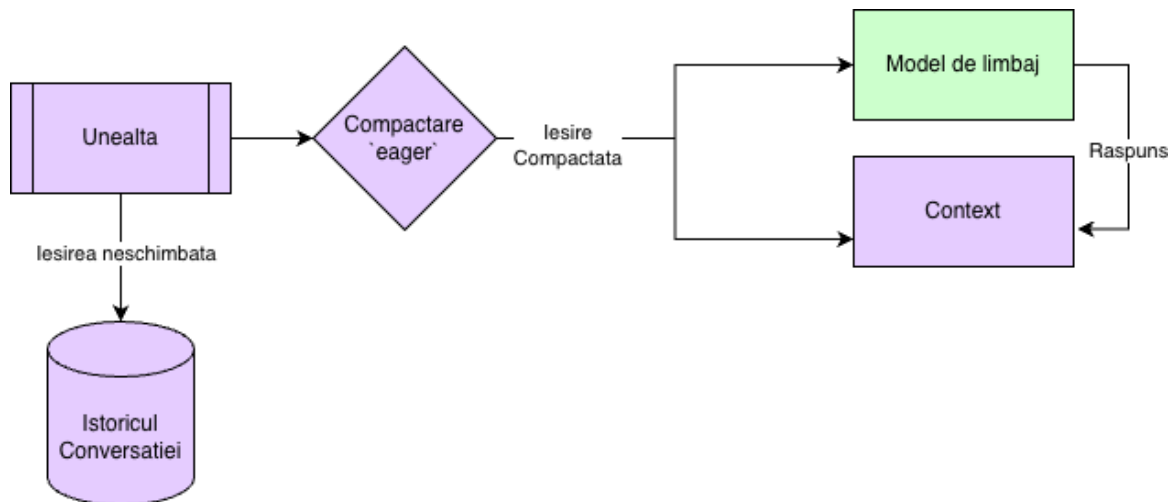


Figura 2.12: Compactarea după apelul uneltei

Spre deosebire de produsele existente, în care aceste setări sunt fixe, am ales **configurabilitatea completă** pentru studierea efectului fiecărei strategii asupra coerenței agentului; configurarea completă este furnizată în anexa 4.5.

Procesul de compactare este pornit la atingerea valorii de prag sau după fiecare apel de unealta, iar pașii activați execută succesiv transformări locale sau globale; pașii ce necesită LLM pot folosi modelul principal sau unul mai mic. Cele 7 stadii expuse sunt: **tool_truncate** (trunchiere cu `max_tool_tokens`, capete configurabile), **tool_summarize** (sumarizare în loc de trunchiere), **format_compress** (formatare per unealta cu prag), **ref_substitute** (extragere în artefact extern peste un prag de tokeni), **dedupe** (prag de similaritate și metrica), **importance_prune** (liste de tipuri păstrate/șterse) și **bulk** (sumarizare globală cu mai multe strategii). Stadiile sunt fie *eager*, fie declanșate la prag, iar unele perechi sunt incompatibile.

2.7.6 Tehnici pentru eficientizarea ferestrei de context

Pe lângă compactare, două tehnici complementare reduc consumul de context: utilizarea inteligentă a uneltelor (comenzi terminal cu `head/tail`, sau căutări punctuale prin `grep/find`) și pornirea de subagenți care preiau cercetarea sau executarea unor sub-sarcini și returnează doar un rezumat, păstrând curat contextul orchestratorului. Ambele tehnici sunt folosite în implementare și detaliate în secțiunea următoare.

2.8 Componente ale arhitecturilor agentice moderne

Pentru a avea un flux de lucru agentic, modelul de limbaj trebuie să fie adaptat pentru a urma instrucțiunii ce îl ghidează în a folosi diverse instrumente, capabilități predefinite sau pentru interacțiunea cu o echipă de agenți.

2.8.1 Unelte

Inima arhitecturii agentice este reprezentata de uneltele pe care modelele de limbaj învață să le utilizeze. Acestea nu reprezintă nimic mai mult decât metode uzuale ale limbajului de programare cu precizarea ca modelul de limbaj are acces la semnătura acestora (gruparea de nume al funcției, parametrii de intrare, date returnate) împreună cu o descriere detaliată de utilizare, un exemplu fiind prezent în figura 2.13.

```
1 @tool
2 def list_directory(path: str = ".", show_hidden: bool = False) -> str:
3     """List files and directories at the given path.
4     Args:
5         path: Directory path to list (default: current directory).
6         show_hidden: Whether to include hidden files (default: false).
7     """
8     try:
9         p = Path(path).expanduser().resolve()
10        if not p.exists():
11            return f"Directory not found: {path}"
12        if not p.is_dir():
13            return f"Not a directory: {path}"
14        entries = sorted(p.iterdir(), key=lambda e: (not e.is_dir(), e.name.lower()))
15        lines = [f"Contents of {p}:", ""]
16        for entry in entries:
17            if not show_hidden and entry.name.startswith("."):
18                continue
19            prefix = "[Folder] " if entry.is_dir() else "[File] "
20            size = ""
21            if entry.is_file():
22                s = entry.stat().st_size
23                if s < 1024:
24                    size = f"({s}B)"
25                elif s < 1_048_576:
26                    size = f"({s / 1024:.1f}KB)"
27                else:
28                    size = f"({s / 1_048_576:.1f}MB)"
29            lines.append(f"{prefix}{entry.name}{size}")
30        if len(lines) == 2:
31            lines.append(" (empty)")
32        return "\n".join(lines)
33    except PermissionError:
34        return f"Permission denied: {path}"
35    except Exception as e:
36        return f"Error listing directory: {e}"
```

Figura 2.13: Implementarea unei funcții pe care agentul o folosește pentru a lista directoriile și fișierele la o anumită cale pe sistem

Aceste unelte sunt alese cu grija pentru a menține un echilibru între numărul total de unelte și suita de capacități necesare agentului.

Numărul de unelte afectează direct modul de gândire al modelului și performanța în execuția sarcinilor complexe. Cu prea puține, acesta poate întâmpina probleme în execuția problemelor cu adevărat sofisticate deoarece îi poate lipsi o capacitate necesară. Pe de altă parte, dacă are prea multe acesta nu se va putea decide asupra unei singure unelte, iar în lipsa unei descrieri detaliate crește riscul de folosire greșită sau ineficientă.

De asemenea, pentru o eficiență sporită, scopul acestora nu trebuie să se suprapună. Uneltele trebuie să fie unice, și să aibă un scop clar. Mai mult, este recomandată exemplificarea cazurilor în care o unealtă ar trebui folosită, evitată sau preferată în detrimentul alteia. Câteva dintre capacitățile utilizate frecvent de agenți sunt:

- Listarea directoarelor și fișierelor
- Citirea fișierelor
- Crearea fișierelor cu sau fără conținut
- Editarea fișierelor
- Căutarea pe internet
- Execuția comenzilor în terminal
- Utilizarea skill-urilor
- Pornirea subagenților

Utilizarea acestor instrumente trebuie îngăduită totuși de noțiuni logice. Execuția acestora este determinată de șiruri logice de evenimente care trebuie îndeplinite anterior execuției. Spre exemplu, un agent nu ar trebui să scrie într-un fișier dacă nu l-a citit înainte, deoarece acesta nu are contextul acelui fișier și poate returna lucruri care nu sunt ancorate în realitate (figura 2.14). Acest lucru este implementat cu ajutorul cadrului de lucru LangGraph printr-un interceptor³ ce este apelat înaintea sau la finalul execuției unei unealte. Rezumându-ne la exemplul cu unelele de citire și scriere din fișiere, aceste interceptoare salvează un SHA⁴ al fișierului citit în memorie, pe care mai apoi unealta de scriere a fișierului îl compara cu SHA-ul calculat individual. Dacă aceste două coduri coincid, rezulta că fișierul citit este exact cel care urmează să fie scris, astfel este garantată integritatea contextului înaintea scrierii. Configurarea interceptoarelor de gardare este detaliată în anexa 4.5. Agentul are acces doar la umplerea parametrilor funcției cu valori arbitrare, dar nu are control asupra logicii efective a unealtei.

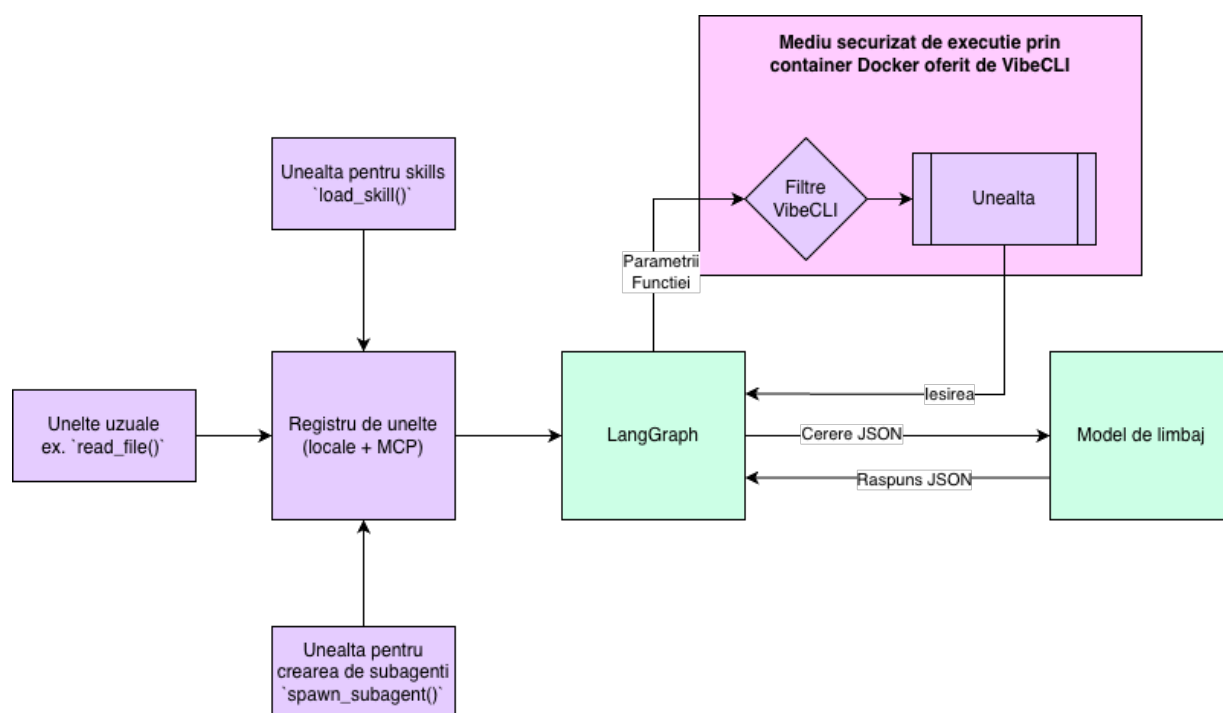


Figura 2.14: Suita de unelte expuse agentului în `vibe-cli` și relațiile dintre acestea

Opțiunea de a configura aceste moduri de protecție revine utilizatorului ce poate porni sau opri aceasta funcționalitate. Fiind o aplicație OSS, dacă utilizatorul este și dezvoltator

³Un interceptor este un punct de interceptare a execuției unei funcții care permite inserarea altor blocuri logice ce sunt rulate în momentul apelării acestuia

⁴Codarea SHA este cea mai comună metoda de verificare a integrității unui fișier: orice mică modificare a fișierului schimbă complet codul, fără a fi necesară citirea integrală a acestuia.

software, acesta poate modifica codul sursa și extinde logica de gardare cu mai multe reguli ce pot fi oprite sau pornite din aceasta secțiune.

Agentul trebuie să utilizeze în mod inteligent aceste unelte. Pe lângă rularea logica a secvenței de instrumente, acesta trebuie să știe cum poate folosi de ele la potențialul maxim. Un bun exemplu pentru acest argument este execuția de cod scris de agenți în terminal. Pentru opțiuni repetitive care ar consuma o buna parte din context, modelele sunt antrenate să scrie porțiuni mici de cod, uzual în limbajul Python sau Bash datorită simplității și a disponibilității crescute. Astfel, agentul va scrie aceste bucăți de cod și le va executa în terminal pentru a realiza acțiuni complexe în doar câteva linii de cod, sporind eficiența cercetării și a testării. Aceasta utilizare este fructificată de promptul de sistem care instruește agentul, dar și din procesul de antrenare.

Antrenarea pentru scrierea, folosirea și execuția de cod a devenit o prioritate pentru agenți, modelele de limbaj fiind antrenate explicit folosind RL să folosească secvențe de cod pentru facilitarea rezolvării problemelor care ar beneficia după urma unei astfel de abordări. Aceasta logica este utilizată și pentru testarea de baza a ideilor și explorarea acestora înainte de implementarea propriu zisă, agentul rulând un mic script demonstrativ în terminal pentru a dovedi ca principiul pe care se bazează idea ce urmează a fi implementată este valid.

2.8.2 Modul de planificare

Planificarea sarcinilor complexe folosește principiul Chain of Thought [12]. Astfel, execuția unei sarcini complexe este împărțită în două părți: planificare și execuție.

În prima fază, agentul realizează cercetarea necesară pentru implementare și documentează totul într-un fișier. Mai apoi, se va folosi de contextul din acel fișier pentru a ține minte toate punctele pe care trebuie să le bifeze. Multe implementări împart planul în câteva secțiuni bine delimitate printre care se regăsesc: ideea și descrierea planului, fișiere afectate, modificări punctuale și cel mai important niște căsuțe pe care acesta trebuie să le bifeze, împărțindu-și o execuție lungă în mai multe sub-sarcini mai mici, individuale și de complexitate redusă.

Acest mod de planificare nu este nimic mai mult decât o schimbare de prompt de sistem într-o combinație cu o antrenare de tip RL care recompensează crearea de planuri detaliate și consistente în situații care necesită așa ceva.

Modul de planificare reprezintă materializarea SDD în contextul programării de către sisteme agentice. O altă abordare folosită în aceste sisteme este aceea de TDD, care permite mai întâi experimentarea și scrierea de cod fără o documentație bine pusă la punct, dar testarea amănunțită la final.

2.8.3 Skill-uri

Skill-urile sau capacitățile nu sunt decât un set predefinit de instrucțiuni pentru realizarea punctuala a unei sarcini repetitive și sunt compuse din două componente care se afla într-un fișier de tip Markdown pentru formatare augmentată.

Prima componentă este în format YAML, și reprezintă date referitoare la skill, precum numele, descrierea, o recomandare pentru cazuri în care utilizarea acestuia este dorită, și un steag ce setează activarea acestei capacități. Aceste date sunt disponibile agentului și sunt inserate în contextul tuturor conversațiilor pentru ca modelul să aibă acces la lista din care poate alege.

A doua componentă este reprezentată de capacitatea propriu zisă. Aceasta este formată din instrucțiuni scrise în limbaj natural. De obicei aceste instrucțiuni sunt structurate în pași bine definiți, cu reguli și exemple.

```

1 ---
2 name: pr-review
3 description: Structured pull-request review - risk surface, testing, naming, error handling.
4 when_to_use: When the user asks for a code review, PR review, or feedback on a diff.
5 enabled: false
6 ---
7 You are now performing a pull-request review. Walk through the diff and respond with these
  sections, in this order. Skip a section only if there's truly nothing to say there.
8
9 1. **Summary** - one short paragraph: what does this change do, in plain language.
10 2. **Risk surface** -- what could break in production. Concurrency, data migrations, error
   paths, rollback story, blast radius.
11 3. **Correctness** -- bugs, off-by-ones, race conditions, mishandled None/empty cases, wrong
   types, security issues (injection, auth, secrets).
12 4. **Tests** -- is the change covered. Are the tests testing the right thing. Any missing edge
   cases.
13 5. **Style & naming** -- readability nits only if they actually hurt comprehension. Don't
   bikeshed.
14 6. **Suggestions** -- concrete proposed changes, ideally with a snippet.
15
16 Rules of engagement:
17 - Be specific: cite file paths and line numbers.
18 - Distinguish blockers from nits - say so explicitly.
19 - If the diff is too large to review thoroughly, say so and pick the riskiest files first.
20 - No fluff, no praise sandwich. The reader is technical and time-constrained.

```

Figura 2.15: Implementarea unei capabilitati de revizuire de PR

Aceste capabilități permite execuția consistentă și reproductibilă (deoarece agentul primește același prompt de fiecare dată) a unui flux de execuție dorit în rezolvarea unei sarcini.

2.8.4 Subagenți

Echipa de agenți constă dintr-un orchestrator cu context integral și mai mulți subagenți specializați cu context limitat la sub-sarcină, care returnează doar rezumate (figura 2.16). Un subagent este efectiv un alt apel LLM cu personalitate diferită, set restrâns de unelte și prompt de sistem dedicat ce specifică și formatul de răspuns așteptat. Subagenții folosesc uzual modele mai puțin performante, deci compactarea contextului este la fel de necesară. Orchestratorul îi pornește printr-o unealtă LangGraph (figura 2.17) și îi monitorizează pentru metrici.

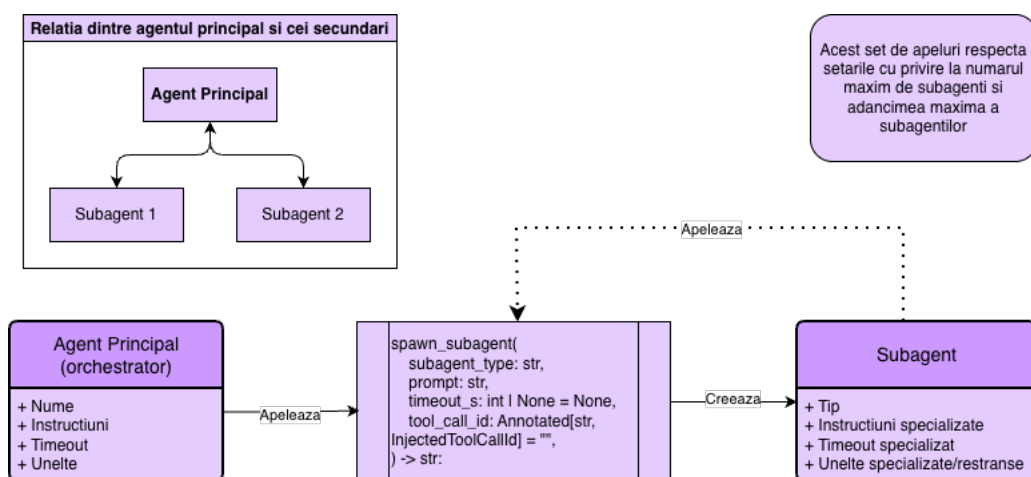


Figura 2.16: Arhitectura orchestrator-subagenți din vibe-cli

```

1 @tool
2 async def spawn_subagent(
3     subagent_type: str, prompt: str, timeout_s: int | None = None,
4     tool_call_id: Annotated[str, InjectedToolCallId] = "",
5 ) -> str:
6     """Spawn a specialised subagent to handle a focused sub-task in isolation.
7     The subagent runs its own tool-calling, then returns a single final report.
8     You only see that report. Use this to to delegate a deep dive without
9     polluting your own context.
10    Pick a 'subagent_type' that matches the work: 'general' - open-ended research,
11    'code_search' - read-only code lookups, 'web_research'
12    Args:
13        subagent_type: One of the registered types above.
14        prompt: The task description for the subagent. Be specific: it has
15            none of your conversation context.
16        timeout_s: Optional hard timeout in seconds. Defaults to the type's
17            built-in default; capped by config.
18    """
19    parent_ctx = _ctx()
20    result = await run_subagent(
21        subagent_type=(subagent_type or "").strip(),
22        prompt=(prompt or "").strip(),
23        parent_ctx=parent_ctx,
24        timeout_s=timeout_s,
25        parent_tool_call_id=tool_call_id or None,
26    )
27    if tool_call_id:
28        parent_ctx.setdefault("pending_subagent_results", {})[tool_call_id] = {
29            "subagent_type": (subagent_type or "").strip(),
30            "prompt": (prompt or "").strip(),
31            "text": result.text,
32            "iterations": result.iterations,
33            "tool_call_count": result.tool_call_count,
34            "timed_out": result.timed_out,
35            "error": result.error,
36            "duration_s": result.duration_s,
37            "usage": result.usage,
38        }
39    return result.text

```

Figura 2.17: Implementarea unei funcții pe care agentul o folosește pentru a porni un subagent

Niciun produs comercial nu permite configurarea în detaliu a subagenților; implementarea noastră permite **configurarea completă** per subagent: model, prompt, posibilitatea de a porni subagenți proprii, număr maxim de iterații, adâncime maxima și timp maxim de așteptare (anexa 4.5). Aplicația furnizează trei subagenți predefiniți (general, cercetare cod, cercetare web), executați de obicei în paralel (cu limita de execuție configurabilă), fiecare cu setul propriu de unelte, instrucțiuni și o setări proprii pentru gestionarea răspunsurilor de la unelte.

2.9 Extensibilitate

Deoarece aplicația este gândită pentru a fi open-source, arhitectura aplicației trebuie să fie ușor extensibilă de utilizatori ce doresc să aducă modificări în modul de funcționare, în special în punctele de extindere. Câteva din aceste puncte de extindere gândite de la bun început sunt: logica de compactare, setul de unelte, setul de subagenți, setul de furnizori și setul de skill-uri.

2.9.1 MCP și adăugarea de unelte

Aplicația conține un client MCP pentru adăugarea uneltelor externe și integrarea cu servicii terțe. Acesta accepta cele 3 metode de comunicare comune în industrie: STDIO (capturarea ieșirii pentru procese locale), HTTP și SSE (pentru servere). Pentru a configura un server MCP și a accesa instrumentele pe care le pune la dispoziție în aplicație, utilizatorul

poate folosi comanda `/mcp` în interiorul interfeței, sau poate adăugă personal serverul în fișierul `/.vibe-cli/preferences.json` și să repornească aplicația.

Serverele MCP pot expune atât unelte cât și skill-uri sau prompt-uri. Acestea pot fi reîncărcate sau gestionate prin comanda `/mcp`

Adăugarea de unelte poate fi făcută cu ușurință, fiind o aplicație open-source. Dacă utilizatorul este un dezvoltator și a clonat proiectul local, acesta poate adăugă un instrument nou după modelul din figura 2.13 și să își implementeze propria logica. Uneltele sunt definite la calea `src/tools.py`. După adăugarea uneltelor, aplicația trebuie reinstalată/repornită.

2.9.2 Adăugarea de subagenți

Adăugarea unui subagent presupune crearea unui fișier în `src/subagents/types` conform modelului din figura 2.18: extinderea clasei `Subagent` și completarea câmpurilor de bază (nume, descriere, rol), urmate de înregistrarea în `config.yml` și repornirea aplicației.

```
1 @register
2 class CodeSearchSubagent(Subagent):
3     name = "code_search"
4     description = (
5         "Fast read-only code search. Finds files/functions/symbols and "
6         "reports paths + line ranges + relevant snippets. Cannot write or "
7         "execute side-effectful commands."
8     )
9     default_timeout_s = 90
10    max_iterations = 15
11    # Read-only toolset. 'run_command' is included for grep/rg/find use.
12    allowed_tools = ["read_file", "list_directory", "run_command", "fetch_artifact"]
13
14    def role_prompt(self, prompt: str, parent_ctx: dict) -> str:
15        return (
16            "ROLE: read-only code-search subagent.\n\n"
17            "Your job is to LOCATE code that matches the orchestrator's "
18            "request and report findings concisely. You are an advisor only "
19            "- even though 'run_command' is available, it is for read-only "
20            "searches (rg / grep / find / ls / cat / wc). NEVER invoke "
21            "installers, builds, test runs, package managers, or any "
22            "command that mutates state.\n\n"
23            "Stop searching once you have enough to answer; do not enumerate "
24            "exhaustively.\n\n"
25            "Final report format:\n"
26            "  - For each match: 'path:line' + a one-line description.\n"
27            "  - When useful, include a short snippet (<=6 lines).\n"
28            "  - End with a one-sentence takeaway for the orchestrator."
29        )
```

Figura 2.18: Înregistrarea unui nou subagent

Definiția unui subagent este o subclasă Python a clasei `Subagent` înregistrată prin decoratorul `@register`, expunând puncte de extensie precum: `role_prompt` (fragment de rol cu acces la contextul părintelui), `allowed_tools/excluded_tools` (intersectate cu uneltele active), interceptoare pentru recalcularea uneltelor și a prompt-ului per invocare, `max_iterations` și `default_timeout_s` (suprascrise din `config.yml`), și model dedicat configurabil prin `/subagents`.

La execuție, fiecare subagent rulează o buclă completă de apelare de unelte cu același flux de permisiuni și `tool_guards` ca părintele (de ex. *read-before-write* prin partajarea `ctx["file_reads"]`), beneficiază de compactare locală a `ToolMessage`-urilor mari, telemetrie Langfuse în span-ul părintelui, agregarea tokenilor și respectă limitele `max_depth/max_parallel`; dacă iese din buclă fără raport final, este reapelat o singură dată pentru emiterea raportului.

Modelul actual nu suporta definiții declarative bazate pe fișiere Markdown, delimitarea serverelor MCP per subagent, flux continuu intermediar al ieșirii și nici politici de *reîncercare/replanificare* dincolo de `max_iterations`.

2.9.3 Creare de skill-uri si personalizare

Crearea de skill-uri este reprezentata de scrierea unui fișier de tip Markdown și de plasarea acestuia fie în containerul global de skill-uri aflat la `/.vibe-cli/skills`, fie în folderul `.vibe` din cadrul proiectului în care se lucrează curent pentru skill-uri locale. Pentru aplicarea schimbărilor sau gestionarea skill-urilor, utilizatorul poate folosi comanda `/skills`

Fișierele de personalizare disponibile local, per proiect sau global pot fi scrise atât de agenți (pentru a nota preferințele utilizatorului extrase din conversație) sau pot fi modificate chiar de utilizator. Un exemplu bun de astfel de preferință utilizată în sarcinile de dezvoltare software ar fi specificarea modului de testare: "La sfârșitul implementării rulează întotdeauna testele unitare și de integrare".

2.9.4 Adăugarea tehnicilor de compactare

Procesul de compactare este un sistem de extensii înmagazinate într-un registru. Fiecare strategie este o subclasă a **Stage** (*eager*, per eveniment) sau **ThresholdStage** (la atingerea pragului, cu vizibilitate globală), declarată în `src/compaction/stages/` și înregistrată în `src/compaction/__init__.py`. Adăugarea unei strategii presupune: implementarea unor metodelor clasei pe care o extinde, adăugarea în registru, opțional înscrierea în lista de excludere mutuală (dacă e incompatibilă cu alta) și introducerea în `config.yml`.

Strategiile asincrone bazate pe LLM folosesc un protocol în două etape: o rulare sincronă produce un rezultat sigur (`truncated_pending`), iar o sarcină de fundal îl înlocuiește ulterior cu varianta rezumată de modelul de limbaj; în caz de eșec, starea trece în `truncated_fallback`. Câmpul `event["content"]` rămâne imutabil, iar fișierul de sesiune este copiat înainte de fiecare trecere, garantând reversibilitatea.

Pe baza acestui mecanism, candidații naturali pentru extindere sunt: memorarea apelurilor de unealtă prin perechi (nume, argumente normalizate); proiecția diferențială a citirilor repetate (dacă un fișier este citit de 2 ori, se salvează doar diferența între cele 2 stări observate); fereastră glisantă de fapte ancorate persistente; proiectori structurați per unealtă (extractori declarativi care păstrează doar câmpurile consumate de agent). Strategiile bazate pe notare cu LLM ca arbitru nu au constituit o prioritate momentan, fiind acoperite determinist de `importance_prune` și `bulk.hybrid`.

2.10 Securitatea în mediul agentic

Încredințarea controlului unui agent cu acces la comenzi periculoase necesită măsuri de siguranță împotriva acțiunilor nedorite. Studiul recent al amenințărilor specifice sistemelor agentice [87] le grupează în trei clase:

1. *prompt injection* indirect prin conținut ingerat din fișiere, pagini web sau ieșirea uneltelor, acestea fiind tehnici de tipul Adversarial Machine Learning (AML) [88];
2. *utilizarea greșită a uneltelor*, prin halucinarea argumentelor sau compunerea unor secvențe inofensive individual dar distructive în ansamblu (`rm -rf` pe cai eronate);
3. *data poisoning* la nivelul memoriei persistente, prin plantarea de fapte false reluate ulterior ca autoritare.

La aceste amenințări se adaugă până și frustrarea agentului, care în unele cazuri decide ca cel mai ușor mod de a scăpa de o eroare este de a șterge teste unitare sau funcționalitatea cauză a erorii.

Întrucât aliniamentul modelului [89] și filtrele de tip *guardrail* [90], [91] nu oferă garanții suficiente în fața acestor vectori, am proiectat o tehnică de apărare pe trei niveluri: izolarea execuției prin sandboxing, restricționarea acțiunilor prin permisiuni explicite și delegarea către subagenți cu privilegii reduse. De asemenea și furnizorii de model includ protecții de intrare/ieșire a modelelor (pentru a limita ingestia sau returnarea de date nocive), dar și modelele sunt antrenate (în unele cazuri) contra acestor lucruri.

2.10.1 Sandboxing

Sandboxing-ul izolează comenzile de shell ale agentului într-un container Docker dedicat sesiunii, cu directorul utilizatorului montat prin `bind-mount` la `/workspace` pentru propagarea directă a modificărilor. Funcționalitatea este opțională și utilizează automat execuția pe gazdă când Docker nu este disponibil.

Configurația (anexa 4.5) acoperă imaginea de bază, modul de rețea, limitele de memorie/Central Processing Unit (CPU), timpul maxim, cai interzise și prag pentru directoare ambigue. La pornire, aplicația aplică o politică de validare pe trei niveluri: refuz pentru căi sensibile (`/`, `/tmp`, `$HOME` și subdirectoarele directe), acceptare pentru directoare cu marcaje de proiect (`.git`, `pyproject.toml`) și validare pe bază de dimensiune/număr de fișiere pentru ambigue, suprascrisa de `VIBE_SANDBOX_FORCE`.

Containerul rulează cu `docker run -d -rm -cap-drop ALL -user <uid>:<gid>`, trăiește pe toată sesiunea și servește comenzile prin `docker exec`. În caz de eșec, modulul nu propaga excepția, iar orchestratorul continuă pe gazdă. Rutarea apelurilor se face înapoi prin `run_command`; uneltele de fișiere rămân pe gazdă (echivalente prin `bind-mount`, cu latentă redusă). Când sandbox-ul e activ, este expusă unealta și `run_command_host` ca metoda condiționată de permisiuni de a executa comenzi pe masina gazda. Subagenții moștenesc aceste setări prin context, beneficiind uniform de izolare.

2.10.2 Permisii

Aprobarea execuției uneltelor de către utilizator este o măsură de apărare necesară: uneltele de citire sunt aprobate din oficiu, dar pentru restul utilizatorul are întotdeauna ultimul cuvânt și poate aproba fiecare comandă în parte, toate comenzile de un tip pentru sesiunea curentă sau permanent, primind ca informație unealta și argumentele acesteia (figura 2.19).



Figura 2.19: Cererea de aprobare a permisiunii de a scrie un fișier

Restricțiile se aplică atât agentului orchestrator cât și subagenților.

2.10.3 Guardrails, intenții periculoase și cereri în afara scopului (Out of Scope (OOS))

O aplicație în producție trebuie să răspundă doar la cereri din scopul declarat, lucru greu de garantat cu un sistem nedeterminist. Tehnicile comune combina instrucțiuni de prompt, antrenament dedicat, liste de cuvinte interzise sau apel către un model extern de validare; furnizorii oferă și propriile măsuri. În aplicație nu am implementat astfel de filtre, lăsând acest aspect în sarcina furnizorilor de modele de limbaj care au deja implementate astfel de soluții.

2.11 Observabilitate

Am proiectat observabilitatea aplicației pe două niveluri complementare: tracing extern, prin integrarea cu Langfuse Cloud, și introspecție locală, prin panoul web analitic livrat împreună cu aplicația. Ambele componente sunt opționale, dezactivate implicit, și nu transmit date către alte servicii în afara celor configurate explicit de utilizator.

2.11.1 Langfuse

Pentru tracing extern, aplicația integrează Langfuse Cloud printr-o interceptare a grafului agentului. Activarea se face din `config.yml` sau prin comanda `/tracing setup`, care persistă cheile API în `preferences.json`. Precedența de rezolvare: preferințe locale, configurație, variabile de mediu `LANGFUSE_*`.

La fiecare invocare, evenimentele sunt publicate asincron către Langfuse ca un trace⁵ principal cu span-uri⁶ ierarhice per apel LLM, unealtă și nod de graf. Pentru fiecare span se înregistrează numele modelului, intrările/ieșirile, consumul de tokeni și latentă. Trace-urile sunt etichetate cu identificator de sesiune (Universally Unique Identifier (UUID)v7), utilizator anonim și tag-uri (mod activ, furnizor, model), permițând agregarea și filtrarea ulterioară.

Pentru bucla de feedback, două comenzi operează pe ultimul trace: `/score` atașează scoruri numerice sau calitative, iar `/dataset` adaugă interacțiunea într-un set de date persistent pentru evaluare. Combinate cu mecanismul *LLM-as-judge* oferit de Langfuse, ele formează fundamentul unei suite de monitorizare a calității.

⁵Un trace este un mesaj JSON folosit în monitorizarea distribuită; conține detalii și poate îngloba mai multe span-uri.

⁶Un span reprezintă o unitate de observare a unei singure acțiuni.

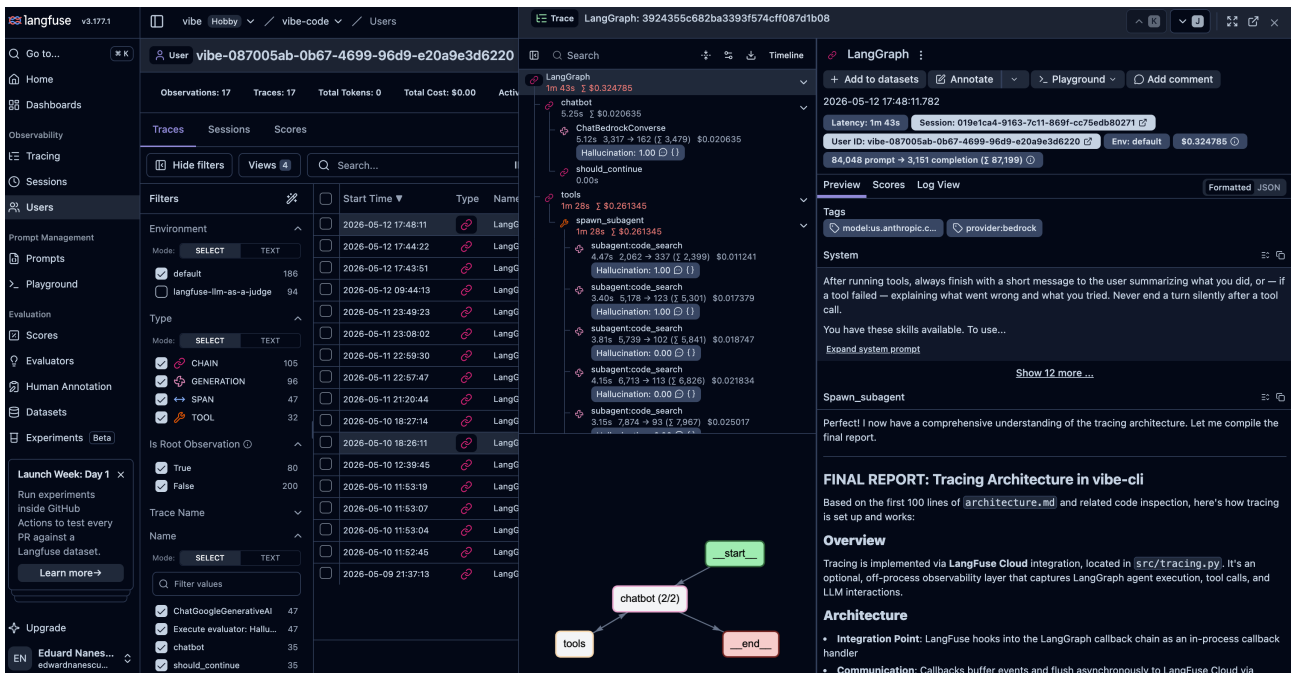


Figura 2.20: Interfața Langfuse

Figura 2.20 prezintă interfața Langfuse: **stânga** conține navigarea (trace-uri per sesiune și utilizator, managementul versionat al prompt-urilor, rubrica de calitate cu evaluatori automați și verificare umană); **centrul** arată meniul *Users* cu sesiunile și trace-urile filtrabile; **dreapta** expune trace-ul deschis, cu graficul de execuție pas cu pas, apelurile uneltelor și răspunsurile, gândirea modelului, istoricul augmentat și metricile per tokeni, plus evaluările automate configurate.

2.11.2 Panou web

Panoul web este o aplicație Flask locală, *read-only*, care vizualizează direct fișierele de sesiune din `~/.vibe-cli/sessions/`, fără bază de date intermediară. Se lansează prin *vibe web*, expusă implicit pe `http://127.0.0.1:5050`, cu adresa și portul configurabile.

Interfața (o singură pagină) oferă selectoare pentru proiect și sesiune și afișează sumarul consumului de tokeni pe categorii de evenimente, evoluția dimensiunii contextului raportată la limita modelului, frecvența și latentă apelurilor către unelte (cu p_{50} , p_{95}), debitul de generare în tokeni/secundă, costul estimat per apel pe baza unui tabel de prețuri configurabil și istoricul compactărilor cu diferențele aferente. Componente dedicate permit inspectarea granulară a fiecărei unelte și reconstrucția turului conversațional eveniment cu eveniment.

La nivel înalt, panoul deschide fiecare sesiune cu o fișă de sinteză ce reunește, într-un singur ecran, furnizorul și modelul folosite, numărul de tururi și de evenimente, totalul de tokeni consumați, utilizarea de vârf a contextului raportată la limita modelului, durata sesiunii și costul estimat alături de prețul de referință (\$/1M tokeni) (figurile 2.21, 2.23, 2.22). Aceasta privire de ansamblu este completată de o reprezentare în cerc (*doughnut*) a ponderii tokenilor pe categorii de evenimente și de o defalcare a apelurilor între unelte native și cele provenite din servere MCP. Scopul acestor vizualizări agregate este de a oferi utilizatorului, dintr-o singură privire, răspunsul la întrebările esențiale legate de propria utilizare: cât a costat o sesiune, unde s-au consumat de fapt tokenii (intrare, ieșire, rezultate ale uneltelor, gândirea modelului), cât de aproape de saturarea contextului a ajuns interacțiunea și ce unelte domina activitatea agentului. Astfel, în loc de a inspecta fișierele de sesiune brute, utilizatorul își poate urmări

tiparul de consum și ajusta configurația (model, unelte, prag de compactare) pe baza unor metrice comparabile între sesiuni.

Sincronizarea folosește SSE pe o singură conexiune persistentă multiplexată; detecția modificărilor compară metadatele de fișier la 1.5s, iar reîncărcările sunt spațiate la 250ms. Un panou inferior afișează ultimele 1000 de linii din jurnalul aplicației cu auto-derulare inteligentă. Când tracing-ul Langfuse este activ, un buton dedicat deschide proiectul în interfața externă, fără a expune cheile API către browser.

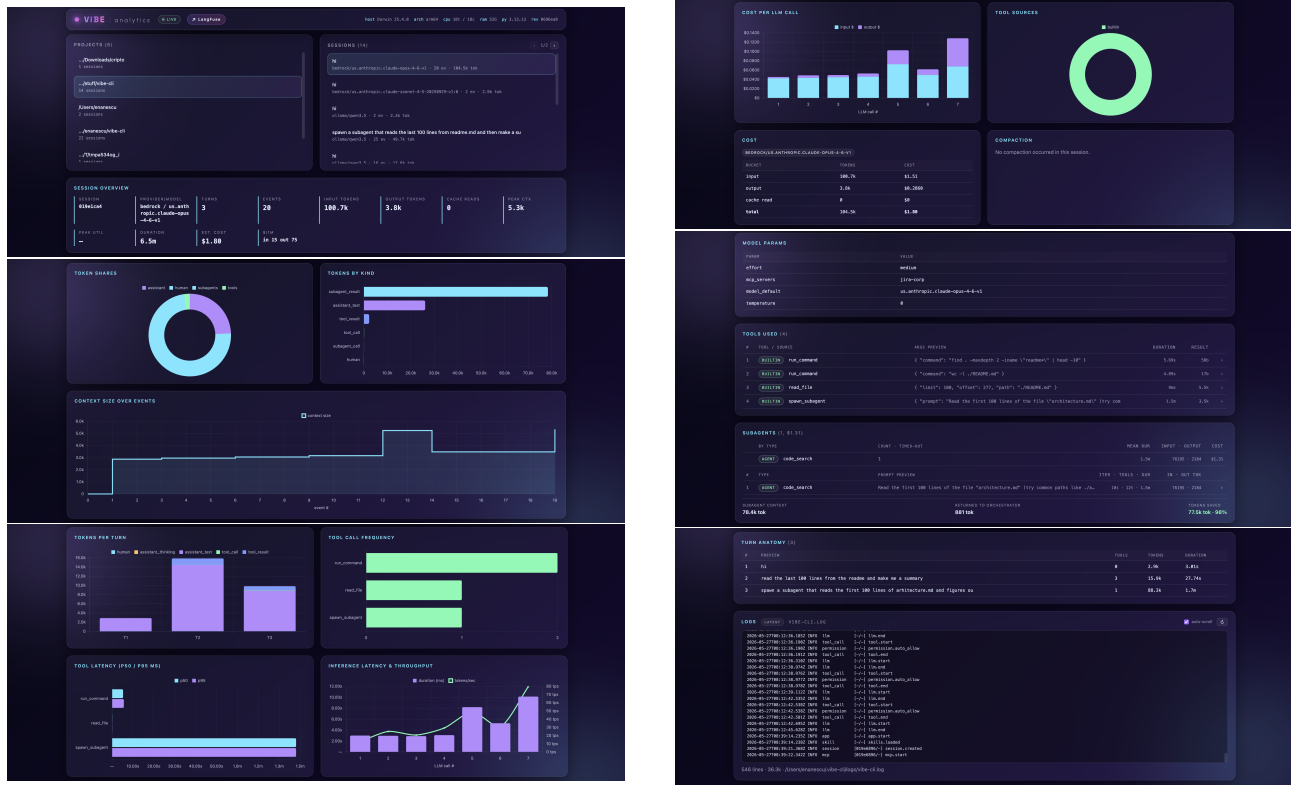


Figura 2.21: Imagini cu interfața web a aplicației

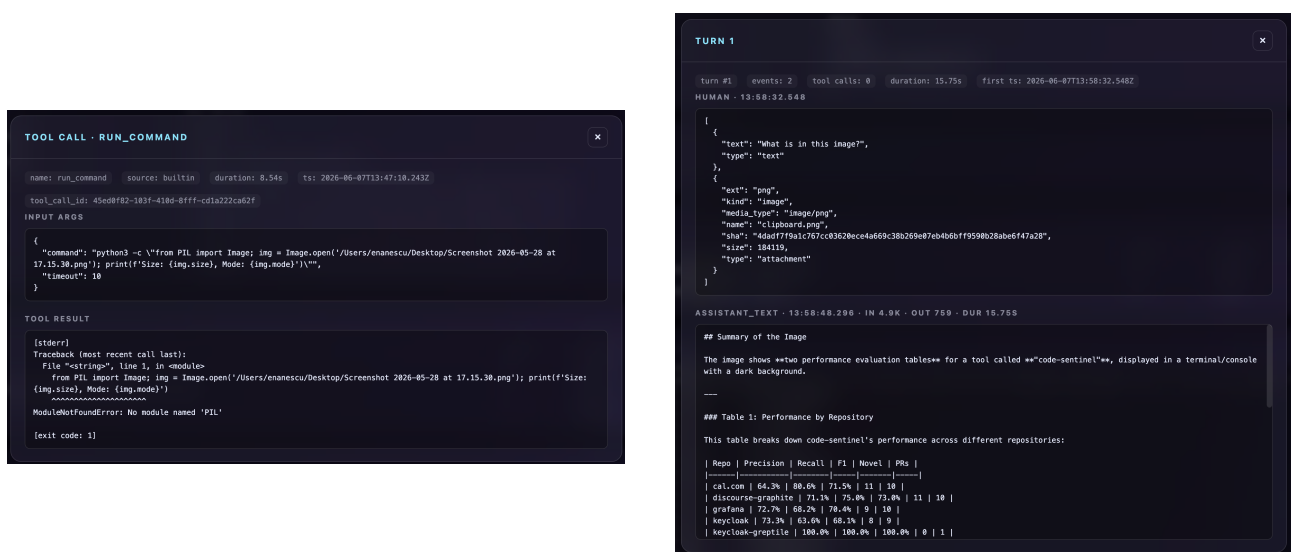
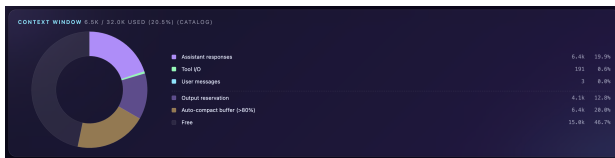


Figura 2.22: Detalii ale panoului web: apelarea uneltelor, și reconstrucția interacțiunilor



COST		
BEDROCK/US.ANTHROPIC.CLAUDE-OPUS-4-6-V1		
BUCKET	TOKENS	COST
input	10.8k	\$0.1625
output	1.2k	\$0.0919
cache read	0	\$0
total	12.1k	\$0.2543

Figura 2.23: Detalii ale panoului web: evoluția contextului și costul estimat

Scopul panoului web este acela de a aduce transparența pentru un produs care are acces la sistemul utilizatorului. Astfel, toate acțiunile sunt contorizate și toate costurile sunt explicate, spre deosebire de restul aplicațiilor disponibile pe piață care ascund aceste numere.

Capitolul 3

REZULTATE EXPERIMENTALE

Având în vedere finalizarea implementării, am condus un set de teste în timpul utilizării asistentului de programare cu diferite variații ale configurației pentru extragerea unui mod optim de utilizare și strângerea de date legate de mai multe aspecte ale folosirii care influențează performanțele aplicației.

Data fiind natura nedeterministă a răspunsurilor modelelor de limbaj, schimbările de configurație precum modelul utilizat sau promptul de sistem, cât și uneltele puse la dispoziție și alte aspecte pot avea un impact consistent asupra datelor de ieșire și acțiunile întreprinse de agent.

3.1 Poziționarea față de concurența

La finalul implementării, rezultatul este comparabil din punct de vedere al funcționalităților dar și al performanței cu aplicațiile existente pe piață în momentul de față.

Calitățile ce diferențiază abordarea noastră de restul competiției sunt: integrarea ușoară cu modele OSS ce pot rula local pe mașina utilizatorului, scăpând de dependența de un furnizor de modele de limbaj și de costurile asociate lor, transparenta interacțiunii cu agentul prin panoul web și integrarea cu Langfuse dar și prin opțiunile exhaustive de configurație prin care utilizatorul poate controla majoritatea funcționalităților vitale.

Pentru a susține cantitativ aceasta diferențiere, în tabelul 3.1 am rezumat principalele caracteristici ale celor mai populare unelte agentice de programare pentru terminal (*Claude Code* [42] dezvoltată de Anthropic, *Codex CLI* [40] dezvoltată de OpenAI și *Copilot CLI* [92] dezvoltată de GitHub) în raport cu soluția propusă.

În tabelul 3.1, se observă mai multe diferențe între implementarea realizată și industrie, în special independența față de un furnizor unic de modele, integrarea cu modele locale OSS, panoul web propriu și tracing-ul nativ cu Langfuse, poziționează soluția propusă într-un punct distinct față de concurența, oferind în același timp un set comparabil de capabilități agentice (subagenți, MCP, sandboxing, configurabilitate avansată).

Tabelul 3.1: Comparație între soluția dezvoltată și principalele unelte agentice de programare pentru terminal disponibile pe piață

Aplicație	Furnizori și model de cost	Puncte forte	Puncte slabe (raportat la soluția propusă)
Claude Code (Anthropic [42]) Sursa închisă, proprietara	Doar modele Anthropic Claude (Amazon Web Services Bedrock și Vertex AI doar în ediția enterprise). Abonament Pro/Max sau cheie API cu plata per-token.	• Maturitate și integrare cu modelele Claude • Extensibilitate prin MCP, subagenți, skills, hooks și extensii • Permisuni granulare și sandboxing pe macOS/Linux/Windows • Suprafețe multiple (terminal, VS Code, JetBrains, web)	• Cod sursa închis, dependență totală de un singur furnizor • Fără suport documentat pentru modele OSS locale • Cost obligatoriu (abonament sau per-token), fără regim gratuit • Fără panou web propriu pentru transparenta; doar consola Anthropic de facturare și comanda pentru context în aplicație. • Tracing nativ minim (suport parțial OpenTelemetry [93])
Codex CLI (GitHub [40]) Licență Apache-2.0	Doar modele OpenAI. Inclus în plan ChatGPT Plus/Pro/Business sau cheie API cu plata per-token.	• Singură soluție comercială cu cod sursa complet deschis (Apache-2.0) • Implementare în Rust, performanță ridicată și executabil unic • Suport pentru MCP, subagenți, skills, reguli, interceptări și sandboxing • Autentificare directă cu un cont ChatGPT existent	• <i>Furnizor unic</i>: doar modele OpenAI, fără rutare alternativă documentată • Fără suport pentru modele OSS locale (Ollama, llama.cpp) • Cost obligatoriu (abonament ChatGPT sau cheie API) • Documentație minimă privind transparenta utilizării (tokeni, cost, istoric) • Fără integrare cu Langfuse sau alt backend de tracing
Copilot CLI (GitHub Docs [92]) Sursa închisă, proprietara	Modele găzduite de GitHub (Claude, GPT); rutare opțională către puncte de acces OpenAI-compatibile (Ollama, vLLM, Azure OpenAI [94], Anthropic) prin variabile de mediu. Abonament Copilot Pro/Business/Enterprise + credite consumate per interacțiune.	• Singură soluție comercială cu rutare explicită către Ollama și puncte de acces locale • Configurabilitate avansată: agenți per utilizator/repo/organizație, MCP, skills, hooks, memorie persistentă • Integrare nativă cu fluxurile de lucru GitHub (PR-uri, tichete, context de repo) • Comenzi de inspecție a sesiunii (/usage, /context, /compact)	• Cod sursa închis, abonament Copilot obligatoriu • Preț opac (credite consumate per interacțiune, fără corespondență publică la cost) • Status de previzualizare publică • Fără panou web sau interfață grafică; analiza disponibilă numai prin comenzi în terminal • Fără integrare cu Langfuse sau OpenTelemetry [93]
VIBE-CLI (soluția propusă) (GitHub [95]) Licență MIT	OpenAI, Anthropic, Google Gemini și Ollama , comutabile pe loc prin /model. Aplicație gratuită; cost zero pentru rulare locală cu Ollama, sau exclusiv tariful furnizorului ales în regim cloud.	• Suport nativ multi-provider, inclusiv modele OSS locale prin Ollama, fără cheie API • Panou web Flask local, read-only, cu metrice per sesiune (tokeni pe categorii, evoluția contextului, cost estimat, latența uneltelor cu p_{50}/p_{95}, debit, compactări, reconstrucția interacțiunii) și Integrare nativă Langfuse prin apeluri inverse LangChain, cu <i>etichetare</i> per subagent și evaluări automate • Suport multi-modal (imagini, PDF, fișiere text și documente), condiționat de capabilitățile modelului selectat • Subagenți tipizați paraleli (general, code_search, web_research) cu model selectabil per tip • Configurație YAML exhaustivă: furnizori, MCP, subagenți, skills, <i>filtre</i> deterministe, permisiuni <i>allow/deny</i>, sandboxing Docker, compactare și tracing	• Maturitate redusă față de produsele furnizorilor mari, ecosistem în dezvoltare • Fără extensie nativă de Integrated Development Environment (IDE) (utilizare exclusiv din terminal) • Suport oficial limitat la macOS și Linux (fără instalare nativă pe Windows) • Curba de învățare mai abruptă pentru configurarea avansată din config.yml

În continuare, în tabelul 3.2 prezentăm o comparație cu cei mai relevanți asistenți de programare OSS din ecosistem (*OpenHands*, *Cline* și *Aider*), care împart cu soluția propusă avantajul codului sursa deschis și al independenței față de un furnizor unic de modele. Menționăm ca

OpenClaw, deși este cel mai popular proiect OSS din tabelul 1.5, nu a fost inclus în această comparație deoarece este poziționat ca asistent personal multi-canal (mesagerie, voce, automatizări), nu ca asistent dedicat dezvoltării software.

Tabelul 3.2: Comparație între soluția propusă și principalii asistenți de programare OSS din ecosistem

Aplicație	Furnizori și model de cost	Puncte forte	Puncte slabe (raportat la soluția propusă)
OpenHands (GitHub [76]) Licență MIT	Agnostic față de furnizor (Claude, GPT, Ollama și altele). Aplicație gratuită; cost limitat la tariful furnizorului ales.	• Sandboxing robust în containere Docker • Interfață grafică web și cli disponibile în paralel • SDK Python pentru construirea de agenți personalizați • Suport Kubernetes și scenarii enterprise • Integrări Slack, Jira, Linear	• Cerințe de resurse ridicate, Docker obligatoriu pentru fluxul principal • Configurare inițială complexă, curba de învățare abruptă • Fără integrare nativă Langfuse pentru tracing și evaluări automate • Fără panou de transparență cost și tokeni la nivel de sesiune • Fără suport explicit pentru subagenți tipizați cu model dedicat per tip
Cline (GitHub [79]) Licență Apache 2.0	Agnostic față de furnizor, cu rutare prin OpenRouter [96] (200+ modele: Anthropic, OpenAI, Ollama). Aplicație gratuită; cost limitat la tariful modelului ales.	• Extensie nativă pentru VS Code și extensie JetBrains • SDK Node.js și cli neinteractiv pentru CI/CD • Echipe multi-agent și agenți programați • Suport MCP și extensii • Acces la peste 200 de modele prin OpenRouter	• Consum ridicat de tokeni datorită contextului mare încărcat permanent • Fără sandboxing nativ incorporat • Dependent de ecosistemul VS Code/JetBrains, fără aplicație autonomă • Fără panou web propriu de transparență tokeni și cost • Fără integrare Langfuse sau alt backend de tracing
Aider (GitHub [83]) Licență Apache 2.0	Agnostic față de furnizor (Claude, GPT, DeepSeek [97], Ollama). Aplicație gratuită; cost limitat la tariful modelului ales.	• Mapare automată a codului sursă (<i>repo map</i>) pentru context relevant • Integrare Git nativă cu commit-uri automate per modificare • Suport pentru peste 100 de limbaje de programare • Linting și testare automată cu reîncercare la eșec • Suport explicit pentru modele locale	• Fără sandboxing • Capabilități agentice limitate, fără subagenți tipizați paraleli • Fără suport MCP pentru extinderea uneltelor • Fără panou web propriu de transparență • Fără integrare Langfuse pentru tracing și evaluări automate

Pentru a sintetiza poziționarea *vibe-cli* față de întregul ansamblu al concurenței (atât unelte comerciale cât și OSS), în continuare listam succint funcționalitățile pe care soluția propusă le preia de la celelalte aplicații (demonstrând caracterul de înlocuitor complet) și, simetric, funcționalitățile prezente la concurența dar neacoperite direct în implementarea curentă, clasificate în trei categorii: funcționalități în afara scopului proiectului, funcționalități care există indirect prin mecanisme deja prezente și funcționalități total absente.

3.1.1 Funcționalități prezente în concordanta cu aplicațiile concurențe proprietare și OSS:

1. Suport multi-provider;
2. Modele OSS locale prin Ollama;
3. Suport MCP;
4. Subagenți specializați paraleli;
5. Skills;
6. Interceptări per tura și *filtre* deterministe;
7. Permisuni de aprobare sau respingere per unealta;
8. Sandboxing opțional în container Docker;

- | | |
|--|--|
| 9. Comenzi cu autocompletare; | 13. Citire, scriere și editare de fișiere; |
| 10. Sesiuni persistente cu posibilitatea de continuare a conversației; | 14. Execuție de comenzi shell; |
| 11. Compactare automată a istoricului conversației; | 15. Persistarea preferințelor utilizatorului; |
| 12. Căutare web integrată; | 16. Intrări multi-modale (imagini, PDF, fișiere text și documente) când modelul selectat le suporta; |

3.1.2 Funcționalități absente față de aplicațiile concurente:

1. Existente indirect, prin mecanisme deja prezente:

- (a) Extensia pentru mediile de dezvoltare este acoperita prin terminalul integrat al oricărui Integrated Development Environment (IDE) sau sistem de operare;
- (b) Integrarea cu GitHub pentru PR-uri sau tichetele GitHub este realizata prin servere MCP sau invocarea utilitarului `gh`;
- (c) Rutare prin agregatoare externe de tip OpenRouter este ușor extensibila prin editarea `PROVIDER_REGISTRY` din codul sursa;
- (d) Nu exista un SDK extern pentru terți, dar modulele Python pot fi importate direct;
- (e) Suport pentru operațiuni de commit Git per modificare nu este suportat nativ dar este realizabil prin skills și prin execuție de comenzi shell;
- (f) Maparea codului sursa de tip *repo map* e acoperita parțial de subagentul `code_search`.

2. În afara scopului proiectului:

- | | |
|--|--|
| (a) Aplicație nativa desktop, mobila sau extensii IDE; | (e) Sandbox în cloud și execuție la distanță; |
| (b) Control prin vorbire; | (f) Suport Kubernetes și orchestrare enterprise; |
| (c) Suport multi-canal (Slack, Discord, WhatsApp); | (g) Magazin propriu de extensii. |
| (d) Artefacte interactive; | |

3. Absente:

- | | |
|---------------------------------|---|
| (a) Instalare nativa pe Windows | (b) Execuție de acțiuni la intervale predefinite sau la momente declarate de timp |
|---------------------------------|---|

Așadar, prin acoperirea functionalitatilor centrale ale concurenței la care se adaugă caracterul open-source multi-provider, panoul web local și integrarea Langfuse, subagenții configurabili și capabilitățile de personalizare, `vibe-cli` se poziționează ca un înlocuitor funcțional complet pentru sarcinile uzuale de programare asistata din terminal, în timp ce diferențele în raport cu concurența țin în principal de suprafețele de utilizare (extensii IDE, aplicații desktop, canale de mesagerie) și de maturitatea ecosistemului din jurul aplicației.

3.2 Influenta lungimii și a exactității prompt-ului asupra performanței. De ce un prompt prea exact nu garantează rezultate mai bune

Unul dintre experimentele rulate a fost evaluarea performanțelor agentului în sarcini de revizuire de PR pe 6 proiecte open-source scrite în limbaje de programare diferite cu două tipuri de prompt de sistem:

1. Un prompt sumar, de tipul "Review these PRs, output your findings in this format". Promptul complet este disponibil [aici](#) [98].
2. Un prompt detaliat, de aproximativ 300 de linii care încearcă să maximizeze găsirea tuturor problemelor și îndeamnă agentul să își dea silința să găsească doar probleme reale și să evite problemele triviale. Promptul complet este disponibil [aici](#) [99]

Experimentul s-a desfășurat în următoarele condiții:

- Baza de date pe care o folosim conține 50 de revizuri umane considerate ideale. Dorim să testăm dacă agentul nostru reușește să găsească toate problemele pe care le-au găsit inginerii umani în aceste proiecte.
- Baza de date conține o listă cu fiecare revizuire umană, linia la care acesta a adăugat comentariul dar și întregul proiect de la momentul producerii revizurii pentru context.
- A fost folosită o arhitectură ce pornește mai multe procese paralele ce rulează agentul cu acces complet la unelte și subagenți, pentru a procesa mai repede cele 50 de revizuri.
- La finalul procesului de revizuire, un alt model de limbaj mai mic se uita la revizuirile umane și la cele generate de agent și verifica dacă acestea relatează aceleași probleme.
- Derivăm scorurile de precizie, recunoaștere și F1 din aceste metrici.
- Modelele folosite au fost Claude Opus 4.7 pentru revizuirea efectivă și Claude Haiku 4.5 pentru agentul judecător și deduplicare.

Pentru a cuantifica performanța agentului, fiecare comentariu produs în raport cu setul de referință uman este clasificat în una dintre următoarele categorii:

- **True Positive (TP):** o problemă semnalată de agent care corespunde unei probleme identificate și în revizuirea umană de referință.
- **False Positive (FP):** o problemă semnalată de agent căreia nu i se asociază nicio observație din revizuirea umană (zgomot sau alertă greșită).
- **False Negative (FN):** o problemă prezentă în revizuirea umană pe care agentul nu a reușit să o identifice (omisiune).
- **True Negative (TN):** o linie de cod pentru care nici agentul, nici recenzentul uman nu au formulat o observație; în contextul revizurii de PR acest număr este dominant și dificil de definit riguros, motiv pentru care nu intervine în metricile raportate.

Pornind de la aceste categorii, definim *precizia* ca fracțiunea de alerte corecte din totalul alertelor emise de agent, *recunoașterea* (engl. *recall*) ca fracțiunea de probleme reale efectiv

detectate, iar scorul F_1 ca media armonică a celor două, oferind o singură valoare scalară ce penalizează dezechilibrul între precizie și recunoaștere:

$$\begin{aligned} \text{Precizie} &= \frac{TP}{TP + FP}, \\ \text{Recunoaștere} &= \frac{TP}{TP + FN}, \\ F_1 &= 2 \cdot \frac{\text{Precizie} \cdot \text{Recunoaștere}}{\text{Precizie} + \text{Recunoaștere}} = \frac{2 TP}{2 TP + FP + FN}, \end{aligned} \quad (3.1)$$

unde Precizie, Recunoaștere, $F_1 \in [0, 1]$, iar valori apropiate de 1 corespund unei performanțe superioare. Un agent care raportează foarte puține probleme poate atinge o precizie ridicată cu o recunoaștere scăzută, în timp ce un agent care raportează agresiv obține recunoaștere mare în detrimentul preciziei; scorul F_1 surprinde acest compromis într-o singură metrică.

Problemele noi sunt probleme pe care modelul judecător le-a considerat ca fiind valide în ciuda faptului că nu se regăseau prin revizuirile umane.

Rezultatele experimentului pot fi regăsite în tabelele 3.3 și 3.4.

Prompt	Proiect	Precizie	Recunoaștere	F1	Probleme noi	PRs
Detaliat	cal.com	64.3%	80.6%	71.5%	11	10
Detaliat	discourse-graphite	71.1%	75.0%	73.0%	11	10
Detaliat	grafana	72.7%	68.2%	70.4%	9	10
Detaliat	keycloak	73.3%	63.6%	68.1%	8	9
Detaliat	keycloak-greptile	100.0%	100.0%	100.0%	0	1
Detaliat	sentry	81.2%	52.6%	63.9%	3	6
Detaliat	sentry-greptile	37.0%	75.0%	49.6%	1	4
Detaliat	TOTAL	66.5%	70.6%	68.5%	43	50

(a) Metrice per proiect

Severitate	Precizie	Recunoaștere	F1	TP	FP	FN	Probleme noi
critical	58.8%	88.9%	70.8%	8	7	1	2
high	75.7%	80.5%	78.0%	33	18	8	23
medium	59.5%	72.3%	65.3%	34	34	13	16
low	67.6%	53.8%	60.0%	21	11	18	2

(b) Metrice pe nivele de severitate

Tabelul 3.3: Rezultatele pentru prompt-ul detaliat: metrice per proiect și pe nivele de severitate

Prompt	Proiect	Precizie	Recunoaștere	F1	Probleme noi	PRs
Simplu	cal.com	18.2%	83.9%	30.0%	1	10
Simplu	discourse-graphite	24.4%	100.0%	39.2%	2	10
Simplu	grafana	16.2%	90.0%	27.5%	0	9
Simplu	keycloak	25.8%	95.5%	40.7%	2	9
Simplu	keycloak-greptile	33.3%	100.0%	50.0%	0	1
Simplu	sentry	27.8%	73.7%	40.3%	1	6
Simplu	sentry-greptile	15.1%	83.3%	25.5%	1	4
Simplu	TOTAL	20.9%	88.8%	33.8%	7	49

(a) Metrici per proiect

Severitate	Precizie	Recunoaștere	F1	TP	FP	FN	Probleme noi
critical	17.3%	100.0%	29.5%	9	43	0	0
high	25.0%	95.0%	39.6%	38	120	2	2
medium	22.6%	89.4%	36.1%	42	161	5	5
low	16.3%	78.9%	27.0%	30	154	8	0

(b) Metrici pe nivele de severitate

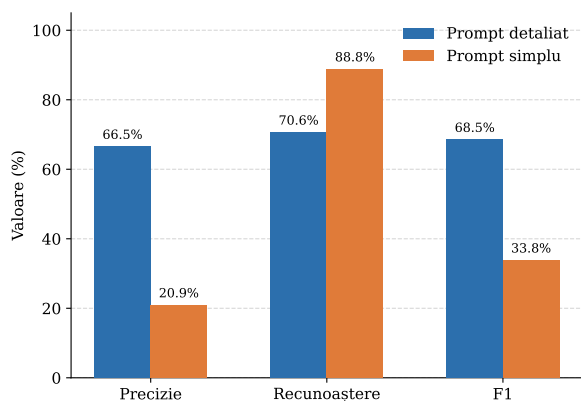
Tabelul 3.4: Rezultatele pentru prompt-ul simplu: metrici per proiect și pe nivele de severitate

Analizând datele din tabelele 3.3 și 3.4, putem observa un rezultat neașteptat. Având în vedere tehnica "Chain of Thought" [12], ne-am fi așteptat ca un agent căruia i se spune pas cu pas cum trebuie să opereze să aibă performanțe mai bune decât unul care este lăsat liber.

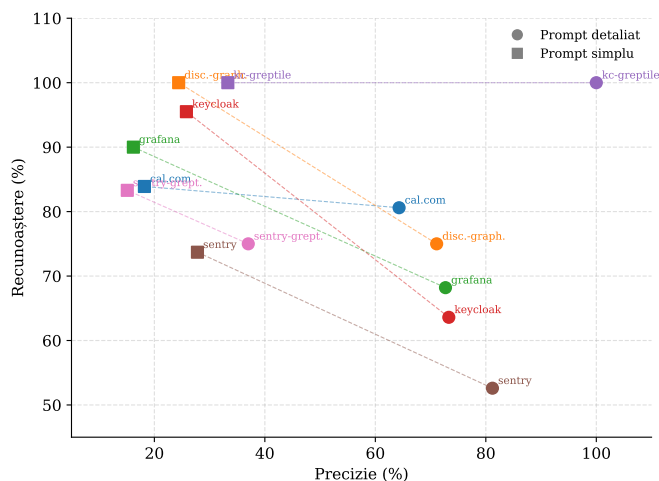
Totuși, uitându-ne mai atent la datele de tip "False Positive", putem intui care este cauza acestor rezultate. Din cauza restricțiilor impuse de prompt, modelul este reticent în a spune sau cerceta probleme din afara ariei pe care o acoperă prompt-ul pe sistem. Acesta încearcă să maximizeze precizia și scorul F1, astfel face revizuiți despre care este sigur ca sunt corecte, având astfel un scor F1 mare și o precizie buna, dar o recunoaștere mai mică decât prompt-ul simplu. Numeric, acest tip de prompt a produs în total 70 de rezultate fals pozitive.

Pe de alta parte, folosind prompt-ul simplu, agentul a fost lăsat să scrie orice problemă, verificata sau nu. Acesta a scris orice pseudo-probleme (reală sau nu) care ar fi putut fi posibilă. Astfel, semnalând mai multe probleme (fictive sau nu) a reușit să acopere o suprafață mai mare de probleme, iar probabilitatea ca o problemă reală să se afle prin cele sugerate crește. Acest lucru este clar uitându-ne la secțiunea de "False Positives" unde, în loc de 70 de revizuiți ca și în cazul prompt-ului detaliat, aici avem în total 478 de comentarii marcate ca fals pozitive.

Acest studiu reflecta o problemă dominantă în lumea IA: alegerea între precizie și recunoaștere. Cum realizăm un agent care are o recunoaștere perfectă și identifica corect toate instanțele ce trebuiesc recunoscute fără a returna rezultate nedorite. În domenii sensibile precum cel medical sau militar, acest subiect este foarte dezbătut, deoarece în general dacă încercăm să mărim recunoașterea, precizia va scădea. O alternativă este folosirea scorului F1, dar nu ajută când doar putem modifica doar prompt-ul.



(a) Metrici globale: prompt detaliat vs. simplu



(b) Precizie vs. recunoaștere per proiect

Figura 3.1: Comparație vizuală între prompt-ul detaliat și cel simplu

În experimentul de față, se ridică aceeași problemă: preferam un agent care găsește toate problemele critice dintr-un sistem, dar acest detaliu este ascuns printr-o sumedenie de raportări de severitate mică sau false, ajungând să fie ignorat de ingineri? Sau ne bazăm ca inginerul este judecătorul final și ca agentul este doar o unealta, în final tot inginerul ajungând să facă o revizuire completă, bazându-se cât mai puțin pe IA din cauza ca poate rata probleme majore?

Răspunsul este cel mai probabil, undeva la mijloc. Proiectantul sistemului agentic trebuie să găsească un echilibru bun între precizie și recunoaștere astfel încât produsul să aibă sens.

Agentul depistează probleme pe care revizorii umani le-au omis. Un rezultat semnificativ al experimentului, parțial eclipsat de discuția precizie-recunoaștere, este că agentul a identificat probleme *valide* care nu se regăseau în nicio revizuire umană de referință: **43 de probleme noi** cu prompt-ul detaliat și 7 cu cel simplu, validate ulterior de modelul judecător. Raportat la cele 50 de PR-uri analizate, aceasta înseamnă că agentul a adus în medie aproape o problemă nouă per PR față de setul uman. Constatarea este relevantă în practică: chiar dacă agentul nu înlocuiește complet revizuirea umană, el acționează ca un al doilea verficator care identifică omisiuni reale. Valoarea unei astfel de unelte nu se limitează la acoperirea problemelor umane cunoscute, ci și la lărgirea suprafeței de inspecție dincolo de acestea.

3.3 Contextul în mediul agentic și compactare. Consecințele metodelor de compactare

Compactarea este un procedeu necesar, după cum am menționat și în capitolele anterioare. Dar, în practică, acesta nu este un procedeu perfect și are un număr de consecințe, în funcție de metoda, sau setul de metode de compactare alese. Din utilizarea soluției agentice dezvoltate cu mai multe combinații de metode de compactare, am extras un set de repercusiuni ale folosirii acestor metode, pe categorii, dar și un set de metode recomandat pentru performanțe optime în raport cu integritatea informațiilor din fereastra de context.

Cea mai utilă clasificare în funcție de momentul în care rulează compactarea (imediat după apelul uneltelor sau când depășește o limită), ci pe natura pierderii pe care o introduc în context. Stadiile care taie literal blocuri din rezultatele uneltelor (`tool_truncate` și `ref_substitute`) pierd date întregi, dar lasă o urmă vizibilă în conversație: un marker de trunchiere sau o referință `@artifact:<sha>`. Stadiile care parafrazează prin LLM (`tool_summarize` pe rezultatele individuale ale uneltelor, `bulk` cu strategie `summarize` sau `hybrid` pe întregul istoric) păstrează

întinderea conversației, dar pierde nuanțe fără nicio urmă vizibilă pentru agent. Toate trei stadiile care rescriu rezultatele uneltelor (`tool_truncate`, `tool_summarize`, `ref_substitute`) operează asupra aceluiași slot proiectat și sunt declarate mutual exclusive în procesul `vibe-cli`; oricare configurație validă conține cel mult unul dintre ele.

3.3.1 Metode care taie date din context

Spre exemplu, `tool_truncate` păstrează un cap și o coadă peste `max_tool_tokens` (implicit $800 + 400$ la prag de 2000) și externalizează restul în artefacte externe. `ref_substitute` merge mai departe: lasă doar un extras de `excerpt_tokens` și o referință. În ambele cazuri, în sesiunile testate, agentul ajunge în scurt timp să **reapeleze unealta cu argumente mai înguste** (`grep` după un identificator, `read_file` pe un interval mai mic) pentru a recupera fragmentul dispărut, în loc să cheme `fetch_artifact`. Costul real al acestor stadii nu este, deci, pierderea de informație care este recuperabilă, ci tura suplimentară pe care agentul o consumă cautându-o, vizibilă în creșterea numărului de apeluri de unealtă per sarcină.

3.3.2 Metode care parafrazează date din context

În contrast, `tool_summarize` este, în cazul de succes, strict superior trunchierii pure: rulează în două faze (o trunchiere sincronă, apoi o sumarizare LLM asincronă care înlocuiește forma trunchiată când termină), iar în caz de eșec rămâne la forma trunchiată. `bulk` face același tip de transformare la nivelul întregului istoric. Diferența față de stadiile care taie este că **pierderea nu lasă urma**: rezultatul are aceeași formă textuală, dar afirmații cantitative (numere de linii, nume de fișiere, identificatori) pot dispărea silențios. Efectul practic observat este pierderea de *recunoaștere*: agentul nu mai cere date pe care nu le mai are, pentru că rezumatul îi spune că le-a văzut deja. Când modelul de compactare este unul mai mic decât cel principal (configurabil via `preferences.compaction_model`), același mecanism produce ocazional afirmații incorecte în rezumat, care apoi se propaga ca premisa în turele următoare.

3.3.3 Metode auxiliare

Restul stadiilor operează cu alte principii și pot însoți orice combinație de mai sus. `format_compress` reformatează JSON-ul fără apel LLM și fără pierdere semantică: în practică nu am observat efecte negative și poate fi ținut activat necondiționat. `dedupe` elimină apeluri identice și rezultate similare peste un prag (implicit 0,92); pragurile sub 0,90 au șters în sesiunile noastre rezultate de teste cu output similar dar semnificație diferită, ceea ce a mascat regresii reale. `importance_prune` este robust cu setările implicite: pierderea evenimentelor `assistant_thinking` reduce vizibil contextul fără consecințe observabile, iar păstrarea doar a ultimului `tool_result` per tură funcționează pentru că rezultatele intermediare sunt subsuimate de cel final.

3.3.4 Secvența de metode de compactare recomandată

Configurația care a oferit cel mai bun compromis între reamintire și ocupare a ferestrei de context, în experimentele rulate, are următoarea formă:

1. `format_compress` activat aduce câștig de spațiu garantat, pierdere zero;
2. `tool_summarize` ca singur reprezentant al grupei mutual exclusive de rescriere a rezultatelor uneltelor (`tool_truncate` rămâne fallback-ul sau sincron, deci este implicit prezent);

3. **dedupe** cu pragul implicit 0,92, poziționat după **format_compress** pentru ca normalizarea formatării crește rata de detecție;
4. **importance_prune** cu configurarea implicită;
5. **bulk** cu strategia **hybrid** și **min_turns_kept** = 4, suficient pentru a acoperi o iterație completă cerere-investigare-răspuns.

Pentru sarcini scurte de tip întrebare și răspuns un proces complet dezactivat este perfect funcțional. Pentru sarcini cu citiri exhaustive de cod, înlocuirea **tool_summarize** cu **ref_substitute** reduce semnificativ ocuparea, cu condiția ca promptul de sistem să instruiască explicit agentul când să apeleze **fetch_artifact**. În modul **tool_persistence: true**, în care întregul istoric al uneltelor trăiește între interacțiuni, prezenta a cel puțin unui stadiu ce rulează imediat devine obligatorie: în lipsa lui, fereastra se poate umple într-un număr relativ mic de ture cu volum normal de unelte, înainte ca pragul **threshold** să apuce să declanșeze **bulk**.

3.4 Interacțiunea dintre modele, context și unelte. Modele optime pentru sarcini de programare

Nu toate modelele au capacități similare când vine vorba despre sarcini de dezvoltare software. Pe lângă necesitatea funcționalităților de gândire rațională adâncă, acestea trebuie să fie antrenate pentru a apela funcții în mod eficient și pentru a respecta dogme de programare precum SDD sau TDD.

Pentru a cuantiza exact cât de importanta este alegerea unui tip de model pentru sarcini de dezvoltare software, am încercat rularea programului cu mai multe tipuri de modele, cu capacități diferite și mărime (ca număr de parametri) diferită.

Modelele folosite, împreună cu caracteristicile lor relevante, sunt prezentate în tabelul 3.5. Selecția acoperă atât modele accesate prin API (familiile Claude și Gemini), cât și modele open-weight rulate local prin **ollama**, într-o plajă largă de dimensiuni (de la 1.5 miliarde la 32 miliarde de parametri) și specializări (uz general, raționament, generare de cod).

Model	Tip	Parametrii	Mărime pe disc	Multi-modal
Claude Opus 4.7	LLM raționament	nepublicat	nepublicat	da
Claude Sonnet 4.6	LLM uz general	nepublicat	nepublicat	da
Claude Haiku 4.5	LLM compact, uz general	nepublicat	nepublicat	da
Gemini 2.5 Flash	LLM uz general	nepublicat	nepublicat	da
deepseek-r1:32b	LLM, raționament	32 B	19 GB	nu
gemma4:26b	LLM, uz general	26 B	17 GB	da
phi4:14b	LLM, uz general	14 B	9.1 GB	nu
llama3.2:3b	Small Language Model, uz general	3 B	2.0 GB	nu
devstral-small-2	LLM, generare cod	24 B	15 GB	nu
qwen3-coder:30b	LLM, generare cod	30 B	18 GB	nu
qwen2.5-coder:7b	LLM, generare cod	7 B	4.7 GB	nu
qwen2.5-coder:1.5b	Small Language Model, generare cod	1.5 B	986 MB	nu

Tabelul 3.5: Modelele testate: tip, număr de parametri, mărime pe disc și suport multi-modal.

Pentru rezultate comparabile, toate modelele au avut de realizat aceleași operații. Testul a constat din 6 sarcini de intensitate diferită, executate pe un proiect real. Fiecare etapa a avut propriul mod de recompensare: de la metrici de similaritate, la LLM-as-judge, la teste ascunse.

Toate acestea testează fie exactitatea cu care agentul a relatat faptele, uneltele sau skill-urile pe care le-a apelat și dacă în final operațiunea trece testul. Aceste metrice sunt fie procentaje, binare (admis sau respins) sau un scor în funcție de cât de mult a fost efectuată rezolvarea și logica cu care modelul a gândit.

Configurația testului împreună cu sarcinile pe care agenții le-au avut de rezolvat pot fi consultate aici [100]

3.4.1 Rezultate

Rezultatele agregate ale rulărilor sunt prezentate în tabelul 3.6, sortate după scorul mediu obținut pe ansamblul cazurilor de test.

#	Furnizor	Model	Param. (Mld.)	Scor	Durata	Cost (USD)	Tokeni totali
1	anthropic	claude-opus-4-7	—	0.9062	7 min 27 s	13.40	803 585
2	anthropic	claude-sonnet-4-6	—	0.8743	10 min 4 s	0.06	17 606
3	anthropic	claude-haiku-4-5	—	0.8081	7 min 0 s	0.02	17 361
4	gemini	gemini-2.5-flash	—	0.7871	3 min 6 s	0.16	400 841
5	ollama	devstral-small-2	24	0.7051	14 min 22 s	0.00	14 620
6	ollama	gemma4	26	0.6803	9 min 49 s	0.00	6 241
7	gemini	gemini-2.5-pro	—	0.6431	4 min 14 s	0.67	435 404
8	ollama	qwen3-coder	30	0.6256	4 min 33 s	0.00	13 926
9	ollama	gemma4	8	0.3803	3 min 26 s	0.00	6 852
10	ollama	qwen3.5	9.7	0.2963	7 min 28 s	0.00	15 031
11	ollama	deepseek-r1	32	0.0747	4 min 15 s	0.00	1 484
12	ollama	llama3.2:	3	0.0590	35 s	0.00	5 380
13	ollama	qwen2.5-coder	7	0.0442	32 s	0.00	834
14	ollama	qwen2.5-coder	1.5	0.0298	21 s	0.00	828
15	ollama	phi4	14	0.0000	Eroare	0.00	—
16	gemini	gemini-2.0-flash	—	0.0000	Eroare	0.00	—

Tabelul 3.6: Clasamentul modelelor după scor mediu, durata totala de rulare și cost. Modelele rulate local prin `ollama` au cost zero. Modelele proprietare nu au datele referitoare la mărimea modelului publice

3.4.2 Analiza rezultatelor

Interpretând rezultatele obținute putem face mai multe observații:

1. Numărul de parametri nu este întotdeauna direct proporțional cu capabilitățile. O buna parte din capacitatea de rezolvare a problemelor în dezvoltarea software este extrasă din specializarea modelelor pe sarcini de acest tip. De aceea modelele care au fost pregătite pentru așa ceva au performanțe mai bune în ciuda numărului de parametri. Un viitor experiment ar fi încercarea modelelor de la Anthropic care aici au dominat tabela pe alte domenii de activitate;

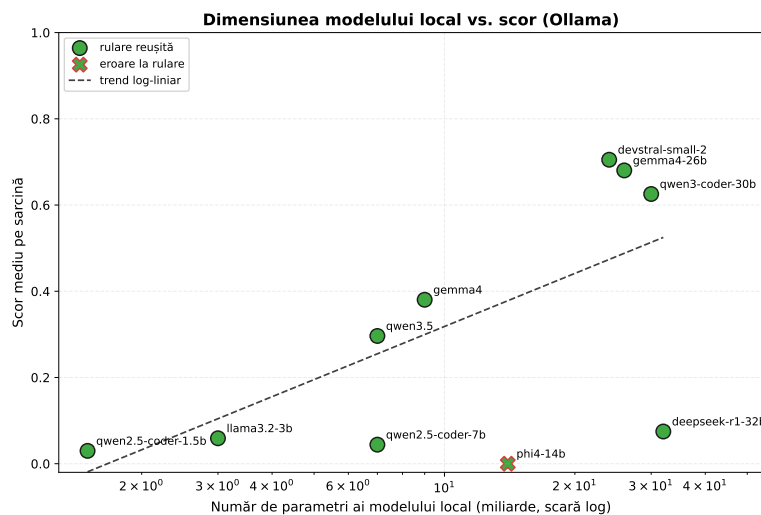


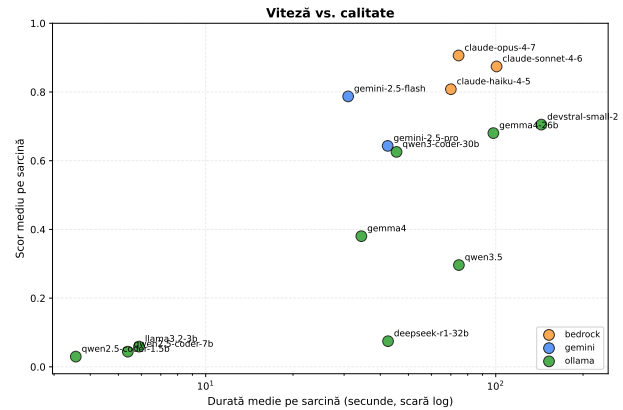
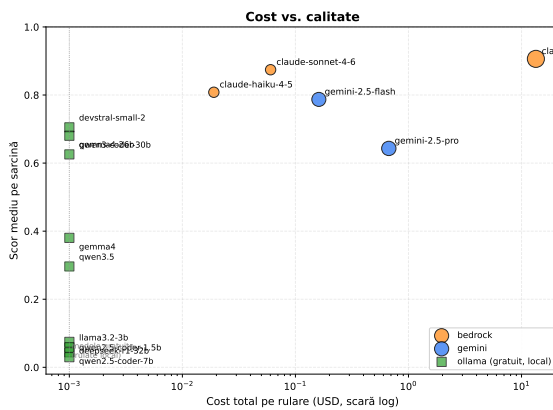
Figura 3.2: Număr de parametri ai modelelor locale

2. Putem vedea influența competențelor de gândire adâncă asupra numărului de tokeni folosiți (și automat a costului). Prin tehnica "Chain of Thought" [12] aceste modele reușesc să își formeze o logică superioară care ajută la rezolvarea problemelor complexe. Dar, acest procedeu vine cu un cost. Iar în cazul Opus 4.7, acest cost este mare. Pentru 6 sarcini de programare acesta a reușit să producă prin gândire și cercetarea fișierelor 800.000 de tokeni, în valoare de aproximativ 13 dolari — de **≈45 de ori** mai mulți tokeni față de Sonnet 4.6 (17.606) și Haiku 4.5 (17.361), diferență atribuibilă aproape în întregime gândirii extinse (*chain-of-thought*). Un cost enorm comparat cu celelalte modele, chiar și de la Anthropic;
3. Modelele prea mici fie nu suportă apelarea de unelte din cauza depășirii ferestrei de context, fie nu sunt antrenate pentru așa ceva și pur și simplu întorc o eroare. Gemini 2.0 Flash și Phi4 pur și simplu întorc erori care denotă incompatibilitatea cu sarcini agentice. Restul modelelor cu scoruri sub 10%, pur și simplu au halucinat rezolvarea problemei și nu au apelat nici o unealtă cu adevărat. Unele dintre ele, chiar dacă ar fi reușit să apeleze uneltele, nu ar fi putut să ducă o conversație prea complexă din cauza ferestrei mici de context;
4. Modelele locale necesită hardware performant pentru a rezolva probleme reale. Deși rezultatele sunt bune și comparabile cu cele ale modelelor de top, la o fracțiune din numărul de parametrii, timpul de execuție încă crează o experiență a utilizatorului neoptimal pe sisteme neperformante;
5. Variante ale modelelor antrenate special pentru sarcini de dezvoltare software arată o diferență semnificativă în performanță, precum Qwen3.5 și Qwen3-Coder.

Anomalii în rezultate:

1. Modele care dețin în teorie capacități de gândire (Sonnet, DeepSeek R1) nu au exercitat în timpul testului aceste capacități la potențialul maxim, ceea ce sugerează setări implicite de efort ale modelului reduse;
2. Modelul Haiku a avut o performanță surprinzătoare având în vedere că acesta este considerat cel mai slab din flota Anthropic. Acest lucru se datorează în mare discrepanței de concurență din cadrul testului. Nu am avut acces la modele de aceeași generație OpenAI

sau Gemini cu care să putem face o comparație corectă, iar experimentul urmarea în principal relația dintre cost și performanță;



(a) Cost vs. calitate (scară logaritmică pentru cost).

(b) Viteză vs. calitate (durată medie pe sarcină, scară logaritmică).

3. Modele cu mai putini parametrii au avut un scor mult mai mic decât modele cu număr mai mic de parametrii. Spre exemplu DeepSeek R1 nu a fost antrenat pentru apelarea uneltelor și deși acesta știa teoretic cum ar rezolva problema, nu a apelat nici o unealta. Pe scurt, pur și simplu a fabricat o narațiune, o poveste despre cum ar rezolva el problema, iar la final a declarat-o rezolvata.

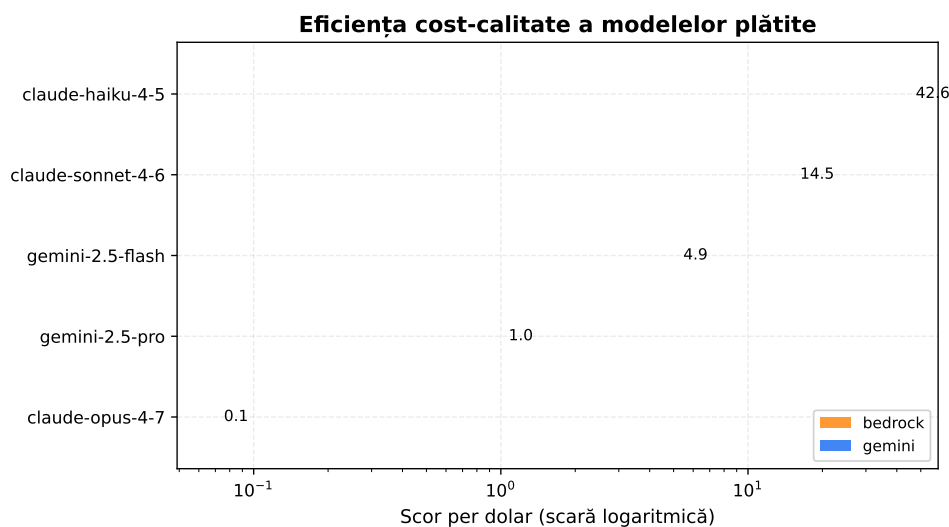


Figura 3.4: Eficiența cost-calitate: scorul mediu obținut per dolar cheltuit (scară logaritmică). Modelele locale Ollama nu sunt incluse deoarece au cost zero.

Recomandări extrase din acest experiment:

1. Pentru rularea modelelor locale, este necesar un hardware bun. Timpul de inferență poate fi mai crescut decât la modelele furnizorilor, sau mai mic pentru unele modele;
2. Costul modelelor crește odată cu capabilitățile de gândire și exponențial cu mărimea contextului;
3. În general, modelele prea mici (mai mici de 20 de miliarde de parametrii) nu reușesc să se descurce în sarcini de programare. Pot răspunde la întrebări punctuale, dar nu pot lucra într-un mediu agentic. Risc mare de halucinații;

4. Modelele neantrenate pe sarcini agentice (apel de unelte, gândire) nu au performanțe în mediul agentic pentru dezvoltarea software. Risc mare de halucinații;
5. Modelele mai mici ale furnizorilor de modele de limbaj sunt mai avantajoase din punct de vedere al raportului calitate-preț;
6. Gândirea crește și ea timpul de inferență. O comparație buna ar fi între Devstral Small 2 și Gemma4:26b. Deși Devstral este un model mai mic, din numărul de tokeni utilizați putem deduce ca a avut o gândire mai adâncă ce a dus la timpul ridicat. În același timp, Gemma a avut rezultate similare cu un timp mai scăzut de inferență (deși Gemma are și o arhitectura diferită, mai performantă);
7. Modele open-source recomandate: Gemma4 pentru echilibrul dintre scor și performanță, Devstral pentru scor, cu dezavantajul timpului mare de inferență și Qwen3-Coder pentru timpul foarte scurt de inferență și scor bun;
8. Din punctul de vedere al modelelor proprietare, Sonnet și Haiku au avut performanțe foarte bune, dar și Gemini 2.5 Flash a surprins cu performanțele sale (având în vedere ca nu mai este de ultima generație în familia Gemini);
9. Folosirea limbii engleza pentru discuțiile cu modelele de limbaj. Acestea, în special cele de mărimi mici sunt antrenate preponderent folosind limba engleza. Performanta acestora poate scădea folosind limbi diferite;
10. Similar, pentru limbajul de programare, unele modele vor avea performanțe mai ridicate pe anumite limbaje (precum Python) și mai slabe în sarcini pe alte limbaje, în funcție de datele de antrenament.

3.4.3 Modelele open-source: evaluarea valorii și a potențialului de utilizare

La finalul analizei rezultatelor, putem rezumă performanța modelelor open-source într-un singur cuvânt: surprinzătoare și într-un sens negativ, dar și într-unul pozitiv.

A fost neașteptat ca 6 din cele 16 modele să aibă scor aproximativ 0 în cadrul experimentului, dar nu improbabil. Era de așteptat ca modelele sub 10 miliarde de parametri să întâmpine probleme, motiv pentru care le-am și ales. Dar cele între 10 și 20 de miliarde, dar și DeepSeek R1 au surprins prin neajunsurile lor.

Pe de alta parte am fost foarte plăcut impresionat de rezultatele Devstral Small 2, Qwen3-Coder și Gemma 4. Acestea au un potențial imens, mai ales cele două din urma care oferă un timp de inferență foarte bun. Gemma4 este interesant și deoarece are o arhitectura speciala de tipul Mixture Of Experts.

În final, consider ca folosirea modelelor open-source locale pentru sarcini de dezvoltare software (considerând deținerea unui sistem performant) este motivata de costurile inexistente, controlul asupra datelor, timpul scurt de inferență dar și performanțele bune (care pe măsură ce se dezvoltă vor crește).

Capitolul 4

DEZVOLTĂRI VIITOARE

Finalizând analiza post-implementare, împreună cu experimentele rulate, putem extrage un set de potențiale îmbunătățiri pe care le putem aduce aplicației în viitor.

4.1 Automatizări, integrări cu alte programe

O zona potențială de dezvoltare este suportul pentru rularea de sarcini agentice la un interval de timp, sau la un moment în timp predefinit. Astfel de implementări sunt deja regăsite în unelte precum Claude Code, prin comanda `/loop`. Aceasta funcționalitate permite rularea unui prompt automat la un moment determinat. Un exemplu bun pentru aceasta utilizare în domeniul dezvoltării software ar fi un prompt de tipul:

```
Realizează un rezumat al celor mai importante tichete sau PR-uri apărute  
în ultimele 24 de ore pe GitHub, și analizează canalele de Slack, Discord,  
Teams la care ai acces. Fa-mi un rezumat complet cu ceea ce găsești.
```

Acest prompt ar fi foarte util pentru ingineri pentru a fi la curent cu starea proiectelor la care lucrează și la deciziile ce se întâmplă cât timp alți colegi lucrează în alte fusuri orare.

4.2 Integrarea cu mediile de dezvoltare

Deși interfața de tip CLI este foarte versatila, compatibilitatea cu mediile de dezvoltare aduce o granularitate mai mare asupra interacțiunii cu agentul. Acesta poate sugera schimbări restrânse (linie cu linie), pe care utilizatorul le poate accepta sau respinge individual.

4.3 Suport nativ pentru Windows

Sistemul de operare Windows nu este la fel de prietenos pentru dezvoltatorii software precum macOS sau Linux din cauza terminalului (și a comenzilor) diferite și a structurii fișierelor, motiv pentru care implementarea inițială are suport doar pentru

cele doua sisteme de operare listate anterior.

Acestea fiind spuse, o bună completare viitoare ar fi suportul nativ pentru Windows, în ciuda diferențelor dintre sistemele de operare, pentru a deschide calea către o nouă audiență de utilizatori.

4.4 Construirea unui graf de cunoștințe pentru optimizarea înțelegerii proiectelor

O metoda inovativa care are potențialul de a îmbunătăți contextul unui agent din punctul de vedere al fabricării legăturilor între componentele unui proiect, dar mai ales din punctul

de vedere al numărului de tokeni de intrare și al reducerii apelului de unelte este aceea de a construi grafuri ale legăturilor între diverse părți ale proiectului.

Una dintre soluțiile pentru implementarea acestei facilități este utilizarea Graphify [101]. Acest program construiește grafurile necesare înțelegerii proiectului și folosește modele de limbaj pentru a detalia grafurile din punct de vedere semantic. Acest lucru reduce numărul de apeluri la unelte pentru citirea fișierelor, îmbunătățind astfel utilizarea contextului și sporind cunoașterea amănunțită a proiectul pe care se lucrează.

Integrarea este una simplă, graful se construiește o dată (și ulterior automat dacă apar schimbări în cod) iar aplicațiile agentice precum Claude Code, pot folosi integrarea oficială sau pot folosi un server MCP pentru a interacționa cu acesta.

4.5 Suport pentru metode de compactare și sandboxing avansate și SDK pentru agenți

Tehnicile avansate de compactare bazate pe similaritatea semantică a frazelor ce urmează a fi șters din cauza criteriilor de similaritate sau din cauza categorisirii ca fiind neimportante sunt o completare bună la setul de metode deja existente. Aceste tehnici pot cimanta consistența acestora dacă sunt considerate informații vitale, excluzându-le de la metodele de compactare uzuale.

Din punctul de vedere al securității, containerele Docker nu sunt impenetrabile. O îmbunătățire ce poate fi adusă în zona de sandboxing ar fi introducerea suportului pentru platforme cloud specializate în acest domeniu precum E2B.

O ultimă schimbare care ar ușura munca potențialilor contributori la dezvoltarea programului ar fi dezvoltarea unui SDK prin care aceștia ar putea apela cu ușurință metodele programului aducând astfel un plus de valoare.

CONCLUZII

Lucrarea a urmărit proiectarea și realizarea unui asistent inteligent de programare, agnostic față de furnizorul de modele de limbaj, transparent și extensibil, capabil să ruleze fie local prin Ollama, fie în cloud prin furnizorii principali. Obiectivul a fost atins prin aplicația `vibe-cli`, livrată cu un set complet de capabilități agentice (sandboxing, permisiuni granulare, subagenți tipizați complet configurabili, suport MCP pe toate metodele de comunicare uzuale, intrări multi-modale și proces configurabil de compactare) și cu tehnologii proprii de observabilitate pe două niveluri, ce expune utilizatorului metrice detaliate despre consumul de tokeni, cost, latență și comportamentul agentului.

Ca și contribuții proprii, am combinat un proces de compactare configurabil în întregime din YAML și extensibil prin subclase, un sistem de filtre deterministe peste apelurile de unelte (*read-before-write* cu hash SHA pentru integritatea contextului), un mecanism de detectare automată a capabilităților multi-modale impus de arhitectura agnostică și un panou web local care expune metrice ascunse în produsele comerciale studiate. În același timp, am reluat conștient soluții deja validate de proiecte precum Claude Code, Aider sau Codex CLI acolo unde acestea s-au dovedit cele mai potrivite (tampon nativ de terminal, graf LangGraph cu salvarea stărilor precedente, confirmarea umană a acțiunilor și tracing extern Langfuse).

De asemenea am implementat și rulat o suită de experimentale proprii: evaluarea de revizuire de PR pe 50 de referințe umane din 6 proiecte OSS, investigarea stadiilor de compactare și compararea a 16 modele pe 6 sarcini de programare — din care am extras un set de recomandări transferabile la ecosistemul mai larg de unelte similare.

În ansamblu, lucrarea arată că o soluție agentică ce rulează local, agnostică de furnizor și transparentă în utilizarea modelelor de limbaj este nu doar fezabilă, ci poate ocupa un loc distinct în peisajul actual, dominat de produse strâns legate de ecosistemele furnizorilor mari. Direcțiile de evoluție identificate în capitolul *Dezvoltări viitoare* (integrări IDE, suport Windows, grafuri de cunoștințe, sandboxing avansat, SDK pentru terți) pot transforma prototipul actual într-o platformă matură, capabilă să acompanieze evoluția rapidă a domeniului.

BIBLIOGRAFIE

- [1] JetBrains. Pycharm: The python ide for professional developers. disponibil la: <https://www.jetbrains.com/pycharm/>, 2026. accesat la data: 08.06.2026.
- [2] Microsoft. Visual studio code. disponibil la: <https://code.visualstudio.com>, 2026. accesat la data: 12.04.2026.
- [3] LangChain. Langchain: Applications that can reason. powered by langchain. disponibil la: <https://www.langchain.com>, 2026. accesat la data: 02.05.2026.
- [4] LangChain. Langgraph: Build resilient language agents as graphs. disponibil la: <https://www.langchain.com/langgraph>, 2026. accesat la data: 21.04.2026.
- [5] Google. Agent development kit (adk) documentation. disponibil la: <https://google.github.io/adk-docs/>, 2026. accesat la data: 12.04.2026.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- [7] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models.
- [8] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer.
- [9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. version: 1.
- [10] Ruixiang Zhang, Richard He Bai, Huangjie Zheng, Navdeep Jaitly, Ronan Collobert, and Yizhe Zhang. Embarrassingly simple self-distillation improves code generation.
- [11] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc.
- [12] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. 35:24824–24837.
- [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models.
- [14] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. ReWOO: Decoupling reasoning from observations for efficient augmented language models.
- [15] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. 12:157–173.
- [16] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks?
- [17] Anthropic. Build with claude: Context windows. disponibil la: <https://platform.claude.com/docs/en/build-with-claude/context-windows>, 2026. accesat la data: 17.05.2026.
- [18] Anthropic. Claude.ai settings: Usage dashboard. disponibil la: <https://claude.ai/settings/usage>, 2026. accesat la data: 14.04.2026.

- [19] Letta. Guide to context engineering. disponibil la: <https://www.letta.com/blog/guide-to-context-engineering>, 2026. accesat la data: 30.05.2026.
- [20] Google. Google. disponibil la: <https://www.google.com>, 2026. accesat la data: 10.04.2026.
- [21] Anthropic. Anthropic. disponibil la: <https://www.anthropic.com>, 2026. accesat la data: 19.05.2026.
- [22] OpenAI. Openai. disponibil la: <https://openai.com>, 2026. accesat la data: 19.05.2026.
- [23] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain.
- [24] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. 86(11):2278–2324.
- [25] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. 9:1735–1780.
- [26] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners.
- [27] Yichun Xu, Navjot K. Khaira, and Tejinder Singh. KV cache optimization strategies for scalable and efficient LLM inference. version: 1.
- [28] Weian Mao, Xi Lin, Wei Huang, Yuxin Xie, Tianfu Fu, Bohan Zhuang, Song Han, and Yukang Chen. TriAttention: Efficient long reasoning with trigonometric KV compression.
- [29] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding.
- [30] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding.
- [31] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate.
- [32] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. 36:10088–10115.
- [33] Zhongmou He, Yee Man Choi, Kexun Zhang, Ivan Bercovich, Jiabao Ji, Junting Zhou, Dejia Xu, Aidan Zhang, Yixiao Zeng, and Lei Li. HARDTESTGEN: A high-quality RL verifier generation pipeline for LLM algorithmic coding.
- [34] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, Hyojung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidya-dhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques.
- [35] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines.
- [36] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822. Curran Associates, Inc.
- [37] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634. Association for Computational Linguistics.
- [38] Kiran Vodrahalli, Santiago Ontanon, Nilesch Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, Rohan Anil, Ethan Dyer, Siamak Shakeri, Roopali Vij, Harsh Mehta, Vinay Ramasesh, Quoc Le, Ed Chi, Yifeng Lu, Orhan Firat, Angeliki Lazaridou, Jean-Baptiste Lespiau, Nithya Attaluri, and Kate Olszewska. Michelangelo: Long context evaluations beyond haystacks via latent structure queries. version: 2.

- [39] Dongfang Li, Zixuan Liu, Gang Lin, Baotian Hu, and Min Zhang. LycheeCluster: Efficient long-context inference with structure-aware chunking and hierarchical KV indexing.
- [40] OpenAI. Codex cli: Lightweight coding agent that runs in the terminal, github. disponibil la: <https://github.com/openai/codex>, 2026. accesat la data: 28.04.2026.
- [41] Google. Gemini cli: An open-source ai agent, github. disponibil la: <https://github.com/google-gemini/gemini-cli>, 2026. accesat la data: 20.04.2026.
- [42] Anthropic. Claude code documentation. disponibil la: <https://docs.anthropic.com/en/docs/claude-code/overview>, 2026. accesat la data: 04.04.2026.
- [43] Anthropic. Anthropic api: Models overview. disponibil la: <https://platform.claude.com/docs/en/about-claude/models/overview>, 2026. accesat la data: 08.05.2026.
- [44] OpenAI. Openai api: Models documentation. disponibil la: <https://developers.openai.com/api/docs/models>, 2026. accesat la data: 25.05.2026.
- [45] Google. Gemini api: Models documentation. disponibil la: <https://ai.google.dev/gemini-api/docs/models>, 2026. accesat la data: 15.04.2026.
- [46] DeepSeek. Deepseek api: Pricing. disponibil la: https://api-docs.deepseek.com/quick_start/pricing, 2026. accesat la data: 04.04.2026.
- [47] Alibaba Cloud. Qwen model family on hugging face. disponibil la: <https://huggingface.co/Qwen>, 2026. accesat la data: 13.04.2026.
- [48] Mistral AI. Mistral ai models overview. disponibil la: https://docs.mistral.ai/getting-started/models/models_overview/, 2026. accesat la data: 28.04.2026.
- [49] Python Software Foundation. Python programming language official website. disponibil la: <https://www.python.org>, 2026. accesat la data: 12.05.2026.
- [50] Google Cloud. Vertex ai: Build, deploy, and scale ml models. disponibil la: <https://cloud.google.com/vertex-ai>, 2026. accesat la data: 09.06.2026.
- [51] BerriAI. Litellm: Call all llm apis using the openai format. disponibil la: <https://www.litellm.ai>, 2026. accesat la data: 19.05.2026.
- [52] OpenAI. Openai python sdk, github. disponibil la: <https://github.com/openai/openai-python>, 2026. accesat la data: 14.05.2026.
- [53] Langfuse. Langfuse: Open-source llm engineering platform. disponibil la: <https://langfuse.com>, 2026. accesat la data: 04.05.2026.
- [54] Textualize. Textual: Tui framework for python documentation. disponibil la: <https://textual.textualize.io>, 2026. accesat la data: 11.04.2026.
- [55] Jonathan Slenders. Python prompt toolkit documentation. disponibil la: <https://python-prompt-toolkit.readthedocs.io>, 2026. accesat la data: 25.04.2026.
- [56] Will McGugan. Rich: Python library for rich text and beautiful formatting in the terminal. disponibil la: <https://rich.readthedocs.io>, 2026. accesat la data: 09.04.2026.
- [57] Urwid Project. Urwid: Console user interface library for python. disponibil la: <https://urwid.org>, 2026. accesat la data: 30.04.2026.
- [58] Anthropic. Claude.ai: Web application. disponibil la: <https://claude.ai>, 2026. accesat la data: 18.04.2026.
- [59] Slack Technologies. Slack: Channel-based messaging platform. disponibil la: <https://slack.com>, 2026. accesat la data: 30.04.2026.
- [60] Google. Gemini: Ai assistant by google. disponibil la: <https://gemini.google.com>, 2026. accesat la data: 04.04.2026.
- [61] Google. Android operating system. disponibil la: <https://www.android.com>, 2026. accesat la data: 13.04.2026.
- [62] Apple Inc. ios operating system. disponibil la: <https://www.apple.com/ios/>, 2026. accesat la data: 06.05.2026.
- [63] Google. Notebooklm: Ai research assistant. disponibil la: <https://notebooklm.google.com>, 2026. accesat la data: 08.06.2026.

- [64] Google. Firebase studio: Cloud-based development environment. disponibil la: <https://firebase.google.com/products/studio>, 2026. accesat la data: 30.04.2026.
- [65] OpenAI. Chatgpt. disponibil la: <https://chatgpt.com>, 2026. accesat la data: 29.04.2026.
- [66] GitHub Inc. Github: Where the world builds software. disponibil la: <https://github.com>, 2026. accesat la data: 28.05.2026.
- [67] Microsoft. Microsoft corporation. disponibil la: <https://www.microsoft.com>, 2026. accesat la data: 29.04.2026.
- [68] GitHub Inc. Github copilot: Ai pair programmer. disponibil la: <https://github.com/features/copilot>, 2026. accesat la data: 06.05.2026.
- [69] JetBrains. JetBrains: Developer tools. disponibil la: <https://www.jetbrains.com>, 2026. accesat la data: 22.04.2026.
- [70] OpenClaw Contributors. Openclaw: Open-source ai assistant platform, github. disponibil la: <https://github.com/openclaw/openclaw>, 2026. accesat la data: 14.04.2026.
- [71] Meta Platforms. Whatsapp messenger. disponibil la: <https://www.whatsapp.com>, 2026. accesat la data: 30.04.2026.
- [72] Telegram FZ-LLC. Telegram messenger. disponibil la: <https://telegram.org>, 2026. accesat la data: 08.05.2026.
- [73] Discord Inc. Discord: Group chat platform. disponibil la: <https://discord.com>, 2026. accesat la data: 05.04.2026.
- [74] Docker Inc. Docker: Accelerated container application development. disponibil la: <https://www.docker.com>, 2026. accesat la data: 16.05.2026.
- [75] Apple Inc. macos operating system. disponibil la: <https://www.apple.com/macos/>, 2026. accesat la data: 29.05.2026.
- [76] OpenHands Contributors. Openhands: Code less, make more, github. disponibil la: <https://github.com/OpenHands/OpenHands>, 2026. accesat la data: 21.04.2026.
- [77] Cloud Native Computing Foundation. Kubernetes: Production-grade container orchestration. disponibil la: <https://kubernetes.io>, 2026. accesat la data: 15.05.2026.
- [78] Atlassian. Jira: Issue & project tracking software. disponibil la: <https://www.atlassian.com/software/jira>, 2026. accesat la data: 21.04.2026.
- [79] Cline Contributors. Cline: Autonomous coding agent for vs code, github. disponibil la: <https://github.com/cline/cline>, 2026. accesat la data: 14.05.2026.
- [80] OpenJS Foundation. Node.js: Javascript runtime built on chrome's v8 engine. disponibil la: <https://nodejs.org>, 2026. accesat la data: 29.05.2026.
- [81] Block Inc. Goose: Local, extensible, open-source ai agent, github. disponibil la: <https://github.com/aaif-goose/goose>, 2026. accesat la data: 25.05.2026.
- [82] Rust Foundation. Rust programming language official website. disponibil la: <https://www.rust-lang.org>, 2026. accesat la data: 08.04.2026.
- [83] Aider Contributors. Aider: Ai pair programming in your terminal, github. disponibil la: <https://github.com/Aider-AI/aider>, 2026. accesat la data: 06.05.2026.
- [84] Git Project. Git: Distributed version control system. disponibil la: <https://git-scm.com>, 2026. accesat la data: 29.04.2026.
- [85] Amazon Web Services. Amazon bedrock: The easiest way to build and scale generative ai applications. disponibil la: <https://aws.amazon.com/bedrock/>, 2026. accesat la data: 06.05.2026.
- [86] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.
- [87] Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque, and Md Rizwan Parvez. A survey on agentic security: Applications, threats and defenses.

- [88] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In Leon Derczynski, Jekaterina Novikova, and Muhao Chen, editors, *Proceedings of the The First Workshop on LLM Security (LLMSEC)*, pages 101–114. Association for Computational Linguistics.
- [89] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI feedback.
- [90] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In Yansong Feng and Els Lefever, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445. Association for Computational Linguistics.
- [91] Yue Liu, Hongcheng Gao, Shengfang Zhai, Yufei He, Jun Xia, Zhengyu Hu, Yulin Chen, Xihong Yang, Jiaheng Zhang, Stan Z. Li, Hui Xiong, and Bryan Hooi. GuardReasoner: Towards reasoning-based LLM safeguards.
- [92] GitHub Inc. About copilot cli documentation. disponibil la: <https://docs.github.com/en/copilot/concepts/agents/about-copilot-cli>, 2026. accesat la data: 12.04.2026.
- [93] Cloud Native Computing Foundation. Opentelemetry: High-quality, ubiquitous, and portable telemetry. disponibil la: <https://opentelemetry.io>, 2026. accesat la data: 10.06.2026.
- [94] Microsoft. Azure openai service documentation. disponibil la: <https://azure.microsoft.com/en-us/products/ai-services/openai-service>, 2026. accesat la data: 02.05.2026.
- [95] Eduard-Daniel Nănescu. Vibe-cli: Coding assistant proposed in this thesis, github. disponibil la: <https://github.com/Moshulika/vibe-cli>, 2026. accesat la data: 01.04.2026.
- [96] OpenRouter. Openrouter: A unified interface for llms. disponibil la: <https://openrouter.ai>, 2026. accesat la data: 11.04.2026.
- [97] DeepSeek. Deepseek. disponibil la: <https://www.deepseek.com>, 2026. accesat la data: 18.05.2026.
- [98] Eduard-Daniel Nănescu. Prompt sumar pentru revizuirea pr-urilor, github gist. disponibil la: <https://gist.github.com/Moshulika/b3f1d85fa269bae8c06b216d8e2e8776>, 2026. accesat la data: 24.05.2026.
- [99] Eduard-Daniel Nănescu. Prompt complet pentru revizuirea pr-urilor, github gist. disponibil la: <https://gist.github.com/Moshulika/6f2859da2291ec38544f6deaa209613a>, 2026. accesat la data: 26.04.2026.
- [100] Eduard-Daniel Nănescu. Configurația testului de evaluare agentică, github gist. disponibil la: <https://gist.github.com/Moshulika/7c5aecdc01c3e7756c1b5584f39613cd>, 2026. accesat la data: 09.06.2026.
- [101] Graphify. Graphify: Code knowledge graphs for ai agents. disponibil la: <https://graphify.net>, 2026. accesat la data: 19.05.2026.

ANEXE

Anexa 1: config/config.yml

```
1 providers:
2   ollama:
3     base_url: ${OLLAMA_URL}
4     models:
5       - qwen3.5
6       - gemma4
7       - deepseek-r1:32b
8       - gemma4:26b
9       - phi4:14b
10      - llama3.2:3b
11      - devstral-small-2
12      - qwen3-coder:30b
13      - qwen2.5-coder:7b
14      - qwen2.5-coder:1.5b
15      temperature: 0.6
16   openai:
17     api_key: ${OPENAI_API_KEY}
18     models:
19       - gpt-4.1
20       - gpt-5.5
21       - gpt-5.4
22     temperature: 0
23   claude:
24     api_key: ${ANTHROPIC_API_KEY}
25     models:
26       - claude-opus-4-7
27       - claude-sonnet-4-6
28       - claude-haiku-4-5-20251001
29     temperature: 0
30   bedrock:
31     api_key: ${BEDROCK_API_KEY}
32     region: us-west-2
33     models:
34       - us.anthropic.claude-sonnet-4-6
35       - us.anthropic.claude-opus-4-7
36       - us.anthropic.claude-haiku-4-5-20251001-v1:0
37     temperature: 0
38   gemini:
39     api_key: ${GOOGLE_API_KEY}
40     models:
41       - gemini-2.5-flash
42       - gemini-2.5-pro
43       - gemini-2.0-flash
44     temperature: 0
45   multimodal:
46     pdf:
47       small_threshold_pages: 15
48       render_dpi: 144
49       max_pages_per_call: 20
50     enforce_session_model: true
51   trigger:
52     threshold: 0.80
53     min_turns_kept: 4
54   pipeline:
55     - tool_truncate:
56       enabled: false
57       max_tool_tokens: 2000
58       head_tail: [800, 400]
59     - tool_summarize:
60       enabled: false
61       max_tool_tokens: 2000
62       head_tail: [800, 400]
63       timeout_s: 30
64       max_summary_tokens: 500
65     - model: null
66     - format_compress:
67       enabled: false
68     - per_tool: {}
69     - ref_substitute:
70       enabled: false
71       min_ref_tokens: 1500
72       excerpt_tokens: 200
73     - dedupe:
74       enabled: false
75       similarity: 0.92
76       backend: string
77       importance_prune:
78         enabled: false
79         keep_kinds: [human, system_prompt, assistant_text,
80           tool_result_last]
81         drop_kinds: [assistant_thinking]
82       - bulk:
83         enabled: false
84         strategy: hybrid # trim | summarize | hybrid
85         summarize:
86           model: ${COMPACTION_MODEL}
87           max_summary_tokens: 1500
88       storage:
89         backup: true
90         keep_backups: 3
91         artifacts: true
92       sandboxing:
93         enabled: false
94         image: python:3.13-slim
95         network: bridge
96         memory: 2g
97         cpus: 2
98         timeout_default: 30
99         blocked_paths:
100          - "/"
101         ambiguous_max_mb: 500
102         ambiguous_max_files: 10000
103       ui:
104         pastes:
105           threshold: 200
106           max_per_turn: 5
107         skills:
108           max_active: 10
109           vibe_md:
110             enabled: true
111             max_bytes_per_file: 8192
112           mcp:
113             max_active: 10
114             logging:
115               enabled: true
116               level: INFO
117               dir: ~/.vibe-cli/logs
118               text_max_bytes: 5242880
119               text_backups: 10
120               jsonl_retention_days: 30
121             tool_guards:
122               enabled: true
123               read_before_write:
124                 enabled: true
125                 check_freshness: true
126             subagents:
127               enabled: true
128               default_timeout_s: 120
129               max_timeout_s: 600
130               max_depth: 2
131               max_parallel: 3
132               child_context:
133                 max_tool_tokens: 4000
134                 head_tail: [800, 400]
135             types:
136               general:
137                 max_iterations: 25
138                 default_timeout_s: 180
139                 code_search:
140                   max_iterations: 15
141                   default_timeout_s: 90
142                 web_research:
143                   max_iterations: 18
144                   default_timeout_s: 120
145             tracing:
146               enabled: true
147               project: vibe-code
148               public_key: ${LANGFUSE_PUBLIC_KEY}
149               secret_key: ${LANGFUSE_SECRET_KEY}
150               host: ${LANGFUSE_HOST}
```